

**LAPORAN AKHIR PRAKTIKUM
PEMROGRAMAN MOBILE**



Oleh:

Akmallullail Sya'ban

NIM. 2310817220002

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2025**

LEMBAR PENGESAHAN
LAPORAN AKHIR PRAKTIKUM PEMROGRAMAN MOBILE

Laporan Akhir Praktikum Pemrograman Mobile ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Akhir Praktikum ini dikerjakan oleh:

Nama Praktikan : Akmallullail Sya'ban

NIM : 2310817310010

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Raka Azwar
NIM. 2210817210012

Ir. Eka Setya Wijaya, S.T., M.Kom
NIP. 198205082008011010

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI.....	3
DAFTAR GAMBAR	5
DAFTAR TABEL.....	8
MODUL 1 : ANDROID BASIC WITH KOTLIN	10
SOAL 1	10
A. Source Code.....	12
B. Output Program.....	24
C. Pembahasan	26
MODUL 2 : ANDROID LAYOUT	30
SOAL 1	30
A. Source Code.....	32
B. Output Program.....	48
C. Pembahasan	51
SOAL 2	57
A. Pembahasan	57
MODUL 3 : BUILD A SCROLLABLE LIST.....	59
SOAL 1	59
A. Source Code.....	61
B. Output Program.....	98
C. Pembahasan	103
SOAL 2	111

A. Pembahasan	111
MODUL 4 : VIEWMODEL AND DEBUGGING	112
SOAL 1	112
A. Source Code.....	112
B. Output Program.....	156
C. Pembahasan	162
SOAL 2	175
A. Pembahasan	175
MODUL 5 : CONNECT TO THE INTERNET	176
SOAL 1	176
A. Source Code.....	176
B. Output Program.....	223
C. Pembahasan	227
Tautan Git.....	247

DAFTAR GAMBAR

Gambar 1. Tampilan Awal Aplikasi Soal 1 Modul 1.....	10
Gambar 2. Tampilan Dadu Setelah Di Roll Soal 1 Modul 1	11
Gambar 3. Tampilan Roll Dadu Double Soal 1 Modul 1	12
Gambar 4. Screenshot Hasil Tampilan Awal Soal 1 Modul 1 XML	24
Gambar 5. Screenshot Hasil Tampilan Dadu Tidak Sama Soal 1 Modul 1 XML	24
Gambar 6. Screenshot Hasil Tampilan Dadu Sama Soal 1 Modul 1 XML	25
Gambar 7. Screenshot Hasil Tampilan Awal Soal 1 Modul 1 Compose	25
Gambar 8. Screenshot Hasil Tampilan Dadu Tidak Sama Soal 1 Modul 1 Compose	26
Gambar 9. Screenshot Hasil Tampilan Dadu Sama Soal 1 Modul 1 Compose	26
Gambar 10. Tampilan Awal Aplikasi Soal 1 Modul 2.....	30
Gambar 11. Tampilan Aplikasi Setelah Dijalankan Soal Modul 2.....	31
Gambar 12. Tampilan Aplikasi Setelah Dijalankan Soal Modul 2.....	31
Gambar 13. Screenshot Hasil Tampilan Awal Soal 1 Modul 2 XML	48
Gambar 14. Screenshot Hasil Tampilan Pilihan Soal 1 Modul 2 XML	49
Gambar 15. Screenshot Hasil Tampilan Setelah Dijalankan Soal 1 Modul 2 XML...	49
Gambar 16. Screenshot Hasil Tampilan Awal Soal 1 Modul 2 Compose	50
Gambar 17. Tampilan Pilihan Persentasi Tip Compose	50
Gambar 18. Screenshot Hasil Tampilan Setelah Dijalankan Soal 1 Modul 2 Compose	51
Gambar 19. Contoh UI List Soal 1 Modul 3	60
Gambar 20. Contoh UI Detail Soal 1 Modul 3	61
Gambar 21. Screenshot Hasil Tampilan Awal Soal 1 Modul 3 XML	98
Gambar 22. Screenshot Hasil Tampilan Ketika di Scroll Soal 1 Modul 3 XML.....	99
Gambar 23. Screenshot Hasil Tampilan Tombol Detail Soal 1 Modul 3 XML	99
Gambar 24. Screenshot Hasil Tampilan List (landscape) Soal 1 Modul 3 XML	100
Gambar 25. Screenshot Hasil Tampilan Detail (landscape) Soal 1 Modul 3 XML..	100
Gambar 26. Screenshot Hasil Tampilan Tombol IMDB Soal 1 Modul 3.....	100
Gambar 27. Screenshot Hasil Tampilan Awal Soal 1 Modul 3 Compose	101

Gambar 28. Screenshot Hasil Tampilan Ketika di Scroll Soal 1 Modul 3 Compose	101
Gambar 29. Screenshot Hasil Tampilan Tombol Detail Soal 1 Modul 3 Compose .	102
Gambar 30. Screenshot Hasil Tampilan List (landscape) Soal 1 Modul 3 Compose	102
Gambar 31. Screenshot Hasil Tampilan Detail (landscape) Soal 1 Modul 3 Compose	103
Gambar 32. Screenshot Hasil Tampilan Tombol IMDB Soal 1 Modul 3.....	103
Gambar 33. Screenshot Hasil Tampilan Awal Soal 1 Modul 4 XML	156
Gambar 34. Screenshot Hasil Tampilan Ketika di Scroll Soal 1 Modul 4 XML.....	156
Gambar 35. Screenshot Hasil Tampilan Tombol Detail Soal 1 Modul 4 XML	157
Gambar 36. Screenshot Hasil Tampilan List (landscape) Soal 1 Modul 4 XML	157
Gambar 37. Screenshot Hasil Tampilan Detail (landscape) Soal 1 Modul 4 XML..	157
Gambar 38. Screenshot Hasil Tampilan Tombol IMDB Soal 1 Modul 4.....	158
Gambar 39. Screenshot Hasil Tampilan Awal Soal 1 Modul 4 Compose	158
Gambar 40. Screenshot Hasil Tampilan Ketika di Scroll Soal 1 Modul 4 Compose	159
Gambar 41. Screenshot Hasil Tampilan Tombol Detail Soal 1 Modul 4 Compose .	159
Gambar 42. Screenshot Hasil Tampilan List (landscape) Soal 1 Modul 4 Compose	160
Gambar 43. Screenshot Hasil Tampilan Detail (landscape) Soal 1 Modul 4 Compose	160
Gambar 44. Screenshot Hasil Tampilan Tombol IMDB Soal 1 Modul 4.....	161
Gambar 45. Gambar Debugger Di XML Modul 4.....	161
Gambar 46. Gambar Debugger Di Jetpack Compose Modul 4	162
Gambar 47. Tampilan Aplikasi Ketika Offline Modul 5	223
Gambar 48. Tampilan Aplikasi Ketika Offline dalam Light Mode Modul 5	223
Gambar 49. Tampilan Tombol Ubah Dark/Light Mode Modul 5	224
Gambar 50. Tampilan Tombol Ubah Dark/Light Mode Modul 5	224
Gambar 51. Tampilan Aplikasi Saat Retry Ditekan dan Online dalam Light Mode Modul 5	225
Gambar 52. Tombol Detail Ditekan dalam Light Mode Modul 5	225
Gambar 53. Tampilan Aplikasi Saat Retry Ditekan dan Online dalam Dark Mode Modul 5	226

Gambar 54. Tombol Detail Ditekan dalam Dark Mode Modul 5	226
Gambar 55. Tombol TMDB Saat Ditekan Modul 5	227

DAFTAR TABEL

Tabel 1. Source Code MainActivity.kt Soal 1 Modul 1 XML	12
Tabel 2. Source Code activity_main.xml Soal 1 Modul 1 XML	15
Tabel 3. Source Code MainActivity.kt Soal 1 Modul 1 Compose.....	17
Tabel 4. Source Code AndroidManifest.xml Soal 1 Modul 1 Compose	22
Tabel 5. Source Code MainActivity.kt Soal 1 Modul 2 XML	32
Tabel 6. Source Code activity_main.xml Soal 1 Modul 2 XML	35
Tabel 7. Source Code MainActivity.kt Soal 1 Modul 2 Compose.....	39
Tabel 8. Source Code MainActivity.kt Soal 1 Modul 3 XML	61
Tabel 9. Source Code Movie.kt Soal 1 Modul 3 XML	62
Tabel 10. Source Code ListMovieAdapter.kt Soal 1 Modul 3 XML	63
Tabel 11. Source Code HomeFragment.kt Soal 1 Modul 3 XML	65
Tabel 12. Source Code DetailFragment.kt Soal 1 Modul 3 XML	69
Tabel 13. Source Code activity_main.xml Soal 1 Modul 3 XML	71
Tabel 14. Source Code fragment_detail.xml Soal 1 Modul 3 XML	71
Tabel 15. Source Code fragment_home.xml Soal 1 Modul 3 XML	74
Tabel 16. Source Code item_movie.kt Soal 1 Modul 3 XML	75
Tabel 17. Source String Data Soal 1 Modul 3 Compose	81
Tabel 18. Source String Code DataMovie.kt Soal 1 Modul 3 Compose	84
Tabel 19. Source Code ListMovie.kt Soal 1 Modul 3 Compose	85
Tabel 20. Source Code DetailActivity.kt Soal 1 Modul 3 Compose	86
Tabel 21. Source Code MainActivity.kt Soal 1 Modul 3 Compose.....	90
Tabel 22. Source Code MainActivity.kt Soal 1 Modul 4 XML	112
Tabel 23. Source Code Movie.kt Soal 1 Modul 4 XML	114
Tabel 24. Source Code ListMovieAdapter.kt Soal 1 Modul 4 XML	114
Tabel 25. Source Code HomeFragment.kt Soal 1 Modul 4 XML	117
Tabel 26. Source Code DetailFragment.kt Soal 1 Modul 4 XML	122
Tabel 27. Source Code MovieViewModel.kt Soal 1 Modul 4 XML	124
Tabel 28. Source Code MovieViewModelFactory.kt Soal 1 Modul 4 XML.....	125
Tabel 29. Source Code MyApplication.kt Soal 1 Modul 4 XML	126

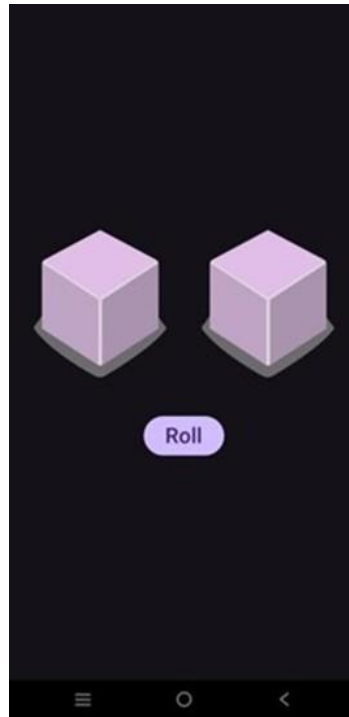
Tabel 30. Source Code activity_main.xml Soal 1 Modul 4 XML	126
Tabel 31. Source Code fragment_detail.xml Soal 1 Modul 4 XML	127
Tabel 32. Source Code fragment_home.xml Soal 1 Modul 4 XML	130
Tabel 33. Source Code item_movie.xml Soal 1 Modul 4 XML	131
Tabel 34. Source String Data Soal 1 Modul 4 Compose	136
Tabel 35. Source String Code DataMovie.kt Soal 1 Modul 4 Compose	139
Tabel 36. Source Code ListMovie.kt Soal 1 Modul 4 Compose	140
Tabel 37. Source Code DetailActivity.kt Soal 1 Modul 4 Compose	141
Tabel 38. Source Code MainActivity.kt Soal 1 Modul 4 Compose.....	145
Tabel 39. Source Code MovieViewModel.kt Soal 1 Modul 4 Compose.....	152
Tabel 40. Source Code MovieViewModelFactory.kt Soal 1 Modul 4 Compose	154
Tabel 41. Source Code AppDatabase.kt Soal 1 Modul 5 Compose.....	177
Tabel 42. Source Code MovieDao.kt Soal 1 Modul 5 Compose	178
Tabel 43. Source Code MovieEntity.kt Compose.....	179
Tabel 44. Source Code SettingManager.kt Compose	179
Tabel 45. Source Code MovieApi.kt Compose	181
Tabel 46. Source Code MovieResponse.kt Compose.....	181
Tabel 47. Source Code ListMovie.kt Compose	182
Tabel 48. Source Code RetrofitClient.kt Compose.....	182
Tabel 49. Source Code TmdbApiService.kt Compose	185
Tabel 50. Source Code MovieRepository.kt Compose.....	185
Tabel 51. Source Code DetailActivity.kt Compose	188
Tabel 52. Source Code MovieListScreen.kt Compose	193
Tabel 53. Source Code SettingScreen.kt Compose.....	207
Tabel 54. Source Code Constants.kt Compose	211
Tabel 55. Source Code Resource.kt Compose	211
Tabel 56. Source Code RetrofitClient.kt Compose.....	212
Tabel 57. Source Code MovieViewModelFactory.kt Compose	217
Tabel 58. Source Code MovieViewModelFactory.kt Compose	218
Tabel 59. Source Code MainActivity.kt Compose	219

MODUL 1 : ANDROID BASIC WITH KOTLIN

SOAL 1

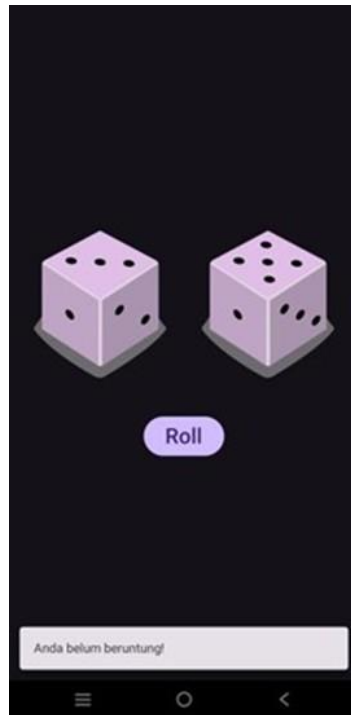
Buatlah sebuah aplikasi yang dapat menampilkan 2 (dua) buah dadu yang dapat berubah-ubah tampilannya pada saat user menekan tombol “*Roll Dice*”. Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



Gambar 1. Tampilan Awal Aplikasi Soal 1 Modul 1

2. Setelah user menekan tombol “Roll Dice” maka masing-masing dadu akan memunculkan sisi dadu masing-masing dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2 maka akan menampilkan pesan “Anda belum beruntung!” seperti dapat dilihat pada Gambar 2.



Gambar 2. Tampilan Dadu Setelah Di Roll Soal 1 Modul 1

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan “Selamat anda dapat dadu double!” seperti dapat dilihat pada Gambar 3.
4. Buatlah aplikasi tersebut menggunakan XML dan Jetpack Compose.
5. Upload aplikasi yang telah anda buat kedalam repository github ke dalam **folder Module 2 dalam bentuk project**. Jangan lupa untuk melakukan **Clean Project** sebelum mengupload pekerjaan anda pada repo.
6. Untuk gambar dadu dapat didownload pada link berikut:
https://drive.google.com/u/0/uc?id=147HT2IIH5qin3z5ta7H9y2N_5OMW81Ll&export=download



Gambar 3. Tampilan Roll Dadu Double Soal 1 Modul 1

A. Source Code

XML:

MainActivity.kt

Tabel 1. Source Code MainActivity.kt Soal 1 Modul 1 XML

1	package com.example.dicerollerxml
2	
3	import android.os.Bundle
4	import android.widget.Button
5	import androidx.activity.enableEdgeToEdge
6	import androidx.appcompat.app.AppCompatActivity
7	import
	com.example.dicerollerxml.databinding.ActivityMainBi
	nding

```

8 import com.google.android.material.snackbar.Snackbar
9
10 class MainActivity : AppCompatActivity() {
11     private lateinit var binding:
ActivityMainBinding
12
13     override fun onCreate(savedInstanceState:
Bundle?) {
14         super.onCreate(savedInstanceState)
15         enableEdgeToEdge()
16         binding =
ActivityMainBinding.inflate(layoutInflater)
17         setContentView(binding.root)
18
19         binding.button.setOnClickListener {
20             rollDice()
21         }
22     }
23
24     private fun rollDice()
25     {
26         val dice = Dice(6)
27         val sides = dice.diceRoll()
28         val sides2 = dice.diceRoll()
29         val button =
findViewById<Button>(R.id.button)
30         val dR1 = when(sides)
31         {
32             1 -> R.drawable.dice_1
33             2 -> R.drawable.dice_2

```

```

34         3 -> R.drawable.dice_3
35         4 -> R.drawable.dice_4
36         5 -> R.drawable.dice_5
37         else -> R.drawable.dice_6
38     }
39
40     val dR2 = when (sides2)
41     {
42         1 -> R.drawable.dice_1
43         2 -> R.drawable.dice_2
44         3 -> R.drawable.dice_3
45         4 -> R.drawable.dice_4
46         5 -> R.drawable.dice_5
47         else -> R.drawable.dice_6
48     }
49     binding.imageView1.setImageResource(dR1)
50     binding.imageView2.setImageResource(dR2)
51
52     val message = if (sides == sides2) {
53         "Selamat, anda mendapatkan dadu double"
54     } else {
55         "Anda belum beruntung"
56     }
57     Snackbar.make(button, message,
58         Snackbar.LENGTH_LONG)
59         .show()
60     }
61 class Dice(private val numSides : Int)
62 {

```

63	fun diceRoll() : Int
64	{
65	return (1..numSides).random()
66	}
67	}

activity_main.xml

Tabel 2. Source Code activity_main.xml Soal 1 Modul 1 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	xmlns:android="http://schemas.android.com/apk/res/an droid"
4	xmlns:app="http://schemas.android.com/apk/res- auto"
5	xmlns:tools="http://schemas.android.com/tools"
6	android:id="@+id/main"
7	android:layout_width="match_parent"
8	android:layout_height="match_parent"
9	tools:context=".MainActivity">
10	
11	<ImageView
12	android:id="@+id/imageView1"
13	android:layout_width="186dp"
14	android:layout_height="231dp"
15	android:layout_marginTop="195dp"
16	android:layout_marginEnd="16dp"
17	app:layout_constraintBottom_toTopOf="@+id/button"
18	app:layout_constraintEnd_toEndOf="parent"

19	app:layout_constraintHorizontal_bias="1.0"
20	app:layout_constraintStart_toEndOf="@+id/imageView2"
21	app:layout_constraintTop_toTopOf="parent"
22	app:layout_constraintVertical_bias="1.0"
23	app:srcCompat="@drawable/dice_0" />
24	
25	<Button
26	android:id="@+id/button"
27	android:layout_width="wrap_content"
28	android:layout_height="wrap_content"
29	android:layout_marginStart="160dp"
30	android:layout_marginEnd="161dp"
31	android:layout_marginBottom="256dp"
32	android:text="Roll"
33	app:layout_constraintBottom_toBottomOf="parent"
34	app:layout_constraintEnd_toEndOf="parent"
35	app:layout_constraintStart_toStartOf="parent" />
36	
37	<ImageView
38	android:id="@+id/imageView2"
39	android:layout_width="195dp"
40	android:layout_height="231dp"
41	android:layout_marginStart="14dp"
42	android:layout_marginTop="195dp"
43	app:layout_constraintBottom_toTopOf="@+id/button"

44	app:layout_constraintEnd_toStartOf="@+id/imageView1"
45	app:layout_constraintStart_toStartOf="parent"
46	app:layout_constraintTop_toTopOf="parent"
47	app:layout_constraintVertical_bias="1.0"
48	app:srcCompat="@drawable/dice_0" />
49	
50	</androidx.constraintlayout.widget.ConstraintLayout>

Compose:

MainActivity.kt

Tabel 3. Source Code MainActivity.kt Soal 1 Modul 1 Compose

1	package com.example.diceroller
2	
3	import android.os.Bundle
4	import android.view.Gravity
5	import android.widget.Toast
6	import androidx.activity.ComponentActivity
7	import androidx.activity.compose.setContent
8	import androidx.activity.enableEdgeToEdge
9	import androidx.annotation.ContentView
10	import androidx.annotation.GravityInt
11	import androidx.compose.foundation.Image
12	import
	androidx.compose.foundation.layout.Arrangement
13	import androidx.compose.foundation.layout.Column
14	import androidx.compose.foundation.layout.Row
15	import androidx.compose.foundation.layout.Spacer
16	import

	<code>androidx.compose.foundation.layout.fillMaxSize</code>
17	<code>import</code>
	<code>androidx.compose.foundation.layout.fillMaxWidth</code>
18	<code>import androidx.compose.foundation.layout.height</code>
19	<code>import androidx.compose.foundation.layout.padding</code>
20	<code>import androidx.compose.foundation.layout.size</code>
21	<code>import androidx.compose.foundation.layout.width</code>
22	<code>import</code>
	<code>androidx.compose.foundation.layout.wrapContentSize</code>
23	<code>import androidx.compose.material.icons.Icons</code>
24	<code>import androidx.compose.material3.Button</code>
25	<code>import</code>
	<code>androidx.compose.material3.ExtendedFloatingActionButton</code>
	<code>utton</code>
26	<code>import androidx.compose.material3.Icon</code>
27	<code>import androidx.compose.material3.Text</code>
28	<code>import androidx.compose.runtime.Composable</code>
29	<code>import androidx.compose.runtime.mutableStateOf</code>
30	<code>import androidx.compose.ui.Alignment</code>
31	<code>import androidx.compose.ui.Modifier</code>
32	<code>import androidx.compose.ui.res.painterResource</code>
33	<code>import androidx.compose.ui.res.stringResource</code>
34	<code>import androidx.compose.ui.tooling.preview.Preview</code>
35	<code>import</code>
	<code>com.example.diceroller.ui.theme.DiceRollerTheme</code>
36	<code>import androidx.compose.ui.unit.dp</code>
37	<code>import androidx.compose.runtime.getValue</code>
38	<code>import androidx.compose.runtime.setValue</code>
39	<code>import androidx.compose.runtime.remember</code>
40	<code>import androidx.compose.ui.focus.focusModifier</code>

```

41 import
42 androidx.compose.ui.graphics.painter.Painter
43 import androidx.compose.ui.platform.LocalContext
44 import kotlinx.coroutines.launch
45
46 class MainActivity : ComponentActivity() {
47     override fun onCreate(savedInstanceState:
48 Bundle?) {
49         super.onCreate(savedInstanceState)
50         enableEdgeToEdge()
51         setContent {
52             DiceRollerTheme {
53                 DiceRollerApp()
54             }
55         }
56     }
57
58     @Composable
59     fun DiceWithButtonAndImage(modifier: Modifier =
60 Modifier) {
61         var result by remember { mutableStateOf(0) }
62         var result2 by remember { mutableStateOf(0) }
63         val context = LocalContext.current
64
65         val imageResource = when (result) {
66             0 -> R.drawable.dice_0
67             1 -> R.drawable.dice_1
68             2 -> R.drawable.dice_2
69             3 -> R.drawable.dice_3

```

69	4 -> R.drawable.dice_4
70	5 -> R.drawable.dice_5
71	else -> R.drawable.dice_6
72	}
73	
74	val imageResource2 = when (result2) {
75	0 -> R.drawable.dice_0
76	1 -> R.drawable.dice_1
77	2 -> R.drawable.dice_2
78	3 -> R.drawable.dice_3
79	4 -> R.drawable.dice_4
80	5 -> R.drawable.dice_5
81	else -> R.drawable.dice_6
82	}
83	Column(
	horizontalAlignment =
84	Alignment.CenterHorizontally,
85	modifier = modifier
86	) {
87	Row(
	horizontalArrangement =
88	Arrangement.Center,
89	modifier = Modifier.fillMaxWidth()
90	) {
91	Image(
	painter =
92	painterResource(imageResource),
	contentDescription =
93	result.toString(),
94	modifier = Modifier.height(200.dp)

95	)
	Spacer(modifier =
96	Modifier.height(100.dp))
97	Image(
	painter =
98	painterResource(imageResource2),
	contentDescription =
99	result2.toString(),
	modifier = Modifier.height(200.dp)
100	)
101	}
102	
103	Spacer(modifier = Modifier.height(16.dp))
104	
105	Button(onClick = {
106	result = (1..6).random()
107	result2 = (1..6).random()
108	val resultText = if (result ==
	result2)
109	"Selamat, anda mendapatkan dadu
	double"
110	else
111	"Anda belum beruntung"
112	val toast =
	Toast.makeText(context, resultText,
	Toast.LENGTH_SHORT)
113	toast.setGravity(Gravity.BOTTOM or
	Gravity.CENTER_HORIZONTAL, 0, 150)
114	
115	toast.show()

116	
117	}) {
118	Text(stringResource(R.string.Roll))
119	}
120	}
121	}
122	
123	@Preview(showBackground = true)
124	@Composable
125	fun DiceRollerApp() {
126	DiceWithButtonAndImage(
127	modifier = Modifier
128	.fillMaxSize()
129	.wrapContentSize(Alignment.Center)
130	)
131	}

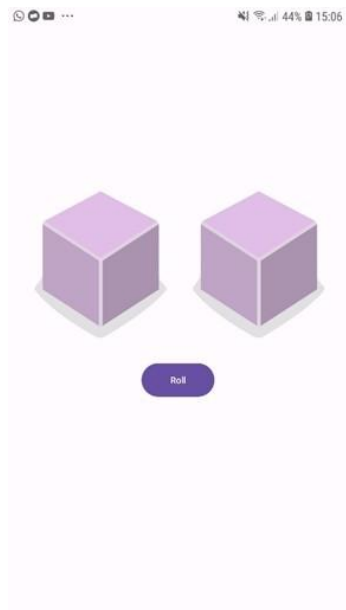
AndroidManifest.xml

Tabel 4. Source Code AndroidManifest.xml Soal 1 Modul 1 Compose

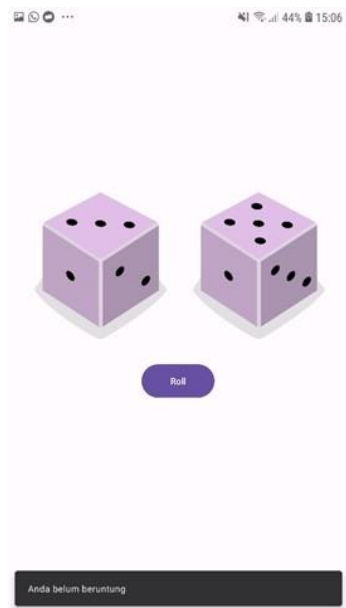
1	<?xml version="1.0" encoding="utf-8"?>
2	<manifest
	xmlns:android="http://schemas.android.com/apk/res/an
	droid"
3	xmlns:tools="http://schemas.android.com/tools" >
4	
5	<application
6	android:allowBackup="true"
7	android:dataExtractionRules="@xml/data_extraction_ru

	les"
8	android:fullBackupContent="@xml/backup_rules"
9	android:icon="@mipmap/ic_launcher"
10	android:label="@string/Roll"
11	android:roundIcon="@mipmap/ic_launcher_round"
12	android:supportsRtl="true"
13	android:theme="@style/Theme.DiceRoller"
14	tools:targetApi="31" >
15	<activity
16	android:name=".MainActivity"
17	android:exported="true"
18	android:label="@string/Roll"
19	android:theme="@style/Theme.DiceRoller"
	>
20	<intent-filter>
21	<action
	android:name="android.intent.action.MAIN" />
22	
23	<category
	android:name="android.intent.category.LAUNCHER" />
24	</intent-filter>
25	</activity>
26	</application>
27	
28	</manifest>

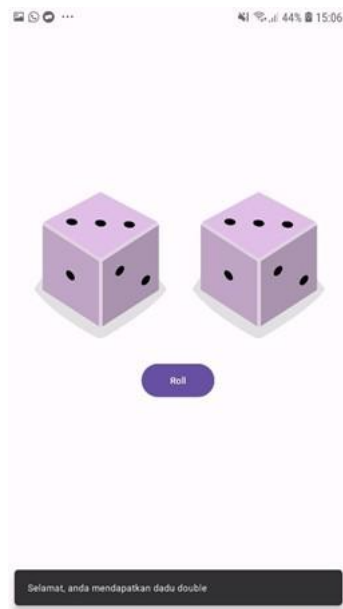
B. Output Program



Gambar 4. Screenshot Hasil Tampilan Awal Soal 1 Modul 1 XML



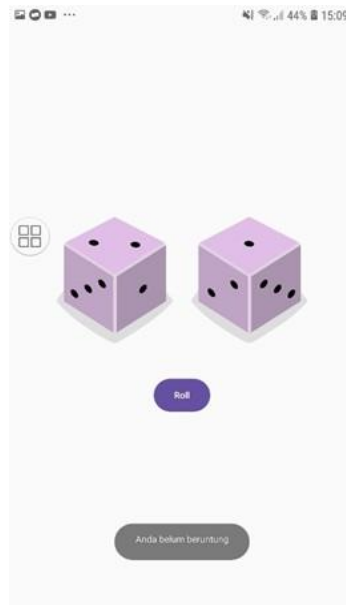
Gambar 5. Screenshot Hasil Tampilan Dadu Tidak Sama Soal 1 Modul 1 XML



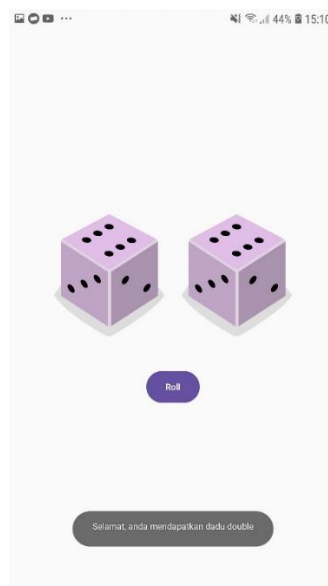
Gambar 6. Screenshot Hasil Tampilan Dadu Sama Soal 1 Modul 1 XML



Gambar 7. Screenshot Hasil Tampilan Awal Soal 1 Modul 1 Compose



Gambar 8. Screenshot Hasil Tampilan Dadu Tidak Sama Soal 1 Modul 1 Compose



Gambar 9. Screenshot Hasil Tampilan Dadu Sama Soal 1 Modul 1 Compose

C. Pembahasan

A. XML:

MainActivity.kt:

Pada baris [11], `private lateinit var binding: ActivityMainBinding` memiliki fungsi untuk mengakses komponen dari `activity_main.xml` tanpa `findViewById`

Pada baris [13 – 22], adalah blok kode yang menghubungkan layout dengan binding, dan root dari binding nya adalah root dari XML. Binding button berfungsi ketika tombol di klik, maka akan memanggil fungsi `rollDice()`

Pada baris [26 – 28], terdapat 3 variable yang pertama mendeklarasikan objek dice dengan 6 sisi, dan membuat objek yang berfungsi untuk meng-roll dua dadu dan menyimpan hasilnya di `sides` dan `sides2`

Pada baris [29], memanggil objek tombol dengan id button yang terhubung di `activity_main.xml` dengan id button.

Pada baris [30 – 48], fungsi `when` untuk menentukan gambar hasil roll, dan `R.drawable.dice` adalah sebagai id dari gambar yang ada pada drawable. Hal ini dilakukan berulang kali pada dua sisi dadu.

Pada baris [49 – 50], menampilkan hasil gambar dadu masing masing dengan `imageView` dan dari dadu yang sudah di kondisikan pada `when` secara acak dari 1 – 6

Pada baris [52 – 58], pada blok ini adalah komponen `snackbar` yang berfungsi untuk menampilkan pop up kecil di bawah layar yang menampilkan hasil dadu ketika sama atau pun tidak dengan pesan yang sudah ditentukan.

Pada baris [61 – 67], pada blok ini adalah class dadu yang berisi `numSides` sebagai jumlah sisi dadu dengan fungsi `diceRoll` yang akan menghasilkan angka acak dari satu sampai enam sisi dadu.

activity_main.xml:

Pada baris [10 – 22], adalah image view layout dadu kanan dengan nama `ImageView1` dengan tinggi, lebar, jarak, margin atas, bawah yang sudah ditentukan.

Pada baris [24 – 34], adalah button yang dipanggil pada `MainActivity.kt` yang akan diposisikan pada tengah tengah layar di bagian bawah dengan `wrap_content` yang lebar dan tinggi mengikuti ukuran teks tombol. Pada bagian ini juga mengatur margin kiri dan kanan untuk tetap berada di tengah

Pada baris [36 – 47], adalah image view layout dadu kiri dengan nama ImageView2 dengan tinggi, lebar, jarak, margin atas, bawah yang sudah ditentukan. hampir mirip dengan ImageView1.

B. Jetpack Compose:

MainActivity.kt:

Pada baris [45 – 55], terdapat beberapa fungsi seperti enableEdgeToEdge untuk aplikasi menggunakan seluruh layar, setContent untuk UI pada Jetpack Compose dan DiceRollerTheme menerapkan tema aplikasi untuk di terapkan di DiceRollerApp

Pada baris [58 – 60], adalah baris kode yang menampilkan fungsi utama untuk dua dadu dan tombol untuk roll dadu, dengan menyimpan nilai dadu secara dinamis, remember menjaga nilai, dan mutableStateOf untuk nilai berubah saat tombol ditekan.

Pada baris [61], ini akan digunakan sebagai Toast untuk mengambil context

Pada baris [63 – 81], adalah baris kode yang menyesuaikan angka dadu dengan gambar di drawable dengan fungsi when yang akan disimpan di variable result dengan nama ImageResource. Begitu juga pada ImageResource2 melakukan hal yang sama untuk dadu yang kedua.

Pada baris [82 – 85], adalah column untuk menampilkan hasil secara vertikal dan rata tengah.

Pada baris [86 – 89], adalah row untuk menampilkan hasil secara horizontal bersamping-sampingan.

Pada baris [90 – 101], adalah fungsi Image yang berisi beberapa objek seperti painter yang menampilkan gambar dari drawable resource, contentDescription sebagai aksesibilitas berdasarkan hasil roll dadu, dan mengatur tinggi gambar.

Pada baris [105 – 117], adalah membuat sebuah tombol dengan parameter onClick yang berfungsi ketika di tekan. Selain itu ada fungsi result untuk mengrandom nilai dari 1 sampai 6 untuk dadu dan disimpan pada variable result juga. Ada juga variable yang menyimpan hasil result jika dadu sama ataupun tidak sama, jika dadu sama menampilkan pesan “anda mendapatkan dadu double”

dan jika tidak “anda belum beruntung”. Terdapat juga variable untuk menampilkan pesan dengan komponen toast untuk membuat pesan ketika tombol ditekan dan mengeluarkan hasilnya dengan show

Pada baris [118], merupakan isi dari tombolnya dengan nama yang akan muncul yaitu Roll

Pada baris [123 – 131], pada blok ini berfungsi sebagai preview dan tampilan aplikasi utama diceRollerApp

AndroidManifest.xml:

Pada baris [5 – 14], ini merupakan beberapa hal penting yang dimana terdapat backup, icon, nama aplikasi yang muncul, dan tema dari aplikasi dengan target API 31

Pada baris [15 – 19], ini merujukkan kemana file aplikasi yaitu MainActivity dan nama aplikasinya juga ditentukan yaitu Roll.

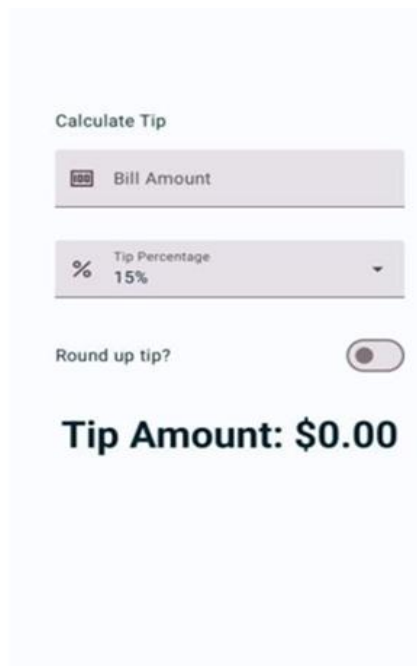
Pada baris [20 – 24], mengartikan bahwa MainActivity adalah apa yang utama dari aplikasi dengan menunjukkan main sebagai aktivitas pertama yang dibuka dan launcher agar bisa dimunculkan di device seperti HP.

MODUL 2 : ANDROID LAYOUT

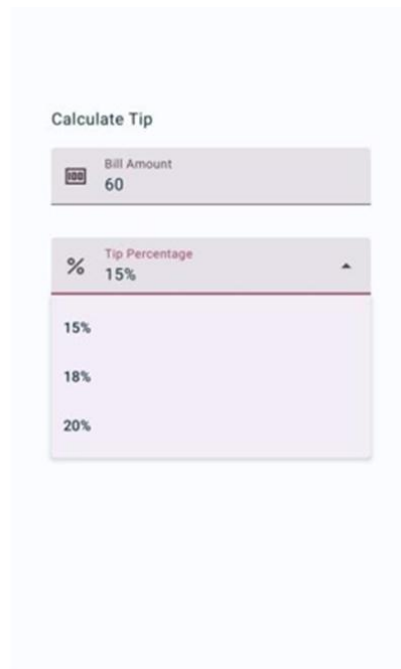
SOAL 1

Buatlah sebuah aplikasi kalkulator tip yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:

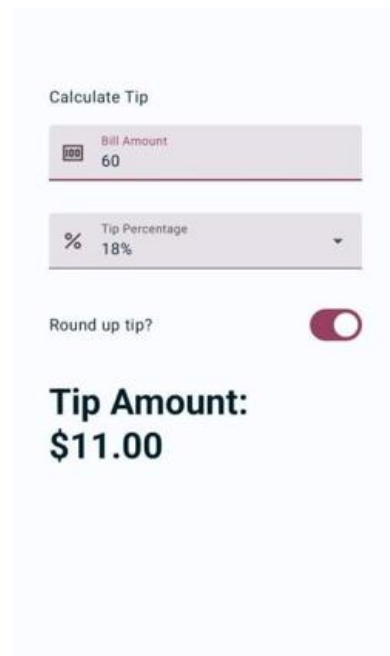
- Input Biaya Layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
- Pilihan Persentase Tip: Pengguna dapat memilih persentase tip yang diinginkan dari opsi yang disediakan, yaitu 15%, 18%, dan 20%.
- Pengaturan Pembulatan Tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
- Tampilan Hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.



Gambar 10. Tampilan Awal Aplikasi Soal 1 Modul 2



Gambar 11. Tampilan Aplikasi Setelah Dijalankan Soal Modul 2



Gambar 12. Tampilan Aplikasi Setelah Dijalankan Soal Modul 2

A. Source Code

MainActivity.kt

Tabel 5. Source Code MainActivity.kt Soal 1 Modul 2 XML

1	package com.example.tipcalculator
2	
3	import android.os.Bundle
4	import android.text. <i>Editable</i>
5	import android.text. <i>TextWatcher</i>
6	import android.widget. <i>ArrayAdapter</i>
7	import androidx.appcompat.app.AppCompatActivity
8	import
	com.example.tipcalculator.databinding.ActivityMainB
9	inding
10	import java.text.NumberFormat
11	import kotlin.math.ceil
12	
13	class MainActivity : AppCompatActivity() {
14	
15	private lateinit var binding:
16	ActivityMainBinding
17	private val tipOptions = <i>listOf</i> ("15%", "18%",
18	"20%")
19	
20	override fun onCreate(<i>savedInstanceState</i> :
21	Bundle?) {
	super.onCreate(<i>savedInstanceState</i>)
22	binding =
23	ActivityMainBinding.inflate(<i>layoutInflater</i>)
24	setContentView(binding.root)


```

25         val adapter = ArrayAdapter(this,
26     android.R.layout.simple_dropdown_item_1line,
27     tipOptions)
28
29         binding.autoCompleteTip.setAdapter(adapter)
30         binding.autoCompleteTip.setText("15%",
31     false)
32         binding.autoCompleteTip.setOnClickListener
33     {
34         binding.autoCompleteTip.showDropDown()
35     }
36
37     binding.autoCompleteTip.setOnItemClickListener { _,
38     _, _, _ ->
39         updateTip()
40     }
41
42     binding.etBill.addTextChangedListener(object :
43     TextWatcher {
44         override fun afterTextChanged(s:
45     Editable?) = updateTip()
46         override fun beforeTextChanged(s:
47     CharSequence?, start: Int, count: Int, after: Int)
48     {}
49         override fun onTextChanged(s:
50     CharSequence?, start: Int, before: Int, count: Int)
51     {}
52     })
53
54     binding.switchRound.setOnCheckedChangeListener { _,

```

	<code>_ -> updateTip() } }</code>
46	<code>private fun updateTip() {</code>
47	<code> val amount =</code>
48	<code>binding.etBill.text.toString().toDoubleOrNull() ?:</code>
49	<code>0.0</code>
50	<code> val tipText =</code>
51	<code>binding.autoCompleteTip.text.toString()</code>
52	<code> val tipPercent =</code>
	<code>tipText.removeSuffix("%").toDoubleOrNull() ?: 15.0</code>
	<code> val roundUp = binding.switchRound.isChecked</code>
	<code> val tip = calculateTip(amount, tipPercent,</code>
	<code>roundUp)</code>
	<code> binding.tvTotalTip.text =</code>
	<code>getString(R.string.tip_amount, tip)</code>
	<code>}</code>
	<code>private fun calculateTip(amount: Double,</code>
	<code>tipPercent: Double, roundUp: Boolean): String {</code>
	<code> var tip = tipPercent / 100 * amount</code>
	<code> if (roundUp) {</code>
	<code> tip = ceil(tip)</code>
	<code> }</code>
	<code> return</code>
	<code>NumberFormat.getCurrencyInstance().format(tip)</code>
	<code>}</code>
	<code>}</code>

activity_main.xml

Tabel 6. Source Code activity_main.xml Soal 1 Modul 2 XML

1	<?xml	version="1.0"	encoding="utf-8"?>		
2	<androidx.constraintlayout.widget.ConstraintLayout				
3	xmlns:android="http://schemas.android.com/apk/res/a	ndroid"			
4	xmlns:app="http://schemas.android.com/apk/res-	auto"			
5	xmlns:tools="http://schemas.android.com/tools"				
6	android:layout_width="match_parent"				
7	android:layout_height="match_parent"				
8	android:padding="24dp"				
9	tools:context=".MainActivity">				
10					
11					
12	<TextView				
13	android:id="@+id/textViewLabel"				
14	android:layout_width="wrap_content"				
15	android:layout_height="wrap_content"				
16	android:layout_marginTop="120dp"				
17	android:text="@string/calculate_tip"				
18	android:textSize="18sp"				
19					
20	app:layout_constraintBottom_toTopOf="@id/etBill"				
21	app:layout_constraintEnd_toEndOf="parent"				
22					
23	app:layout_constraintHorizontal_bias="0.067"				
24					
25	app:layout_constraintStart_toStartOf="parent"				
26	app:layout_constraintTop_toTopOf="parent"				

```

27         />
28
29     <EditText
30         android:id="@+id/etBill"
31         android:layout_width="0dp"
32         android:layout_height="48dp"
33         android:layout_marginTop="2dp"
34         android:hint="@string/bill_amount"
35         android:background="#ffe9f4"
36         android:autofillHints=""
37         android:paddingStart="8dp"
38         android:drawableStart="@drawable/money"
39         android:drawablePadding="8dp"
40         android:inputType="numberDecimal"
41
41     app:layout_constraintTop_toBottomOf="@+id/textViewL
42     abel"
43
44     app:layout_constraintStart_toStartOf="parent"
45         app:layout_constraintEnd_toEndOf="parent"
46         app:layout_constraintHorizontal_bias="0.5"
47         app:layout_constraintWidth_percent="0.9"
48     />
49
50 <com.google.android.material.textfield.TextInputLay
51 out
52     android:id="@+id/tipInputLayout"
53
54     style="@style/Widget.MaterialComponents.TextInputLa

```

```

55 yout.OutlinedBox"
56     android:layout_width="0dp"
57     android:background="#ffe9f4"
58     app:boxBackgroundMode="outline"
59     android:layout_height="wrap_content"
60     android:layout_marginTop="16dp"
61
62     app:layout_constraintTop_toBottomOf="@id/etBill"
63
64     app:layout_constraintStart_toStartOf="parent"
65     app:layout_constraintEnd_toEndOf="parent"
66     app:layout_constraintWidth_percent="0.9">
67     <AutoCompleteTextView
68         android:id="@+id/autoCompleteTip"
69         android:layout_width="match_parent"
70         android:layout_height="48dp"
71
72         android:drawableStart="@drawable/persen"
73         android:drawablePadding="8dp"
74         android:paddingStart="8dp"
75
76         android:drawableEnd="@drawable/baseline_arrow_drop_
77         down_24"
78         android:hint="@string/tip_percentage"
79         android:inputType="none"
80     />
81
82
83 </com.google.android.material.textfield.TextInputLa

```

```

84 yout>
85
86     <androidx.appcompat.widget.SwitchCompat
87         android:id="@+id/switchRound"
88         android:layout_width="0dp"
89         android:layout_height="48dp"
90         android:layout_marginTop="30dp"
91         android:text="@string/round_up_tip"
92
93 app:layout_constraintTop_toBottomOf="@id/tipInputLa
94 yout"
95
96     app:layout_constraintStart_toStartOf="parent"
97         app:layout_constraintEnd_toEndOf="parent"
98         app:layout_constraintWidth_percent="0.9"
99     />
100
101     <TextView
102         android:id="@+id/tvTotalTip"
103         android:layout_width="0dp"
104         android:layout_height="wrap_content"
105         android:layout_marginTop="16dp"
106         android:textSize="40sp"
107         app:layout_constraintEnd_toEndOf="parent"
108
109 app:layout_constraintStart_toStartOf="parent"
110
111 app:layout_constraintTop_toBottomOf="@+id/switchRou
112 nd"
113
114         app:layout_constraintWidth_percent="0.9"

```

	<pre> android:text="Tip Amount: \$0.00" /> </androidx.constraintlayout.widget.ConstraintLayout > </pre>
--	---

Compose:

MainActivity.kt

Tabel 7. Source Code MainActivity.kt Soal 1 Modul 2 Compose

1	package com.example.tipcalculatorcompose
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.activity.enableEdgeToEdge
7	import androidx.annotation.DrawableRes
8	import androidx.annotation.StringRes
9	import
10	androidx.compose.foundation.layout.Arrangement
11	import androidx.compose.foundation.layout.Column
12	import androidx.compose.foundation.layout.Row
13	import androidx.compose.foundation.layout.Spacer
14	import
15	androidx.compose.foundation.layout.fillMaxSize
16	import
17	androidx.compose.foundation.layout.fillMaxWidth
18	import androidx.compose.foundation.layout.height
19	import androidx.compose.foundation.layout.padding

20	import
21	androidx.compose.foundation.layout.safeDrawingPadd
22	ing
23	import
24	androidx.compose.foundation.layout.statusBarsPaddi
25	ng
26	import
27	androidx.compose.foundation.layout.wrapContentWidt
28	h
29	import
30	androidx.compose.foundation.rememberScrollState
31	import
32	androidx.compose.foundation.text.KeyboardOptions
33	import androidx.compose.foundation.verticalScroll
34	import androidx.compose.material3.DropdownMenuItem
35	import
36	androidx.compose.material3.ExperimentalMaterial3Ap
37	i
38	import
39	androidx.compose.material3.ExposedDropdownMenuBox
40	import
41	androidx.compose.material3.ExposedDropdownMenuDefa
42	ults
43	import androidx.compose.material3.Icon
44	import androidx.compose.material3.MaterialTheme
45	import androidx.compose.material3.Surface
46	import androidx.compose.material3.Switch
47	import androidx.compose.material3.Text
	import androidx.compose.material3.TextField
	import androidx.compose.runtime.Composable

48	import androidx.compose.runtime.getValue
49	import androidx.compose.runtime.mutableStateOf
50	import androidx.compose.runtime.remember
51	import androidx.compose.runtime.setValue
52	import androidx.compose.ui.Alignment
53	import androidx.compose.ui.Modifier
54	import androidx.compose.ui.res.painterResource
55	import androidx.compose.ui.res.stringResource
56	import androidx.compose.ui.text.input.ImeAction
57	import androidx.compose.ui.text.input.KeyboardType
58	import androidx.compose.ui.tooling.preview.Preview
59	import androidx.compose.ui.unit.dp
60	import
61	com.example.tipcalculatorcompose.ui.theme.TipTimeT
62	heme
63	import java.text.NumberFormat
64	
65	class MainActivity : ComponentActivity() {
66	override fun onCreate(savedInstanceState:
67	Bundle?) {
68	enableEdgeToEdge()
69	super.onCreate(savedInstanceState)
70	setContent {
71	TipTimeTheme{
72	Surface(
	modifier =
73	Modifier.fillMaxSize(),
74	) {
75	TipTimeLayout()
76	}

	<pre> } } } 77 } 78 } 79 80 @OptIn(ExperimentalMaterial3Api::class) 81 @Composable 82 fun TipTimeLayout() { 83 var amountInput by remember { 84 mutableStateOf("") } 85 var roundUp by remember { 86 mutableStateOf(false) } 87 88 val options = listOf("15%", "18%", "20%") 89 var selectedOptionText by remember { 90 mutableStateOf(options[0]) } 91 var expanded by remember { 92 mutableStateOf(false) } 93 94 val amount = amountInput.toDoubleOrNull() ?: 95 0.0 96 val tipPercent = 97 selectedOptionText.removeSuffix("%").toDoubleOrNul 98 l() ?: 15.0 99 val tip = calculateTip(amount, tipPercent, 100 roundUp) 101 102 Column(103 modifier = Modifier .statusBarsPadding() </pre>
--	---

104	.padding(horizontal = 40.dp)
105	.verticalScroll(rememberScrollState())
106	.safeDrawingPadding(),
107	horizontalAlignment =
108	Alignment.CenterHorizontally,
109	verticalArrangement = Arrangement.Center
110	) {
111	Text(
112	text =
113	stringResource(R.string.calculate_tip),
114	modifier = Modifier
115	.padding(bottom = 16.dp, top =
116	40.dp)
117	.align(alignment =
	Alignment.Start)
118	)
	EditNumberField(
119	label = R.string.bill_amount,
120	leadingIcon = R.drawable.money,
121	keyboardOptions =
122	KeyboardOptions.Default.copy(
123	keyboardType =
	KeyboardType.Number,
124	imeAction = ImeAction.Next
125	),
126	value = amountInput,
127	onValueChanged = { amountInput = it },
128	modifier = Modifier.padding(bottom =
129	32.dp).fillMaxWidth(),
130	)

131	ExposedDropDownMenuBox (
132	expanded = expanded,
133	onExpandedChange = {expanded =
	!expanded},
134	modifier = Modifier
135	.padding(bottom = 32.dp)
136	.fillMaxWidth()
137	) {
138	TextField(
139	readOnly = true,
140	value = selectedOptionText,
141	onValueChange = {},
	label = { Text("Tip Percentage")
142	},
143	trailingIcon = {
144	
145	ExposedDropDownMenuDefaults.TrailingIcon(expanded
146	= expanded)
147	},
148	modifier = Modifier
149	.menuAnchor()
150	.fillMaxWidth(),
151	colors =
152	ExposedDropDownMenuDefaults.textFieldColors()
153	)
154	ExposedDropDownMenu(
155	expanded = expanded,
156	onDismissRequest = { expanded =
157	false}
158	) {

159	options.forEach{ selectionOption -
160	>
161	DropDownMenuItem(
162	text = {
163	Text(selectionOption)},
164	onClick = {
165	selectedOptionText =
166	selectionOption
167	expanded = false
168	}
169	}
170	}
171	
172	}
173	
174	
175	RoundTheTipRow(
176	roundUp = roundUp,
177	onRoundUpChanged = { roundUp = it },
178	modifier = Modifier.padding(bottom =
179	32.dp)
180	)
181	Text(
182	text =
183	stringResource(R.string.tip_amount, tip),
184	style =
185	MaterialTheme.typography.displaySmall,
186	modifier =
187	Modifier.align(Alignment.Start)

188	)
189	Spacer(modifier = Modifier.height(150.dp))
190	}
191	}
192	
193	@Composable
194	fun EditNumberField(
195	@StringRes label: Int,
196	@DrawableRes leadingIcon: Int,
197	keyboardOptions: KeyboardOptions,
	value: String,
198	onValueChanged: (String) -> Unit,
199	modifier: Modifier = Modifier
200	) {
201	TextField(
202	value = value,
203	singleLine = true,
204	leadingIcon = { Icon(painter =
205	painterResource(id = leadingIcon), null) },
206	modifier = modifier,
207	onValueChange = onValueChanged,
208	label = { Text(stringResource(label)) },
209	keyboardOptions = keyboardOptions
210	)
211	}
	@Composable
	fun RoundTheTipRow(
	roundUp: Boolean,
	onRoundUpChanged: (Boolean) -> Unit,

```

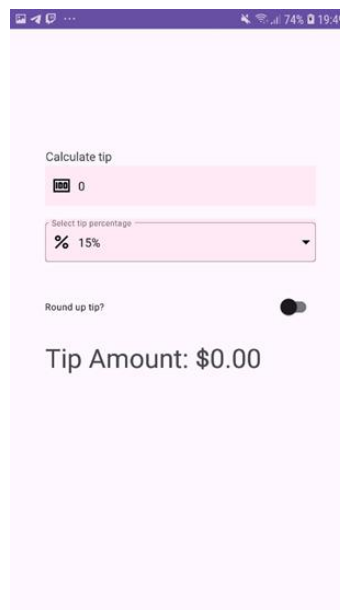
        modifier: Modifier = Modifier
    ) {
        Row(
            modifier = modifier.fillMaxWidth(),
            verticalAlignment =
Alignment.CenterVertically
        ) {
            Text(text =
stringResource(R.string.round_up_tip))
            Switch(
                modifier = Modifier
                    .fillMaxWidth()
                    .wrapContentWidth(Alignment.End),
                checked = roundUp,
                onCheckedChange = onRoundUpChanged
            )
        }
    }

private fun calculateTip(amount: Double,
tipPercent: Double = 15.0, roundUp: Boolean):
String {
    var tip = tipPercent / 100 * amount
    if (roundUp) {
        tip = kotlin.math.ceil(tip)
    }
    return
NumberFormat.getCurrencyInstance().format(tip)
}

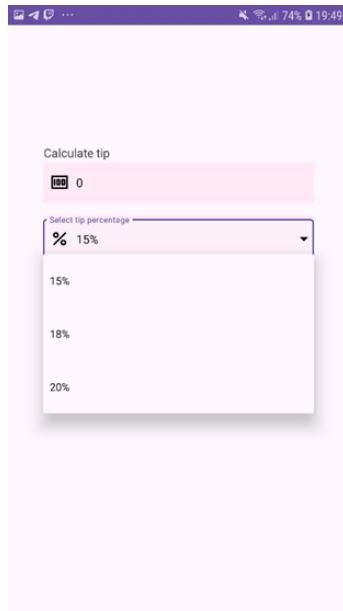
```

	<pre> @Preview(showBackground = true) @Composable fun TipTimeLayoutPreview() { TipTimeTheme{ TipTimeLayout() } } </pre>
--	---

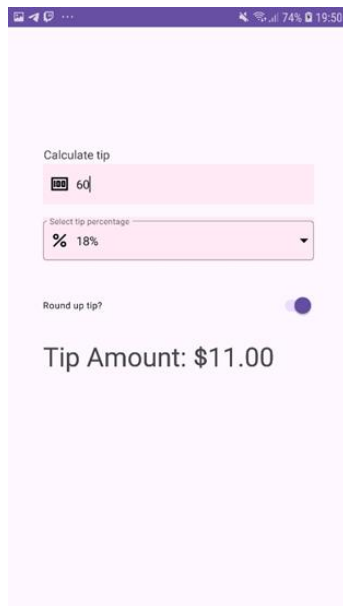
B. Output Program



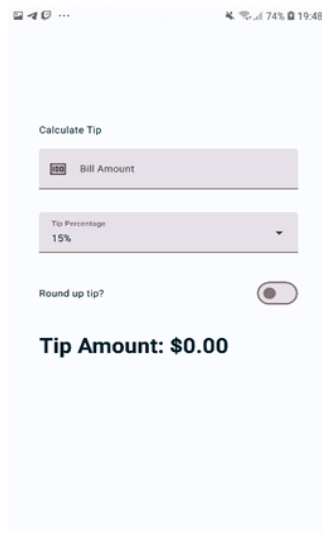
Gambar 13. Screenshot Hasil Tampilan Awal Soal 1 Modul 2 XML



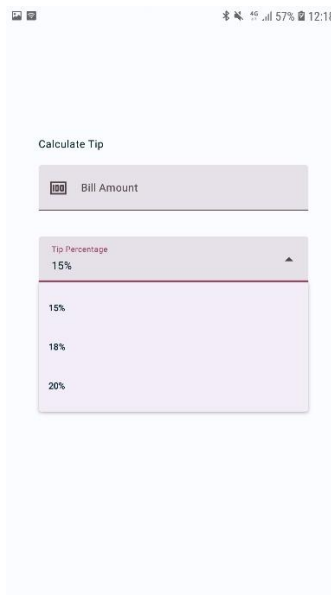
Gambar 14. Screenshot Hasil Tampilan Pilihan Soal 1 Modul 2 XML



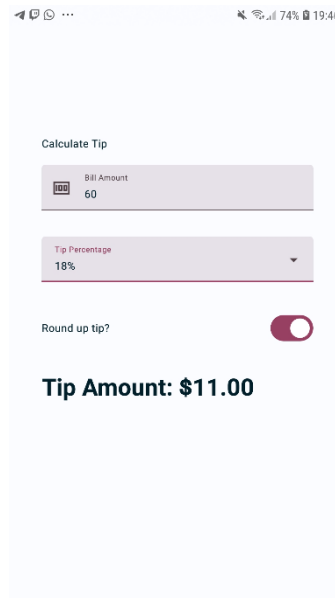
Gambar 15. Screenshot Hasil Tampilan Setelah Dijalankan Soal 1 Modul 2 XML



Gambar 16. Screenshot Hasil Tampilan Awal Soal 1 Modul 2 Compose



Gambar 17. Tampilan Pilihan Persentasi Tip Compose



Gambar 18. Screenshot Hasil Tampilan Setelah Dijalankan Soal 1 Modul 2 Compose

C. Pembahasan

XML:

MainActivity.kt:

Pada baris [1], package com.example.tipcalculator ini menentukan package dari file ini

Pada baris [3 – 10], ini berisikan import import yang digunakan pada logika ini, import perubahan teks, menghubungkan list data ke autoCompleteTextView, base class untuk fitur fitur, view binding, memformat angka jadi bentuk mata uang, dan membulatkan angka keatas

Pada baris [12], class MainActivity : AppCompatActivity() { ini mewarisi dari AppCompatActivity

Pada baris [14], private lateinit var binding: ActivityMainBinding ini adalah properti view binding untuk akses view.

Pada baris [15], private val tipOptions = listOf("15%", "18%", "20%") ini list persentase tip untuk dropdown.

Pada baris [17 – 18], ini adalah fungsi yang pertama kali dipanggil saat activity dibuat.

Pada baris [19], `binding = ActivityMainBinding.inflate(layoutInflater)` ini merupakan inisialisasi view binding

Pada baris [20], `setContentView(binding.root)` menampilkan layout utama dari binding

Pada baris [21], baris ini membuat adapter untuk menghubungkan list `tipOptions` ke dropdown `AutoCompleteTextView`

Pada baris [23], baris ini mengatur adapter ke `AutoCompleteTextView`

Pada baris [24], baris ini menetapkan nilai awal saat aplikasi dibuka yaitu 15%

Pada baris [25 – 27], ini baris yang membuat binding dapat menampilkan dropdown saat di klik

Pada baris [28 – 30], ini baris yang membuat user saat memilih item dari dropdown langsung update nilai tip

Pada baris [31 – 35], menambahkan listener untuk melihat perubahan teks pada `EditText`. Dimana ketika teks berubah langsung hitung ulang tip.

Pada baris [36], membuat switch pembulatan yang jika di on/off langsung hitung ulang nilai tip

Pada baris [37 – 44], pada blok kode ini membuat fungsi menghitung dan menampilkan tip. Mulai dari nilai bill yang dikonversi ke double dan default nya kosong. Teks persentase dari dropdown. Menghapus % lalu ubah ke double jika default 15.0 gagal. Mengambil status switch apakah perlu dibulatkan atau tidak. Hitung nilai tip berdasarkan input dan kondisi pembulatan. Yang terakhir menampilkan hasil di `textview` menggunakan resource format.

Pada baris [45 – 51], pada blok kode ini menghitung fungsi nilai tip dan mengembalikan hasil dalam format mata uang. Nilai tip dengan rumus $(\text{persen}/100) * \text{jumlah}$. Jika pengguna membulatkan maka akan membulatkan ke atas untuk nilai tipnya, lalu hasilnya di format ke bentuk uang dan di return sebagai string.

activity_main.xml:

Pada baris [1 – 8], pada XML ini adalah root untuk seluruh tampilan dengan menggunakan `constraintlayout`, `padding` 24dp semua sisi, dan untuk previewnya di android studio dipakai di `MainActivity`.

Pada baris [10 – 21], ini akan menampilkan layout teks `Calculate Tip` diatas input dan diposisikan diatas layar dan jarak vertikalnya 120dp

Pada baris [23 – 40], ini akan menampilkan layout input bill dengan icon money dikiri dan padding agar tidak terlalu rapat, user hanya diperbolehkan angka dan decimal.

Pada baris [42 – 53], membuat text input layout dengan border dan akan digunakan membungkus `AutoCompleteTextView` agar seperti input biasa, dan mengubah background dengan warna pink muda

Pada baris [55 – 65], `AutoCompleteTextView` menampilkan dropdown berisi pilihan persen dan icon persen di kiri, panah dropdown di kanan. Ada juga input type none yang artinya tidak bisa costum input atau cukup memilih saja.

Pada baris [69 – 79], menampilkan tombol switch untuk membulatkan keatas atau tidak, diposisikan dibawah dropdown tip dan ada label disebelah kiri dengan “Round Up Tip?”

Pada baris [81 – 92], menampilkan hasil perhitungan tip, dengan ukuran teks besar dan dirampilkan dibawah switch rata kiri.

Jetpack Compose:**MainActivity.kt:**

Pada baris [1], package `com.example.tipcalculatorcompose` menyatakan bahwa file berada dalam package ini

Pada baris [3 – 47], adalah seluruh import yang ada di dalam program compose ini

Pada baris [49 – 63], ini adalah class utama aplikasi yang merupakan turunan dari `ComponentActivity`, mengatur konten bisa muncul hingga batas layar, menerapkan tema dari `TipTimeTheme` dan `surface` yang merupakan komponen material design yang bisa di beri `background`, `padding`, dan `fillMaxSize()` untuk memenuhi layar. Selain itu juga memanggil UI utama yang kita buat di `TipTimeLayout()`

Pada baris [67 – 77], pada blok kode ini adalah fungsi `composable` utama yang memiliki beberapa variable seperti input teks tagihan dan menyimpan state selama `recompositionn` dan `mutableStateOf` dengan nilai default kosong. State untuk menyimpan apakah tip dibulatkan atau tidak. Membuat daftar pilihan persen tip yang akan ditampilkan di dropdown. Membuat state untuk menyimpan pilihan yang sedang dipilih dropdown dengan default nilai adalah 15%. State yang mengatur apakah dropdown sedang dibuka atau tidak. State yang mengkonversi input ke string double. Mengambil angka dari pilihan dropdown, dan memanggil fungsi `calculateTip` untuk menghitung nilai tip.

Pada baris [79 – 87], membuat layout vertikal dari atas kebawah, dengan memberi `padding` otomatis untuk menghindari status bar, diberikan margin kiri kanan 40dp dan isi kolom bisa di scroll kalau melebihi layar. Menyusun kolom semua widget agar ditengah horizontal dan diratakan di tengah secara vertikal.

Pada baris [88 – 93], menampilkan teks dari resource string `Calculate Tip` yang diberi `padding` atas dan bawah, serta rata kiri meski kolom center align

Pada baris [94 – 104], memanggil fungsi `EditNumberField` untuk input jumlah tagihan dengan label `bill amount`, dan pakai icon gambar money. Selain itu membuat keyboard dikostum jadi tipe angka dan enter menjadi next. Menyambungkan input dan state saat user mengetik, maka `amountInput` akan diperbarui. Komponen juga diberi `padding` bawah dan mengisi penuh lebar layar.

Pada baris [105 – 140], ini adalah blok kode yang cukup panjang yang berisikan dropdown menu dari pilihan persentasenya. Ada container untuk dropdown menu untuk mengecek apakah dropdownnya terbuka dan saat di klik dengan status `true` atau

false. Menambahkan margin bawah dan agar box selebar parentnya. Pada TextField menampilkan pilihan 15% karena readOnly, user tidak mengetik tapi memilih dari dropdown. Label pada text untuk membuat nama didalamnya. Trailing icon yang menampilkan ikon panah disamping kanan apakah menu sedang dibuka. menuAnchor() adalah titik untuk posisi dropdown dengan fillMaxWidth untuk selebar mungkin. Dan menggunakan warna default untuk TextField. Daftar dalam opsi dropdown nya ditutup saat klik di luar, dengan membuat loop untuk item dari list options dan juga saat item dipilih ke nilai yang baru maka akan diperbarui nilainya dan dropdown ditutup.

Pada baris [143 – 154], blok kode ini adalah untuk fungsi switch. Memanggil fungsi RoundTheTipRow yang berisi roundUp dengan nilai boolean apakah switchnya menyala. onRoundUpChanged akan dipanggil saat user menyalakan atau mematikan switch. Teks hasil tip akan ditampilkan dengan nilai dari tip dari calculateTip()

Pada baris [157 – 164], pada blok fungsi EditNumberField ini adalah fungsi UI. Membuat label resource ID dari string untuk label dengan menggunakan @StringRes dan leadingIcon untuk resource gambar menggunakan @DrawableRes. Parameter selanjutnya yaitu tipe keyboard, nilai dari input, dan fungsi yang dijalankan saat user mengupdate state.

Pada baris [165 – 174], blok kode ini membuat satu baris input dengan parameter value, dan mengambil ikon disebelah kiri input dari resource dengan painterResource. Membuat style dan fungsi update value. Menampilkan label dari resource string. Mengatur jenis keyboard sesuai dengan parameter.

Pada baris [177 – 181], blok kode ini menyatakan apakah switch dalam kondisi true atau false dengan fungsi callback ketika switch dinyalakan atau dimatikan.

Pada baris [182 – 195], membuat baris selebar dengan parent dan teks sejajar tengah secara vertikal. Pada label ini menampilkan label dari resource dan switch yang diujung kanan dengan kondisi sekarang boolean dan onCheckedChange dijalankan saat user ubah switch.

Pada baris [197 – 203], blok ini membuat fungsi yang berisi amount, tipPercent, dan roundUp. Tip akan dihitung dengan rumus tipPercent dibagi 100 lalu dikali dengan amount. Jika user ingin membulatkan maka roundUp akan true dan dibulatkan ke atas ke bilangan bulat terdekat. Selain itu mengubah format tip jadi string dengan format uang lokal.

Pada baris [205 – 211], memanggil tema dari TipTimeTheme dan memanggil TipTimeLayout untuk ditampilkan sebagai preview.

SOAL 2

Jelaskan perbedaan dari implementasi XML dan Jetpack Compose beserta kelebihan dan kekurangan dari masing-masing implementasi.

A. Pembahasan

Kedua hal ini cukup berbeda dalam hal implementasi, contohnya yang paling utama ialah paradigma yang digunakan. Paradigma dari XML sendiri adalah imperative dan Jetpack Compose paradigmanya yaitu declarative. Pada paradigma XML sendiri masih menggabungkan konsep Java + Kotlin, sedangkan pada paradigma Jetpack Compose sudah seluruhnya menggunakan Kotlin.

Kelebihan XML :

1. Pilihan yang bagus untuk memulai sebuah aplikasi android karna mudah dipahami
2. Pemisahan UI dan Logika lebih jelas
3. Tata letak di XML juga bisa digunakan kembali atau reusable

Kekurangan XML :

1. Tata letak yang terlalu banyak, menghasilkan kurangnya efisiensi kode
2. Perlu menulis kode yang panjang
3. Fleksibilitasnya terbatas untuk XML

Kelebihan Jetpack Compose :

1. Sudah menggunakan paradigma deklaratif, yang sangat memudahkan siapapun yang memakainya
2. Mengurangi penulisan kode yang panjang
3. Dapat melihat perubahan UI secara langsung dengan Jetpack Compose.

Kekurangan Jetpack Compose :

1. Perlu pembelajaran lebih giat untuk memahami Jetpack Compose
2. Dokumentasi terbatas karena tergolong cara baru dalam penerapan android
3. Tidak semua bisa menggunakan Jetpack Compose karena masalah kompabilitas.

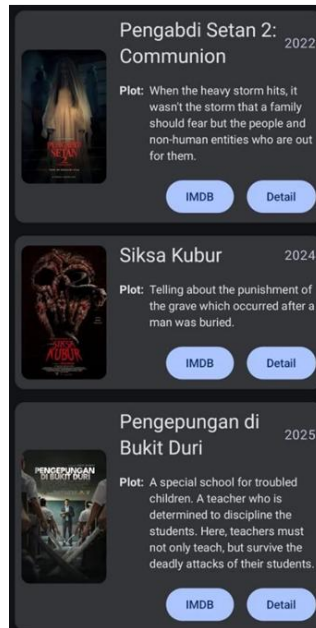
MODUL 3 : BUILD A SCROLLABLE LIST

SOAL 1

Buatlah sebuah aplikasi Android menggunakan XML atau Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:

1. List menggunakan fungsi RecyclerView (XML) atau LazyColumn (Compose)
2. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas
3. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah
4. Terdapat 2 button dalam list, dengan fungsi berikut:
 - a. Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain
 - b. Button kedua menggunakan Navigation component/intent untuk membuka laman detail item
5. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius
6. Saat orientasi perangkat berubah/dirotasi, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang
7. Aplikasi menggunakan arsitektur single activity (satu activity memiliki beberapa fragment)
8. Aplikasi berbasis XML harus menggunakan ViewBinding

UI item list harus berisi 1 gambar, 2 button (intent eksplisit dan navigasi), dan 2 baris teks dan setiap baris memiliki 2 teks yang berbeda. Diusahakan agar desain UI item list menyerupai UI berikut:



Gambar 19. Contoh UI List Soal 1 Modul 3

Desain UI laman detail bebas, tetapi diusahakan untuk mengikuti kaidah desain Material Design dan data item ditampilkan penuh di laman detail seperti contoh berikut:



Gambar 20. Contoh UI Detail Soal 1 Modul 3

A. Source Code

XML:

MainActivity.kt

Tabel 8. Source Code MainActivity.kt Soal 1 Modul 3 XML

1	<code>package com.example.movielistxml</code>
2	
3	<code>import android.os.Bundle</code>
4	<code>import android.util.Log</code>
5	<code>import androidx.appcompat.app.AppCompatActivity</code>
6	
7	<code>class MainActivity : AppCompatActivity() {</code>
8	<code> override fun onCreate(savedInstanceState:</code>
9	<code>Bundle?) {</code>
10	<code> super.onCreate(savedInstanceState)</code>
11	<code> setContentView(R.layout.activity_main)</code>
12	

13	<code>val fragmentManager =</code>
14	<code>supportFragmentManager</code>
	<code>val homeFragment = HomeFragment()</code>
	<code>val fragment =</code>
15	<code>fragmentManager.findFragmentByTag(HomeFragment::cla</code>
16	<code>ss.java.simpleName)</code>
	<code>if (fragment !is HomeFragment){</code>
17	<code>Log.d("MyFragment", "Fragment Name :" +</code>
18	<code>HomeFragment::class.java.simpleName)</code>
19	<code>fragmentManager</code>
	<code>.beginTransaction()</code>
20	<code>.add(R.id.frame_container,</code>
21	<code>homeFragment, HomeFragment::class.java.simpleName)</code>
	<code>.commit()</code>
22	<code>}</code>
23	<code>}</code>
	<code>}</code>

Movie.kt

Tabel 9. Source Code Movie.kt Soal 1 Modul 3 XML

1	<code>package com.example.movieslistxml</code>
2	
3	<code>import android.os.Parcelable</code>
4	<code>import kotlinx.parcelize.Parcelize</code>
5	
6	<code>@Parcelize</code>
7	<code>data class Movie (</code>
8	<code>val title: String,</code>
9	<code>val url: String,</code>

10	<code>val image: Int,</code>
11	<code>val plot: String,</code>
12	<code>val years: String,</code>
13	<code>val desc: String</code>
14	<code>): Parcelable</code>

ListMovieAdapter.kt

Tabel 10. Source Code ListMovieAdapter.kt Soal 1 Modul 3 XML

1	<code>package com.example.movieslistxml</code>
2	
3	<code>import android.view.LayoutInflater</code>
4	<code>import android.view.View</code>
5	<code>import android.view.ViewGroup</code>
6	<code>import android.widget.Button</code>
7	<code>import android.widget.ImageView</code>
8	<code>import android.widget.TextView</code>
9	<code>import androidx.recyclerview.widget.RecyclerView</code>
10	
11	<code>class ListMovieAdapter (</code>
12	<code> private val listMovie: ArrayList<Movie>,</code>
13	<code> private val onImdbClick: (String) -> Unit,</code>
14	<code> private val onDetailClick: (String, Int, String,</code>
15	<code>String, String) -> Unit</code>
	<code>) : RecyclerView.Adapter<ListMovieAdapter.ListViewHolder> (</code>
	<code>) {</code>
16	
17	<code> class ListViewHolder(itemView: View) :</code>
18	<code>RecyclerView.ViewHolder(itemView) {</code>
	<code> val imgPoster: ImageView =</code>

```

19 itemView.findViewById(R.id.img_poster)
        val tvName: TextView =
20 itemView.findViewById(R.id.tv_item_title)
        val years: TextView =
21 itemView.findViewById(R.id.tv_item_years)
        val plot: TextView =
22 itemView.findViewById(R.id.tv_item_plot)
        val desc: TextView =
23 itemView.findViewById(R.id.tv_item_desc)
        val btnImdb: Button =
24 itemView.findViewById(R.id.btn_imdb)
        val btnDetail: Button =
25 itemView.findViewById(R.id.btn_detail)
    }
    override fun onCreateViewHolder(parent: ViewGroup,
26 viewType: Int): ListViewHolder{
        val view: View =
27 LayoutInflater.from(parent.context).inflate(R.layout.ite
28 m_movie, parent, false)
        return ListViewHolder(view)
    }
    override fun getItemCount(): Int = listMovie.size
29
30 override fun onBindViewHolder(holder:
31 ListViewHolder, position: Int){
32     val (title, url, image, plot, years, desc) =
33 listMovie[position]
        holder.tvName.text = title
        holder.plot.text = plot
34 holder.imgPoster.setImageResource(image)
        holder.years.text = years
35 holder.desc.text = desc
36

```


37	holder.btnImdb.setOnClickListener{onImdbClick(url)}
38	
39	holder.btnDetail.setOnClickListener{onDetailClick(title,
40	image, plot, years, desc)}
	}
41	}
42	
43	

HomeFragment.kt

Tabel 11. Source Code HomeFragment.kt Soal 1 Modul 3 XML

1	package com.example.movielistxml
2	
3	import android.content.Intent
4	import android.net.Uri
5	import android.os.Bundle
6	import androidx.fragment.app.Fragment
7	import android.view.LayoutInflater
8	import android.view.View
9	import android.view.ViewGroup
10	import
11	androidx.recyclerview.widget.LinearLayoutManager
	import
12	com.example.movielistxml.databinding.FragmentHomeBi
13	nding
14	
15	class HomeFragment : Fragment() {
16	

17	private var _binding: FragmentHomeBinding? = null
18	private val binding get() = _binding!!
19	
20	private lateinit var movieAdapter:
21	ListMovieAdapter
22	private val list = ArrayList<Movie>()
23	
24	override fun onCreateView(
25	inflater: LayoutInflater, container:
26	ViewGroup?,
27	savedInstanceState: Bundle?
28	): View? {
29	_binding =
30	FragmentHomeBinding.inflate(inflater, container,
31	false)
32	list.clear()
33	list.addAll(getListMovie())
34	setupRecyclerView()
35	
36	return binding.root
37	}
38	
39	private fun setupRecyclerView(){
40	movieAdapter = ListMovieAdapter(
41	list,
42	onImdbClick = {url ->
	val intent =
	Intent(Intent.ACTION_VIEW, Uri.parse(url))
	startActivity(intent)

```

43         },
44         onClick = {title, image, years,
45 plot, desc ->
46             val detailFragment =
47 DetailFragment().apply {
48                 arguments = Bundle().apply {
49                     putString("EXTRA_NAME",
50 title)
51                     putInt("EXTRA_PHOTO", image)
52                     putString("EXTRA_YEARS",
53 years)
54                     putString("EXTRA_PLOT",
55 plot)
56                     putString("EXTRA_DESC",
57 desc)
58                 }
59             }
60 parentFragmentManager.beginTransaction()
61             .replace(R.id.frame_container,
62 detailFragment)
63             .addToBackStack(null)
64             .commit()
65         }
66     )
67     binding.rvMovie.apply {
68         layoutManager
69         =LinearLayoutManager(context)
70         adapter = movieAdapter
71         setHasFixedSize(true)

```

	<pre> } } private fun getListMovie(): ArrayList<Movie>{ val dataMovie = 71 resources.getStringArray(R.array.data_title) 72 val dataLink = 73 resources.getStringArray(R.array.data_link) val dataImage = resources.obtainTypedArray(R.array.data_image) 74 val dataPlot = 75 resources.getStringArray(R.array.plot) 76 val dataYear = 77 resources.getStringArray(R.array.data_years) 78 val dataDesc = 79 resources.getStringArray(R.array.data_description) 80 val listMovie = ArrayList<Movie>() 81 for (i in dataMovie.indices){ 82 val movie = Movie(dataMovie[i], 83 dataLink[i], dataImage.getResourceId(i, -1), 84 dataPlot[i], dataYear[i], dataDesc[i]) listMovie.add(movie) } dataImage.recycle() return listMovie } override fun onDestroyView() { super.onDestroyView() _binding = null </pre>
--	---

	}
	}

DetailFragment.kt

Tabel 12. Source Code DetailFragment.kt Soal 1 Modul 3 XML

1	package com.example.movielistxml
2	
3	import android.os.Bundle
4	import androidx.fragment.app.Fragment
5	import android.view.LayoutInflater
6	import android.view.View
7	import android.view.ViewGroup
8	import
	com.example.movielistxml.databinding.FragmentDetail
9	Binding
10	
11	
12	class DetailFragment : Fragment() {
13	
14	private var _binding: FragmentDetailBinding? =
15	null
16	private val binding get() = _binding!!
17	
18	override fun onCreateView(
19	inflater: LayoutInflater, container:
20	ViewGroup?,
	savedInstanceState: Bundle?
21	): View? {
22	

23	<code>_binding</code>	<code>=</code>
24	<code>FragmentDetailBinding.inflate(inflater, container,</code>	
25	<code>false)</code>	
26		
27	<code>val title</code>	<code>=</code>
28	<code>arguments?.getString("EXTRA_NAME")</code>	
29	<code>val image = arguments?.getInt("EXTRA_PHOTO")</code>	
30	<code>val years</code>	<code>=</code>
31	<code>arguments?.getString("EXTRA_YEARS")</code>	
32	<code>val plot</code>	<code>=</code>
33	<code>arguments?.getString("EXTRA_PLOT")</code>	
34	<code>val desc</code>	<code>=</code>
35	<code>arguments?.getString("EXTRA_DESC")</code>	
36		
37	<code>binding.tvItemTitle.text = title</code>	
38	<code>binding.tvItemPlot.text = years</code>	
39	<code>binding.tvItemYears.text = plot</code>	
40	<code>binding.tvItemDesc.text = desc</code>	
41	<code>image?.let {</code>	
42	<code>binding.imgPoster.setImageResource(it)</code>	
	<code>}</code>	
	<code>return binding.root</code>	
	<code>}</code>	
	<code>override fun onDestroyView() {</code>	
	<code>super.onDestroyView()</code>	
	<code>_binding = null</code>	
	<code>}</code>	
	<code>}</code>	

activity_main.xml

Tabel 13. Source Code activity_main.xml Soal 1 Modul 3 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<FrameLayout
3	
4	xmlns:android="http://schemas.android.com/apk/res/a ndroid"
5	xmlns:tools="http://schemas.android.com/tools"
6	android:id="@+id/frame_container"
7	android:layout_width="match_parent"
8	android:layout_height="match_parent"
9	tools:context=".MainActivity">
10	</FrameLayout>

fragment_detail.kt

Tabel 14. Source Code fragment_detail.xml Soal 1 Modul 3 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
	xmlns:android="http://schemas.android.com/apk/res/ android"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	xmlns:app="http://schemas.android.com/apk/res- auto"
9	tools:context=".DetailFragment"
10	android:layout_margin="10dp">
11	

12	<ImageView
13	android:id="@+id/img_poster"
14	android:layout_width="wrap_content"
15	android:layout_height="400dp"
16	android:layout_marginTop="12dp"
17	android:contentDescription="Poster"
18	tools:src="@drawable/nwh"
19	app:layout_constraintTop_toTopOf="parent"
20	
21	app:layout_constraintStart_toStartOf="parent"
22	/>
23	
24	<TextView
25	android:id="@+id/tv_item_title"
26	android:layout_width="350dp"
27	android:layout_height="wrap_content"
28	tools:text="Ini Judul"
29	
30	app:layout_constraintTop_toBottomOf="@id/img_poster"
31	
32	
33	app:layout_constraintStart_toStartOf="parent"
34	android:textSize="24sp"
35	android:textStyle="bold"
36	android:textColor="@color/black"
37	/>
38	
39	<TextView
40	android:id="@+id/tv_item_years"
41	android:layout_width="wrap_content"

42	android:layout_height="wrap_content"
43	android:layout_marginTop="4dp"
44	tools:text="2020"
45	android:textSize="20sp"
46	app:layout_constraintEnd_toEndOf="parent"
47	app:layout_constraintHorizontal_bias="1.0"
48	app:layout_constraintStart_toEndOf="@id/tv_item_title"
49	
50	app:layout_constraintTop_toBottomOf="@id/img_poster" />
51	
52	<TextView
53	android:id="@+id/tv_item_plot"
54	android:layout_width="wrap_content"
55	android:layout_height="wrap_content"
56	tools:text="Plot :"
57	
58	app:layout_constraintStart_toStartOf="parent"
59	
60	app:layout_constraintTop_toBottomOf="@id/tv_item_title"
61	
62	android:textSize="16sp"
63	android:textStyle="bold"
64	android:textColor="@color/black"
65	/>
66	<TextView
	android:id="@+id/tv_item_desc"

67	android:layout_width="wrap_content"
68	android:layout_height="wrap_content"
	tools:text="deskripsi"
	app:layout_constraintTop_toBottomOf="@id/tv_item_p lot"
	app:layout_constraintStart_toStartOf="parent"
	android:textSize="16sp"
	android:textStyle="bold"
	android:textColor="@color/black"
	/>
	</androidx.constraintlayout.widget.ConstraintLayout>

fragment_home.xml

Tabel 15. Source Code fragment_home.xml Soal 1 Modul 3 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	xmlns:android="http://schemas.android.com/apk/res/a ndroid"
4	xmlns:app="http://schemas.android.com/apk/res- auto"
5	xmlns:tools="http://schemas.android.com/tools"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent"
8	tools:context=".HomeFragment">
9	
10	
11	<androidx.recyclerview.widget.RecyclerView

12	android:id="@+id/rv_movie"
13	android:layout_width="0dp"
14	android:layout_height="0dp"
15	android:layout_margin="15dp"
16	
17	app:layout_constraintBottom_toBottomOf="parent"
18	app:layout_constraintEnd_toEndOf="parent"
19	
	app:layout_constraintStart_toStartOf="parent"
	app:layout_constraintTop_toTopOf="parent" />
	</androidx.constraintlayout.widget.ConstraintLayout
	>

item_movie.kt

Tabel 16. Source Code item_movie.kt Soal 1 Modul 3 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.cardview.widget.CardView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	
5	xmlns:card_view="http://schemas.android.com/apk/res-auto"
6	
7	xmlns:tools="http://schemas.android.com/tools"
8	android:id="@+id/card_view"
9	android:layout_width="match_parent"
10	android:layout_height="wrap_content"
11	android:layout_gravity="center"
12	android:layout_marginStart="8dp"

13	android:layout_marginTop="4dp"
14	android:layout_marginEnd="8dp"
15	android:layout_marginBottom="4dp"
16	card_view:cardCornerRadius="4dp">
17	
18	
19	<androidx.constraintlayout.widget.ConstraintLayout
20	android:layout_width="match_parent"
21	android:layout_height="wrap_content"
22	android:padding="8dp"
23	android:background="@color/gray">
24	
25	<ImageView
	android:id="@+id/img_poster"
26	android:layout_width="70dp"
	android:layout_height="100dp"
27	android:scaleType="centerCrop"
28	
29	card_view:layout_constraintBottom_toBottomOf="parent"
30	nt"
31	
32	card_view:layout_constraintStart_toStartOf="parent"
33	"
34	
35	card_view:layout_constraintTop_toTopOf="parent"
36	tools:src="@drawable/nwh" />
37	
38	<TextView
39	android:id="@+id/tv_item_title"
40	android:layout_width="200dp"

41	<pre> android:layout_height="wrap_content" android:textColor="@color/white" android:layout_marginStart="12dp" android:textSize="24sp" android:textStyle="bold" tools:layout_editor_absoluteX="78dp" tools:layout_editor_absoluteY="2dp" </pre>
42	
43	
44	
45	
46	
47	<pre>card_view:layout_constraintStart_toEndOf="@id/img_</pre>
48	<pre>poster"</pre>
49	
50	<pre>card_view:layout_constraintEnd_toStartOf="@id/tv_i</pre>
51	<pre>tem_years"</pre>
52	<pre> tools:text="Ini Judul" /></pre>
53	
	<pre> <TextView </pre>
54	<pre> android:id="@+id/tv_item_years"</pre>
55	<pre> android:layout_width="wrap_content"</pre>
56	<pre> android:layout_height="wrap_content"</pre>
57	<pre> android:layout_marginStart="16dp"</pre>
58	<pre> android:layout_marginTop="12dp"</pre>
59	<pre> tools:text="2020"</pre>
60	<pre> android:textColor="@color/white"</pre>
61	
62	<pre>card_view:layout_constraintTop_toTopOf="parent"</pre>
63	
64	<pre>card_view:layout_constraintStart_toEndOf="@id/tv_i</pre>
65	<pre>tem_title"</pre>
66	<pre> tools:layout_editor_absoluteY="29dp" /></pre>

67	<TextView
	android:id="@+id/tv_item_plot"
68	android:layout_width="wrap_content"
	android:layout_height="wrap_content"
69	android:layout_marginTop="8dp"
70	android:layout_marginStart="12dp"
71	tools:text="Plot :"
72	android:textColor="@color/white"
73	android:textSize="15sp"
74	android:textStyle="bold"
75	
	card_view:layout_constraintStart_toEndOf="@id/img_
76	poster"
77	card_view:layout_constraintTop_toBottomOf="@id/tv_
78	item_title" />
79	
80	<TextView
81	android:id="@+id/tv_item_desc"
82	android:layout_width="0dp"
83	android:layout_height="wrap_content"
84	android:layout_marginStart="12dp"
85	android:textColor="@color/white"
86	
87	card_view:layout_constraintTop_toTopOf="@id/tv_ite
88	m_plot"
89	card_view:layout_constraintStart_toEndOf="@id/tv_i
	tem_plot"
90	

91	card_view:layout_constraintEnd_toEndOf="parent"
	tools:layout_editor_absoluteY="42dp"
92	tools:text="Deskripsi Film" />
93	
94	<Button
95	android:id="@+id/btn_detail"
96	android:layout_width="wrap_content"
97	android:layout_height="wrap_content"
98	android:layout_marginTop="4dp"
99	android:text="@string/btn_detail"
100	card_view:layout_constraintEnd_toEndOf="parent"
101	card_view:layout_constraintHorizontal_bias="0.508"
102	card_view:layout_constraintStart_toEndOf="@id/btn_
103	imdb"
104	card_view:layout_constraintTop_toBottomOf="@+id/tv_
	_item_desc" />
	<Button
	android:id="@+id/btn_imdb"
	android:layout_width="wrap_content"
	android:layout_height="wrap_content"
	android:layout_marginStart="144dp"
	android:layout_marginTop="4dp"
	android:text="@string/btn_imdb"

	<pre>card_view:layout_constraintStart_toStartOf="parent" " card_view:layout_constraintTop_toBottomOf="@id/tv_ item_desc" /> </androidx.constraintlayout.widget.ConstraintLayout> </androidx.cardview.widget.CardView></pre>
--	--

Compose:

String.xml

Tabel 17. Source String Data Soal 1 Modul 3 Compose

	<pre><resources> <string name="app_name">Movie List</string> <string name="title1">Iron Man 3</string> <string name="title2">Captain America: Civil War</string> <string name="title3">Avengers: Endgame</string> <string name="title4">Avengers: Infinity War</string> <string name="title5">Spider-man: No Way Home</string> <string name="url1">https://www.imdb.com/title/tt1300854/?r ef_=nv_sr_srsg_0_tt_8_nm_0_in_0_q_Iron%2520Man%2520 3</string> <string name="url2">https://www.imdb.com/title/tt3498820/?r ef_=nv_sr_srsg_0_tt_6_nm_0_in_0_q_captain%2520civil %2520war</string> <string name="url3">https://www.imdb.com/title/tt4154756/?r ef_=nv_sr_srsg_0_tt_8_nm_0_in_0_q_infinity%2520war< /string></pre>
--	---

	<pre> <string name="url4">https://www.imdb.com/title/tt4154796/?r ef_=nv_sr_srsg_0_tt_8_nm_0_in_0_q_endgame</string> <string name="url5">https://www.imdb.com/title/tt10872600/? ref_=nv_sr_srsg_0_tt_8_nm_0_in_0_q_no%2520way</stri ng> <string name="releaseDate1">2013</string> <string name="releaseDate2">2016</string> <string name="releaseDate3">2019</string> <string name="releaseDate4">2018</string> <string name="releaseDate5">2021</string> <string name="desc1">Brilliant industrialist Tony Stark / Iron Man faces an enemy that knows no bounds. Tony world is destroyed and he investigates who is responsible for the destruction. Armed with his tools and instincts, Tony sets out to save those he loves. An old question comes back to haunt him; did humans make the Iron Man suit or did the suit define his identity as a human?</string> <string name="desc2">The heavy civilian casualties have led the government to control the superheroes. This heated situation has Iron Man and Captain America camps opposing each other. The battle is inevitable.</string> <string name="desc3">The remaining Avengers try to reverse Thanos actions that destroyed half the universe. They travel through time to collect the </pre>
--	--

	Infinity Stones, then use the energy of the stones to go back in time and change the events that caused the Snap.</string> <string name="desc4">The Avengers and their allies must be willing to sacrifice everything in an attempt to defeat the powerful Thanos before he succeeds in destroying the universe.</string> <string name="desc5">For the first time in Spider-Man big-screen history, the affable hero's true identity is revealed, causing his superpower responsibilities to clash with his normal life, and putting everyone closest to him at risk.</string> <string name="description1">Brilliant industrialist Tony Stark / Iron Man faces an enemy that knows no bounds. Tony's world is destroyed and he investigates who is responsible for the destruction. Armed with his tools and instincts, Tony sets out to save those he loves. An old question comes back to haunt him; did humans make the Iron Man suit or did the suit define his identity as a human?</string> <string name="description2">The heavy civilian casualties have led the government to control the superheroes. This heated situation has Iron Man and Captain America camps opposing each other. The battle is inevitable.</string> <string name="description3">The remaining Avengers try to reverse Thanos' actions that destroyed half the universe. They travel through time to collect the Infinity Stones, then use the energy of
--	--

	the stones to go back in time and change the events that caused the Snap.</string> <string name="description4">The Avengers and their allies must be willing to sacrifice everything in an attempt to defeat the powerful Thanos before he succeeds in destroying the universe.</string> <string name="description5">For the first time in Spider-Man big-screen history, the affable hero true identity is revealed, causing his superpower responsibilities to clash with his normal life, and putting everyone closest to him at risk.</string> </resources>
--	---

DataMovie.kt

Tabel 18. Source String Code DataMovie.kt Soal 1 Modul 3 Compose

1	package com.example.mlistcompose.data
2	
3	import com.example.mlistcompose.R
4	import com.example.mlistcompose.model.ListMovie
5	
6	class DataMovie{
7	fun loadMovieList(): List<ListMovie> = listOf(
8	ListMovie(R.string.title1, R.string.url1,
	R.drawable.ironman3, R.string.releaseDate1,
	R.string.desc1, R.string.description1),
9	ListMovie(R.string.title2, R.string.url2,
	R.drawable.civilwar, R.string.releaseDate2,
	R.string.desc2, R.string.description2),
10	

	<pre> ListMovie(R.string.title3, R.string.url3, R.drawable.infinitywar, R.string.releaseDate3, 11 R.string.desc3, R.string.description3), ListMovie(R.string.title4, R.string.url4, R.drawable.endgame, R.string.releaseDate4, 12 R.string.desc4, R.string.description4), ListMovie(R.string.title5, R.string.url5, R.drawable.nwh, R.string.releaseDate5, 13 R.string.desc5, R.string.description5) 14) } </pre>
--	--

ListMovie.kt

Tabel 19. Source Code ListMovie.kt Soal 1 Modul 3 Compose

1	package com.example.mlistcompose.model
2	
3	import androidx.annotation.DrawableRes
4	import androidx.annotation.StringRes
5	
6	data class ListMovie(
7	@StringRes val title: Int,
8	@StringRes val url: Int,
9	@DrawableRes val image: Int,
10	@StringRes val years: Int,
11	@StringRes val desc: Int,
12	@StringRes val description: Int
13	)

DetailActivity.kt

Tabel 20. Source Code DetailActivity.kt Soal 1 Modul 3 Compose

1	package com.example.mlistcompose
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.compose.foundation.Image
7	import androidx.compose.material3.Surface
8	import androidx.compose.material3.Text
9	import androidx.compose.ui.Alignment
10	import androidx.compose.ui.Modifier
11	import androidx.compose.foundation.layout.*
12	import androidx.compose.ui.graphics.Color
13	import androidx.compose.ui.res.painterResource
14	import androidx.compose.ui.res.stringResource
15	import androidx.compose.ui.text.TextStyle
16	import androidx.compose.ui.text.font.FontWeight
17	import androidx.compose.ui.unit.dp
18	import androidx.compose.ui.unit.sp
19	import
20	com.example.mlistcompose.ui.theme.MListComposeTheme
21	
22	class DetailActivity : ComponentActivity() {
23	override fun onCreate(savedInstanceState:
24	Bundle?) {
25	super.onCreate(savedInstanceState)
26	val title = intent.getIntExtra("title", 0)
27	val years = intent.getIntExtra("years", 0)
	val image = intent.getIntExtra("image", 0)

28	val	description	=
29	intent.getIntExtra("description", 0)		
30			
31	setContent {		
32	MListComposeTheme {		
33	Surface(		
34	modifier	=	
35	Modifier.fillMaxSize(),		
36	color = Color.Black		
37	) {		
38	Column (		
39	modifier = Modifier		
40	.fillMaxWidth()		
41	.height(200.dp)		
42	.padding(16.dp)		
	){		
43	Image(		
44	modifier	=	
	Modifier.height(225.dp).fillMaxWidth(),		
45	painter	=	
46	painterResource(image),		
47	contentDescription	=	
48	stringResource(title)		
49	)		
50	Row (		
51	modifier = Modifier		
52	.fillMaxWidth()		
53	){		
54	Text(		

55		text	=
56	stringResource(title),		
57		style = TextStyle(	
		fontSize	=
58	40.sp,		
59		fontWeight	=
60	FontWeight.Bold,		
61		color	=
62	Color.White		
63		),	
64		modifier	=
65	Modifier.padding(bottom = 4.dp)		
66		.weight(1F)	
		)	
		Text(	
67		text	=
	stringResource(years),		
68		style = TextStyle(	
69		fontSize	=
70	24.sp,		
71		color	=
72	Color.White		
73		),	
74		modifier	=
75	Modifier.padding(bottom = 4.dp)		
76	.wrapContentWidth(Alignment.End)		
77			
78	.align(Alignment.CenterVertically)		
79		)	

80	}
	Text(
81	text = "Plot :",
82	style = TextStyle(
83	fontSize =
84	16.sp,
85	fontWeight =
86	FontWeight.Bold,
87	color =
88	Color.White
89	)
90	)
91	Text(
	text =
	stringResource(description),
	style = TextStyle(
	fontSize =
	16.sp,
	color =
	Color.White
	)
	)
	}
	}
	}
	}
	}
	}

MainActivity.kt

Tabel 21. Source Code MainActivity.kt Soal 1 Modul 3 Compose

1	package com.example.mlistcompose
2	
3	import android.content.Intent
4	import android.os.Bundle
5	import androidx.activity.ComponentActivity
6	import androidx.activity.compose.setContent
7	import androidx.activity.enableEdgeToEdge
8	import androidx.compose.foundation.Image
9	import
10	androidx.compose.foundation.layout.Arrangement
11	import androidx.compose.foundation.layout.Column
12	import
13	androidx.compose.foundation.layout.PaddingValues
14	import androidx.compose.foundation.layout.Spacer
15	import
16	androidx.compose.foundation.layout.fillMaxWidth
17	import androidx.compose.foundation.layout.padding
18	import
19	androidx.compose.foundation.layout.wrapContentHeig
20	ht
21	import androidx.compose.foundation.lazy.LazyColumn
22	import
23	androidx.compose.foundation.shape.RoundedCornerSha
24	pe
25	import androidx.compose.material3.Card
26	import androidx.compose.material3.CardDefaults
27	import androidx.compose.runtime.Composable
28	import androidx.compose.ui.Modifier

29	import androidx.compose.ui.draw.clip
30	import androidx.compose.material3.Text
31	import androidx.compose.ui.layout.ContentScale
32	import androidx.compose.ui.unit.dp
33	import androidx.compose.foundation.lazy.items
34	import androidx.compose.ui.platform.LocalContext
35	import androidx.compose.ui.res.painterResource
36	import androidx.compose.foundation.layout.Row
37	import
38	androidx.compose.foundation.layout.fillMaxSize
39	import androidx.compose.foundation.layout.size
40	import androidx.compose.foundation.layout.width
41	import
42	androidx.compose.foundation.layout.wrapContentWidth
43	h
44	import androidx.compose.material3.Button
45	import androidx.compose.ui.res.stringResource
46	import androidx.compose.ui.tooling.preview.Preview
47	import com.example.mlistcompose.model.ListMovie
48	import com.example.mlistcompose.data.DataMovie
49	import androidx.compose.material3.MaterialTheme
50	import androidx.compose.material3.Surface
51	import androidx.compose.ui.Alignment
52	import androidx.core.net.toUri
53	import androidx.compose.ui.text.TextStyle
54	import androidx.compose.ui.text.font.FontWeight
55	import androidx.compose.ui.unit.sp
56	import androidx.compose.ui.graphics.Color
57	
58	

59	import	
	com.example.mlistcompose.ui.theme.MListComposeThem	
60	e	
61		
62	class MainActivity : ComponentActivity() {	
63	override fun onCreate(savedInstanceState:	
64	Bundle?) {	
65	super.onCreate(savedInstanceState)	
66	enableEdgeToEdge()	
67	setContent {	
	MListComposeTheme {	
68	Surface (	
69	modifier	=
70	Modifier.fillMaxSize(),	
71	color = Color.Black	
72	){	
73	MovieListScreen(movie	=
74	DataMovie().loadMovieList())	
75	}	
76	}	
77	}	
78	}	
79	}	
80	@Composable	
81	fun MovieListScreen(movie: List<ListMovie>) {	
82	LazyColumn (contentPadding	=
83	PaddingValues(16.dp), verticalArrangement	=
84	Arrangement.spacedBy(16.dp))	
85	{	
86	items(movie){item ->	

87	MovieItems(movie = item)	
88	}	
89	}	
90	}	
91	@Composable	
92	fun MovieItems(movie: ListMovie){	
93	val context = LocalContext.current	
94	val url = stringResource(movie.url)	
95	val releaseDate = stringResource(movie.years)	
96		
97	Card(	
	shape = RoundedCornerShape(16.dp),	
98	modifier = Modifier	
99	.fillMaxWidth()	
100	.wrapContentHeight(),	
101	colors = CardDefaults.cardColors(	
102	containerColor = Color(0xFF343539)	
103	),	
104	elevation	=
105	CardDefaults.cardElevation(4.dp)	
106	) {	
107	Row(	
108	modifier = Modifier	
109	.padding(16.dp)	
110	.fillMaxWidth()	
111	) {	
112	Image(	
113	painter	=
114	painterResource(movie.image),	
115		

116	contentDescription	=
117	stringResource(movie.title),	
118	contentScale = ContentScale.Crop,	
119	modifier = Modifier	
120	.size(width = 90.dp, height =	
121	200.dp)	
122		
	.clip(RoundedCornerShape(8.dp))	
123		
124	.align(Alignment.CenterVertically)	
125	)	
126	Spacer(modifier	=
127	Modifier.width(12.dp))	
128	Column(	
129	modifier = Modifier	
130	.fillMaxWidth()	
131		
	.align(Alignment.CenterVertically)	
132	) {	
	Row (	
133	modifier = Modifier	
134	.fillMaxWidth()	
135	.padding(8.dp)	
136	) {	
137	Text(	
138	text	=
139	stringResource(movie.title),	
140	style = TextStyle(	
141	fontSize = 26.sp,	
142		

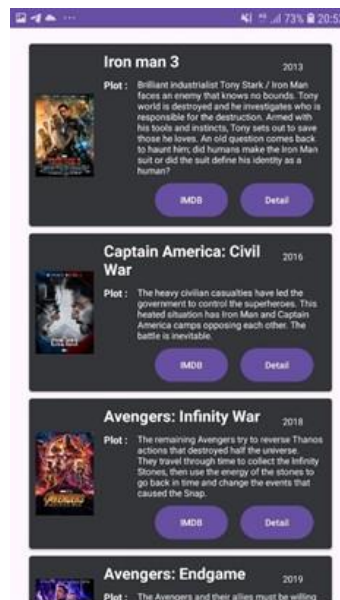
143	fontWeight	=
144	FontWeight.Bold,	
145	color = Color.White	
146	),	
147	modifier	=
148	Modifier.padding(bottom = 4.dp)	
149	.weight(1F)	
150	)	
151	Text(	
152	text = releaseDate,	
153	style = TextStyle(	
154	fontSize = 20.sp,	
	color = Color.White	
155	),	
156	modifier	=
157	Modifier.padding(bottom = 4.dp)	
158		
159	.wrapContentWidth(Alignment.End)	
160		
161	.align(Alignment.CenterVertically)	
162	)	
163	}	
	Row (	
164	modifier = Modifier	
165	.fillMaxWidth()	
166	) {	
167	Text(	
168	text = "Plot :",	
169	style = TextStyle(	
170	fontSize = 16.sp,	

	fontWeight	=
171	FontWeight.Bold,	
172	color = Color.White	
173	)	
174	)	
	Text(	
175	text	=
176	stringResource(movie.desc),	
177	style = TextStyle(	
178	fontSize = 16.sp,	
179	color = Color.White	
180	),	
181	modifier	=
182	Modifier.padding(bottom = 8.dp)	
183	)	
184	}	
185		
186	Row(	
187	modifier	=
188	Modifier.fillMaxWidth(),	
189	horizontalArrangement	=
190	Arrangement.End	
191	) {	
	Button(onClick = {	
	val intent	=
	Intent(Intent.ACTION_VIEW, url.toUri())	
	context.startActivity(intent)	
	}) {	
	Text("IMDB")	

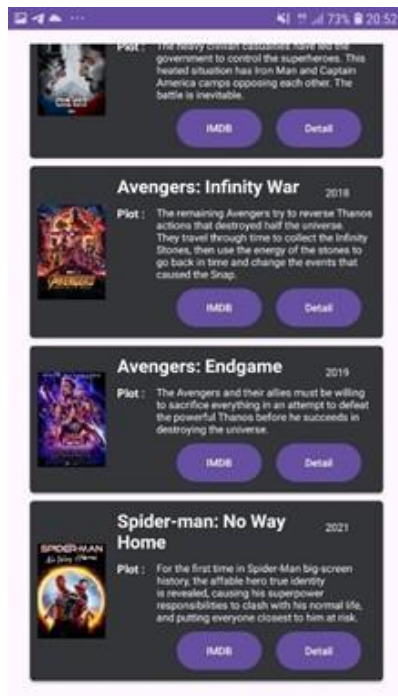
	<pre> } Spacer(modifier = Modifier.width(4.dp)) Button(onClick = { val intent = Intent(context, DetailActivity::class.java).apply { putExtra("title", movie.title) putExtra("years", movie.years) putExtra("image", movie.image) putExtra("description", movie.description) } context.startActivity(intent) }) { Text("Detail") } } } @Preview(showBackground = true) @Composable fun GreetingPreview() { MListComposeTheme { </pre>
--	--

	<pre> MovieListScreen(movie DataMovie().loadMovieList()) } } </pre>	=
--	---	---

B. Output Program



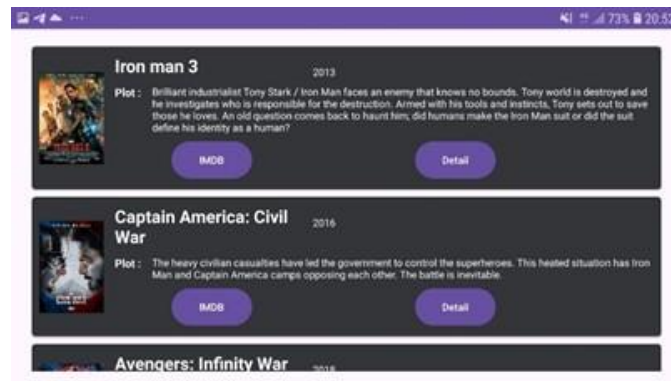
Gambar 21. Screenshot Hasil Tampilan Awal Soal 1 Modul 3 XML



Gambar 22. Screenshot Hasil Tampilan Ketika di Scroll Soal 1 Modul 3 XML



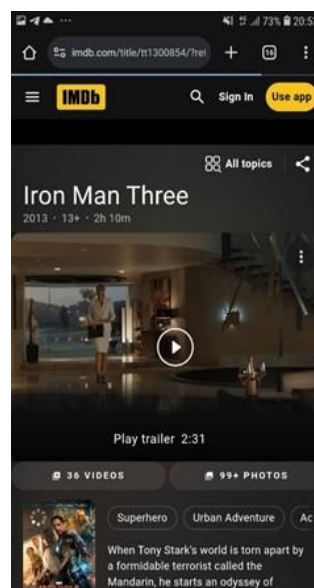
Gambar 23. Screenshot Hasil Tampilan Tombol Detail Soal 1 Modul 3 XML



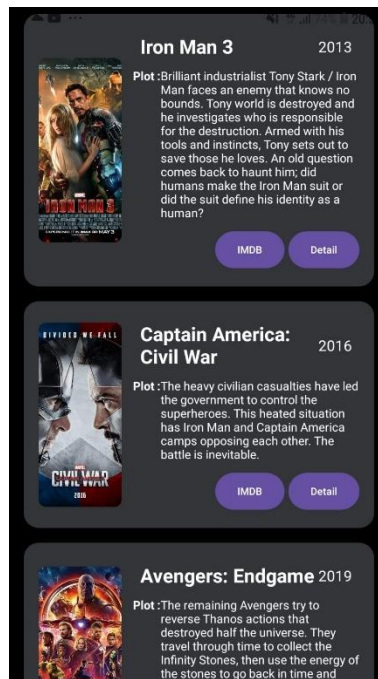
Gambar 24. Screenshot Hasil Tampilan List (landscape) Soal 1 Modul 3 XML



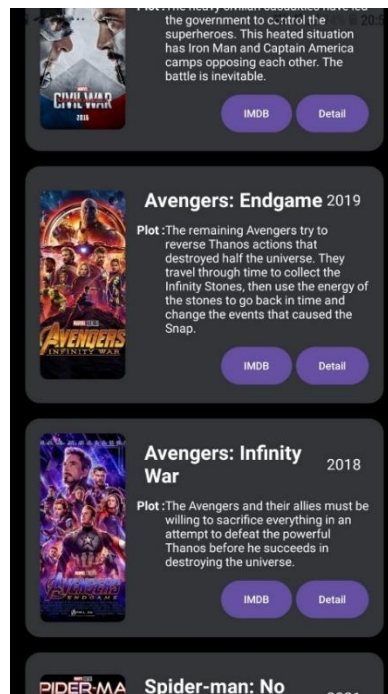
Gambar 25. Screenshot Hasil Tampilan Detail (landscape) Soal 1 Modul 3 XML



Gambar 26. Screenshot Hasil Tampilan Tombol IMDB Soal 1 Modul 3



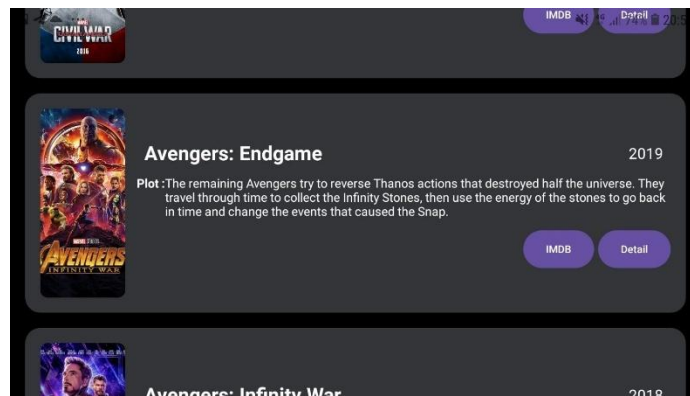
Gambar 27. Screenshot Hasil Tampilan Awal Soal 1 Modul 3 Compose



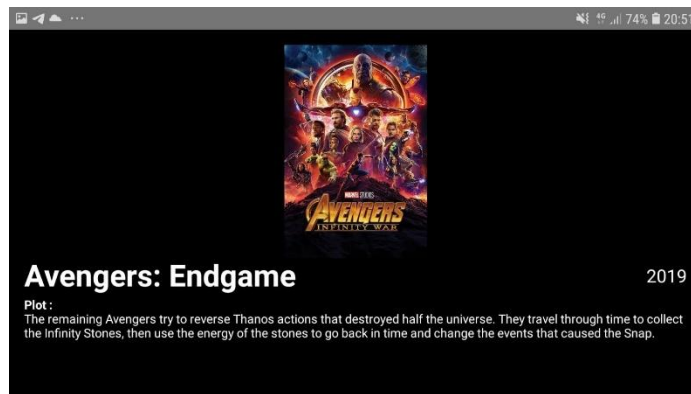
Gambar 28. Screenshot Hasil Tampilan Ketika di Scroll Soal 1 Modul 3 Compose



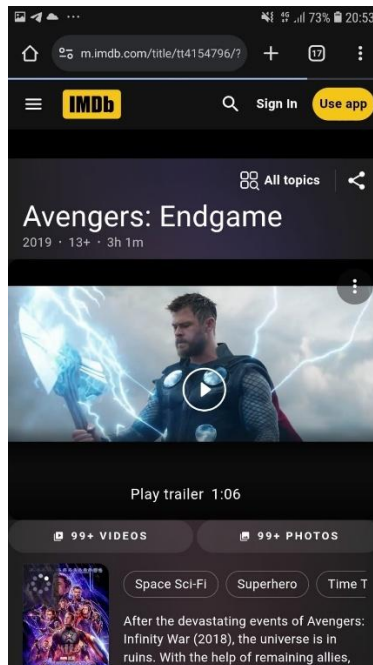
Gambar 29. Screenshot Hasil Tampilan Tombol Detail Soal 1 Modul 3 Compose



Gambar 30. Screenshot Hasil Tampilan List (landscape) Soal 1 Modul 3 Compose



Gambar 31. Screenshot Hasil Tampilan Detail (landscape) Soal 1 Modul 3 Compose



Gambar 32. Screenshot Hasil Tampilan Tombol IMDB Soal 1 Modul 3

C. Pembahasan

XML:

MainActivity.kt

Pada baris [1 – 5], mendeklarasikan package dan import yang digunakan untuk definisi file merupakan bagian dari package com.example.movieslistxml

Pada baris [7], mendeklarasikan activity utama dari AppCompatActivity, yang merupakan MainActivity adalah layar utama aplikasi

Pada baris [8 – 10], fungsi lifecycle dipanggil saat activity dibuat pertama kali dan memastikan bahwa implementasi onCreate dari superclass tetap jalan. Selain itu mengatur layout xml akan digunakan sebagai UI untuk activity ini.

Pada baris [12 – 13], mendapatkan FragmentManager untuk mengelola activity dan membuat instance dari HomeFragment

Pada baris [14 – 15], mengecek apakah Fragment dengan tag nama kelas HomeFragment sudah pernah ditambah, jika belum maka blok if akan jalan.

Pada baris [16 – 21], mencetak log debugging bahwa fragment sudah ditambah, memulai proses perubahan, menambahkan homeFragment ke dalam container R.id.frame_container dan commit untuk menyimpan dan menjalankan perubahan transaksi fragment

Movie.kt

Pada baris [1 – 4]. Mendeklarasikan file dalam package yang sama dan interface untuk memungkinkan objek dikirim antar komponen android antar activity atau fragment. Parcelize untuk anotasi dari Kotlin extension untuk menghindari penulisan parcelable secara manual.

Pada baris [6 – 14], menginstruksikan Kotlin secara otomatis menghasilkan parcelable untuk kelas ini dan data yang digunakan untuk menyimpan dan mengelola data secara sederhana. Terdapat beberapa properti seperti title, url, image, plot, years, dan desc. Parcelable menandakan bahwa objek movie bisa diparcel dan dikirim dengan intent atau bundle.

ListAdapterMovie.kt

Pada baris [1 – 9], mengimpor komponen android yang dibutuhkan untuk membuat dan mengatur tampilan daftar menggunakan RecyclerView

Pada baris [12 – 16], berisi class yang berisi listMovie sebagai daftar film yang akan ditampilkan, lambda function yang menerima url dan detail film saat di tekan. Selain itu mewarisi RecyclerView.Adapter dan ListViewAdapter sebagai ViewHoldernya

Pada baris [18-26], ViewHolder ini menyimpan referensi ke semua komponen UI dari item layout item_movie.xml

Pada baris [27 – 30], membuat objek ViewHolder baru dengan menginflate layout item_movie.xml dan mengembalikannya.

Pada baris [31], RecyclerView bisa tahu berapa banyak item harus ditampilkan sesuai data yang ada

Pada baris [33 – 42], mengisi data ke tampilan berdasarkan posisi item di daftar dan juga mengatur listener pada tombol IMDB dan Detail yang memanggil fungsi onImdbClick dan onDetailClick dengan data lengkap film tersebut

HomeFragment.kt

Pada baris [1 – 11], mengimport komponen UI, intent dan binding layout dengan ViewBinding FragmentHomeBinding untuk akses view dari xml

Pada baris [13 - 16], mendeklarasikan class dan variable binding, _binding untuk menyimpan hasil inflasi dari FragmentHomeBinding dan binding non null untuk akses elemen UI

Pada baris [18 – 19]. Mendeklarasikan adapter dan data list untuk adapter RecyclerView dan list untuk menyimpan daftar film dari resource

Pada baris [21 – 32], inflate layout fragment menggunakan viewbinding, mengisi list dengan data dari getListMovie dan memanggil setupRecyclerView untuk mengatur tampilan daftar film

Pada baris [33 – 62], Membuat adapter dengan dua aksi yang berupa imdb click dan detail click yang mengoper data film melalui bundle, selain itu mengatur RecyclerView dengan LinearLayoutManager (daftar vertikal)

Pada baris [64 – 78], mengambil data dari resource (string.xml) untuk membuat objek Movie dan dataImage.recycle untuk melepaskan resource setelah digunakan.

Pada baris [80 – 83], membersihkan binding saat view dihancurkan untuk mencegah memory leak

DetailFragment.kt

Pada baris [1 – 8], mendeklarasi import untuk menampilkan UI detail dan mengimpor ViewBinding agar lebih aman dan ringkas saat mengakses view

Pada baris [11 – 14], mendeklarsi fragment dan binding untuk properti nullable yang diisi saat onCreateView dan binding untuk mengakses elemen UI tanpa nullable

Pada baris [16 – 20], menginflate layout XML fragment_detail.xml dengan View Binding

Pada baris [22 – 26], mengambil data yang dikirim dari HomeFragment melalui Bundle

Pada baris [28 – 34], mengisi elemen-elemen tampilan dengan data yang diterima dan gambar hanya diatur jika image tidak null.

Pada baris [38 – 41], membersihkan binding saat view dihancurkan untuk mencegah memory leak

***activity_main.xml**

Pada baris [1 – 9], membuat ID di Kotlin MainActivity sebagai tempat menampung fragment, match parent akan mengisi seluruh layar dan contextnya akan terkait dengan MainActivity

Terdapat 2 activity_activity main dengan isi yang sama tapi salah satu nya itu berfungsi sebagai landscape saat layar di rotate

***fragment_detail.xml**

Pada baris [10 – 19], <ImageView layout ini akan digunakan untuk penempatan gambar khusus poster film dengan id `img_poster` yang digunakan pada kode Kotlin. Peletakkannya di bagian atas layout dan rata kiri

Pada baris [21 – 31], <TextView layout ini digunakan untuk penempatan title dengan id `tv_item_title` yang berada tepat di bawah poster.

Pada baris [33 – 43], <TextView layout ini digunakan untuk penempatan tahun dengan id `tv_item_years` yang terletak di sebelah kanan judul

Pada baris [45 – 55], <TextView layout ini digunakan untuk penempatan label teks sebelum deskripsi plot dengan id `tv_item_plot`

Pada baris [57 – 67], <TextView layout ini digunakan untuk penempatan deskripsi film dengan id `tv_item_desc`

Terdapat 2 `fragment_detail` juga dengan isi yang sama tapi yang satu berfungsi untuk menjaga tampilan saat landscape atau layar di rotate

***fragment_home.xml**

Pada baris [1 – 19], membuat container fleksibel untuk penempatan berdasarkan constraint view dan ke parent, id `rv_movie` diakses di `HomeFragment` untuk menampilkan list movie dengan beberapa aturan lebar, tinggi, margin dan layout constraint untuk `Recycler view` memenuhi semua area parent dengan margin

***item_movie.xml**

Pada baris [1 – 19], membuat constraint layout `CardView` untuk mengatur posisi elemen secara fleksible dan memberi warna latar abu abu

Pada baris [21 – 29], <ImageView untuk membuat layout gambar dengan `scaletype center` agar gambar memenuhi area dan constraint ke atas, bawah, kiri parent

Pada baris [31 – 43], <TextView untuk membuat judul film meletakkannya disebelah kanan poster dan diapit `img_poster` dan `tv_item_years`

Pada baris [45 – 55], <TextView untuk membuat tahun film yang peletakannya di kanan judul dan menampilkan rilis filmnya kapan.

Pada baris [57 – 68], <TextView untuk membuat label plot di bawah judul dan dikanan poster

Pada baris [70 – 80], <TextView untuk membuat deskripsi film di sebelah label plot dan menampilkan ringkasan cerita film

Pada baris [82 – 91], <Button untuk tombol detail pindah ke DetailFragment di bawah deskripsi, dikanan bersebelahan dengan tombol IMDB

Pada baris [93 – 101], <Button untuk tombol pindah halaman ke link IMDB dan terletak di bawah deskripsi di kiri.

Jetpack Compose

***strings.xml**

Pada string ini, berisikan data data yang akan digunakan pada kode Kotlin yang akan dijelaskan setelahnya. StringArray ini berfungsi sebagai data yang akan digunakan untuk menampilkan film atau deskripsi dan semacamnya.

DataMovie.kt

Pada baris [1 – 4], menempatkan file di package data, mengimpor R untuk akses ke resource seperti R.string atau R.drawable. mengimpor ListMovie dari class model yang merepresentasikan satu film

Pada baris [6 -13], membuat fungsi loadMovieList untuk menghasilkan daftar film dari List<ListMovie>, mengembalikan data statis yang isinya objek ListMovie dan data ini akan digunakan untuk ditampilkan di UI di LazyColumn, listOf mempunyai format yang nilainya akan diambil dari strings.xml atau drawable

ListMovie.kt

Pada baris [1 – 4], menempatkan file di package model karena representasi data, @StringRes dan @DrawableRes untuk anotasi validasi tipe resource

Pada baris [6 – 13], membuat data class untuk menyimpan data film yang masing masing bertipe data Int karena akan mengambil dari resource ID.

DetailActivity.kt

Pada baris [1 – 19], membuat activity dasar compose dan mengimpor lainnya seperti elemen UI compose seperti Image, Text, Column, Row dan sebagainya

Pada baris [21 – 22], membuat activity yang digunakan untuk menampilkan detail film yang dipilih dari list, dan onCreate untuk format awal eksekusi UI

Pada baris [24 – 27], mengambil data dari intent dan data dikirim dalam bentuk resource ID sesuai @StringRes dan @DrawableRes

Pada baris [29 – 34], surface untuk memberi tempat dengan latar hitam dan MListComposeTheme membungkus UI agar mengikuti tema aplikasi

Pada baris [35 – 40], menggunakan column untuk menampilkan elemen secara vertikal

Pada baris [41 – 45], menampilkan gambar dari resource ID dan contentDescription diisi dengan judul film untuk aksesibilitas

Pada baris [46 – 70], menggunakan row agar judul dan tahun sejajar secara horizontal dan weight 1F pada judul agar mengisi ruang, dan tahun mengikuti di kanan

Pada baris [71 – 85], Text yang akan digunakan untuk label dan isi deskripsi film

MainActivity.kt

Pada baris [1 – 46], package yang menandakan bahwa file ini berada di package yang sama dan mengimport elemen elemen dari jetpack compose dan tema dari aplikasi MListComposeTheme

Pada baris [48 – 63], activity utama yang memanggil `MovieListScreen()` saat app dibuka dengan menggunakan tema compose dan meletakkan di surface untuk interface dengan background hitam. Data list film diambil dari `DataMovie().loadMovieList()`

Pada baris [64 – 72], `LazyColumn` membuat list scrollable dan `items(movie)` menampilkan setiap elemen dengan `MovieItems()`

Pada baris [73 – 77], mengambil context dari compose untuk keperluan Intent dan mengambil resource string dari Int resource ID (url, years)

Pada baris [79 – 183], membuat card berisi satu item film dengan sudut membulat dan warna abu abu gelap. Membuat baris yang digunakan untuk menata elemen secara horizontal dan colum yang digunakan untuk Menyusun elemen secara vertikal. Image untuk menampilkan gambar film dengan ukuran 90x200dp dan menggunakan gambar yang diberikan di `movie.image`. Spacer untuk memberi jarak antara elemen baris sebanyak 12dp. Text menampilkan teks yang ada pada elemen seperti judul film, tahun rilis, dan deskripsi plot dan disesuaikan `textsyle` seperti ukurant font dan ketebalan huruf. Fungsi button `onClick` untuk menampilkan tombol ketika membuka link IMDB dan tombol detail layar film. Selain itu ketika IMBD ditekan akan intent ke browser yang mengarahkan ke URL IMDB, detail akan diarahkan ke `DetailActivity` yang akan menampilkan informasi lebih detail mengenai film

Pada baris [184 – 190], membuat preview dari composable `MovieListScreen` untuk melihat tampilan aplikasi ketika dijalankan, selain itu juga memanggil `MovieListScreen` dengan data film yang dihasilkan oleh `DataMovie().loadMovieList()`

SOAL 2

Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

A. Pembahasan

RecyclerView sendiri masih digunakan karena salah satu komponen yang kuat dan serbaguna untuk menanganai kumpulan data besar dengan mendaur ulang dan menggunakan kembali. Komponen ini menawarkan fleksibilitas dalam tata letak item dan mendukung fitur-fitur seperti pengguliran halus, animasi item, dan penampung tampilan. RecyclerView juga telah banyak digunakan untuk daftar yang kompleks dan dinamis. Meskipun kekurangannya boiler-plate tapi akan sangat mudah untuk dikuasai meskipun itu boilerplate.

Dibandingkan dengan LazyColumn ini karena relative baru di jetpack maka harus membiasakan dengan kerangka kerja compose. Meskipun singkat tapi butuh waktu untuk memahami LazyColumn ini. Pada hal ini orang lebih memilih boilerplate tapi dinamis dan kompleks dibandingkan harus beradaptasi dengan LazyColumn yang tergolong baru. Tapi hal ini bukan berarti buruk bagi LazyColumn, hanya saja butuh waktu lagi untuk mempelajari hal baru.

MODUL 4 : VIEWMODEL AND DEBUGGING

SOAL 1

Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
- b. Gunakan ViewModelFactory untuk membuat parameter dengan tipe data String di dalam ViewModel
- c. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment
- d. Install dan gunakan library Timber untuk logging event berikut:
 - a. Log saat data item masuk ke dalam list
 - b. Log saat tombol Detail dan tombol Explicit Intent ditekan
 - c. Log data dari list yang dipilih ketika berpindah ke halaman Detail
- e. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi. Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out

A. Source Code

XML:

MainActivity.kt

Tabel 22. Source Code MainActivity.kt Soal 1 Modul 4 XML

1	package com.example.movielistxml
2	


```

3 import android.os.Bundle
4 import androidx.appcompat.app.AppCompatActivity
5 import timber.log.Timber
6
7 class MainActivity : AppCompatActivity() {
8     override fun onCreate(savedInstanceState:
9 Bundle?) {
10         super.onCreate(savedInstanceState)
11         setContentView(R.layout.activity_main)
12
13         val fragmentManager = supportFragmentManager
14         val homeFragment = HomeFragment()
15         val fragment =
16 fragmentManager.findFragmentByTag(HomeFragment::cla
17 ss.java.simpleName)
18         if (fragment !is HomeFragment) {
19             Timber.d("MyFragment", "Fragment Name :"+
20 HomeFragment::class.java.simpleName)
21             fragmentManager
22                 .beginTransaction()
23                 .add(R.id.frame_container,
24 homeFragment, HomeFragment::class.java.simpleName)
25                 .commit()
26         }
27     }
28 }

```

Movie.kt

Tabel 23. Source Code Movie.kt Soal 1 Modul 4 XML

1	package com.example.movielistxml
2	
3	import android.os.Parcelable
4	import kotlinx.parcelize.Parcelize
5	
6	@Parcelize
7	data class Movie (
8	val title: String,
9	val url: String,
10	val image: Int,
11	val plot: String,
12	val years: String,
13	val desc: String
14	): Parcelable

ListMovieAdapter.kt

Tabel 24. Source Code ListMovieAdapter.kt Soal 1 Modul 4 XML

1	package com.example.movielistxml
2	
3	import android.view.LayoutInflater
4	import android.view.View
5	import android.view.ViewGroup
6	import android.widget.Button
7	import android.widget.ImageView
8	import android.widget.TextView
9	import androidx.recyclerview.widget.RecyclerView

```

10
11 class ListMovieAdapter (
12     private val listMovie: ArrayList<Movie>,
13     private val onImdbClick: (String) -> Unit,
14     private val onDetailClick: (String, Int, String,
String, String) -> Unit
15 ):RecyclerView.Adapter<ListMovieAdapter.ListViewHol
16 der>() {
17
18     class ListViewHolder(itemView: View) :
RecyclerView.ViewHolder(itemView) {
19         val imgPoster: ImageView =
itemView.findViewById(R.id.img_poster)
20         val tvName: TextView =
itemView.findViewById(R.id.tv_item_title)
21         val years: TextView =
itemView.findViewById(R.id.tv_item_years)
22         val plot: TextView =
itemView.findViewById(R.id.tv_item_plot)
23         val desc: TextView =
itemView.findViewById(R.id.tv_item_desc)
24         val btnImdb: Button =
itemView.findViewById(R.id.btn_imdb)
25         val btnDetail: Button =
itemView.findViewById(R.id.btn_detail)
26     }
27     override fun onCreateViewHolder(parent:
ViewGroup, viewType: Int): ListViewHolder{

```

	<pre> val view: View = 28 LayoutInflater.from(parent.context).inflate(R.layout 29 t.item_movie, parent, false) 30 return ViewHolder(view) 31 } 32 override fun getItemCount(): Int = listMovie.size 33 override fun onBindViewHolder(holder: ViewHolder, position: Int){ 34 val (title, url, image, plot, years, desc) = 35 listMovie[position] 36 holder.tvName.text = title 37 holder.plot.text = plot 38 holder.imgPoster.setImageResource(image) 39 holder.years.text = years holder.desc.text = desc 40 holder.btnImdb.setOnClickListener{onImdbClick(url)} 41 42 holder.btnDetail.setOnClickListener{onDetailClick(t 43 itle, image, plot, years, desc)} 44 } 45 46 fun updateData(newList: List<Movie>) { 47 listMovie.clear() 48 listMovie.addAll(newList) notifyDataSetChanged() } } </pre>
--	---

HomeFragment.kt

Tabel 25. Source Code HomeFragment.kt Soal 1 Modul 4 XML

1	package com.example.movielistxml
2	
3	import android.content.Intent
4	import android.net.Uri
5	import android.os.Bundle
6	import androidx.fragment.app.Fragment
7	import android.view.LayoutInflater
8	import android.view.View
9	import android.view.ViewGroup
10	import androidx.fragment.app.viewModels
11	import androidx.lifecycle.lifecycleScope
12	import
13	androidx.recyclerview.widget.LinearLayoutManager
	import
14	com.example.movielistxml.databinding.FragmentHomeB
15	inding
16	import kotlinx.coroutines.flow.collectLatest
17	import kotlinx.coroutines.launch
18	import timber.log.Timber
19	
20	class HomeFragment : Fragment() {
21	
22	private var _binding: FragmentHomeBinding? =
23	null
24	private val binding get() = _binding!!
25	
26	

27	private lateinit var movieAdapter:
28	ListMovieAdapter
29	
30	private val viewModel: MovieViewModel by
31	viewModels {
32	MovieViewModelFactory("Akmal")
33	}
34	override fun onCreateView(
35	inflater: LayoutInflater, container:
36	ViewGroup?,
37	savedInstanceState: Bundle?
	): View {
38	_binding =
39	FragmentHomeBinding.inflate(inflater, container,
40	false)
41	return binding.root
42	}
43	
44	override fun onViewCreated(view: View,
45	savedInstanceState: Bundle?) {
46	super.onViewCreated(view,
47	savedInstanceState)
48	
49	
50	val movies = getListMovie()
51	viewModel.setMovieList(movies)
52	
53	viewLifecycleOwner.lifecycleScope.launch {

54	viewModel.movieList.collectLatest {
	list ->
55	Timber.i("Movie List updated with
	\${list.size} items")
56	
57	movieAdapter.updateData(ArrayList(list))
	}
58	}
	viewLifecycleOwner.lifecycleScope.launch {
59	viewModel.selectedMovie.collectLatest {
	movie ->
60	movie?.let {
	Timber.i("Movie selected:
61	\${movie.title}")
	val detailFragment =
62	DetailFragment().apply {
63	arguments = Bundle().apply
64	{
65	
	putString("EXTRA_NAME", movie.title)
66	putInt("EXTRA_PHOTO",
67	movie.image)
68	
69	putString("EXTRA_YEARS", movie.years)
70	
71	putString("EXTRA_PLOT", movie.plot)
72	
73	putString("EXTRA_DESC", movie.desc)
74	}
75	}

76	
77	parentFragmentManager.beginTransaction()
78	
	.replace(R.id.frame_container, detailFragment)
79	.addToBackStack(null)
80	.commit()
81	}
82	}
	}
83	}
84	private fun setupRecyclerView(){
85	movieAdapter = ListMovieAdapter(
86	arrayListOf(),
87	onImdbClick = {url ->
88	Timber.i("IMDB clicked with URL:
89	\$url")
90	val intent =
91	Intent(Intent.ACTION_VIEW, Uri.parse(url))
92	startActivity(intent)
93	},
94	onDetailClick = {title, _, _, _ ->
95	Timber.i("Detail clicked with
96	title: \$title")
97	val movie =
	viewModel.movieList.value.find { it.title == title
98	}
	movie?.let {
99	viewModel.selectMovie(it)
	}

100	<pre> }) binding.rvMovie.apply { layoutManager = LinearLayoutManager(context) adapter = movieAdapter setHasFixedSize(true) } } private fun getListMovie(): ArrayList<Movie>{ val dataMovie = resources.getStringArray(R.array.data_title) val dataLink = resources.getStringArray(R.array.data_link) val dataImage = resources.obtainTypedArray(R.array.data_image) val dataPlot = resources.getStringArray(R.array.plot) val dataYear = resources.getStringArray(R.array.data_years) val dataDesc = resources.getStringArray(R.array.data_description) val listMovie = ArrayList<Movie>() for (i in dataMovie.indices){ val movie = Movie(dataMovie[i], dataLink[i], dataImage.getResourceId(i, -1), dataPlot[i], dataYear[i], dataDesc[i]) listMovie.add(movie) } } </pre>
-----	--

	<pre> dataImage.recycle() return listMovie } override fun onDestroyView() { super.onDestroyView() _binding = null } } </pre>
--	---

DetailFragment.kt

Tabel 26. Source Code DetailFragment.kt Soal 1 Modul 4 XML

1	package com.example.movieslistxml
2	
3	import android.os.Bundle
4	import androidx.fragment.app.Fragment
5	import android.view.LayoutInflater
6	import android.view.View
7	import android.view.ViewGroup
8	import
	com.example.movieslistxml.databinding.FragmentDetail
9	Binding
10	
11	class DetailFragment : Fragment() {
12	
13	private var _binding: FragmentDetailBinding? =
14	null
15	private val binding get() = _binding!!
16	

17	override fun onCreateView(
18	inflater: LayoutInflater, container:
19	ViewGroup?,
	savedInstanceState: Bundle?
20	): View {
21	_binding =
22	FragmentDetailBinding.inflate(inflater, container,
23	false)
24	
25	val title =
26	arguments?.getString("EXTRA_NAME")
27	val image = arguments?.getInt("EXTRA_PHOTO")
28	val years =
29	arguments?.getString("EXTRA_YEARS")
30	val plot =
31	arguments?.getString("EXTRA_PLOT")
32	val desc =
33	arguments?.getString("EXTRA_DESC")
34	
35	binding.tvItemTitle.text = title
36	binding.tvItemPlot.text = years
37	binding.tvItemYears.text = plot
38	binding.tvItemDesc.text = desc
39	image?.let {
40	binding.imgPoster.setImageResource(it)
41	}
	return binding.root
	}
	override fun onDestroyView() {

	<pre> super.onDestroyView() _binding = null } }</pre>
--	---

MovieViewModel.kt

Tabel 27. Source Code MovieViewModel.kt Soal 1 Modul 4 XML

1	package com.example.movieslistxml
2	
3	import androidx.lifecycle.ViewModel
4	import kotlinx.coroutines.flow.MutableStateFlow
5	import kotlinx.coroutines.flow.StateFlow
6	import timber.log.Timber
7	
8	class MovieViewModel(private val username: String):
	ViewModel() {
9	private val _movieList =
	MutableStateFlow<List<Movie>>(emptyList())
10	val movieList: StateFlow<List<Movie>> get() =
11	_movieList
12	
	private val _selectedMovie =
13	MutableStateFlow<Movie?>(null)
	val selectedMovie: StateFlow<Movie?> get() =
14	_selectedMovie
15	
16	init {
	Timber.i("MovieViewModel created with
17	username: \$username")

	}
18	fun setMovieList(movies: List<Movie>) {
19	Timber.i("Movie List loaded With:
20	\${movies.size}")
21	_movieList.value = movies
22	}
23	
24	fun selectMovie(movie: Movie) {
25	Timber.i("Movie selected: \${movie.title}")
26	_selectedMovie.value = movie
27	}
28	}

MovieViewModelFactory.kt

Tabel 28. Source Code MovieViewModelFactory.kt Soal 1 Modul 4 XML

1	package com.example.movielistxml
2	
3	import androidx.lifecycle.ViewModel
4	
5	class MovieViewModelFactory(private val username:
6	String) :
7	androidx.lifecycle.ViewModelProvider.Factory {
8	override fun <T : ViewModel> create(modelClass:
9	Class<T>): T {
	if(modelClass.isAssignableFrom(MovieViewModel::cla
10	ss.java)) {
	@Suppress("UNCHECKED_CAST")

	return MovieViewModel(username) as T
	}
	throw IllegalArgumentException("Unknown
11	ViewModel class")
12	}
13	}

MyApplication.kt

Tabel 29. Source Code MyApplication.kt Soal 1 Modul 4 XML

1	package com.example.movielistxml
2	
3	import android.app.Application
4	import timber.log.Timber
5	
6	class MyApplication : Application() {
7	override fun onCreate() {
8	super.onCreate()
9	if (BuildConfig.DEBUG) {
10	Timber.plant(Timber.DebugTree())
11	}
12	}
13	}

activity_main.xml

Tabel 30. Source Code activity_main.xml Soal 1 Modul 4 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<FrameLayout
3	

4	xmlns:android="http://schemas.android.com/apk/res/a
5	ndroid"
	xmlns:tools="http://schemas.android.com/tools"
6	android:id="@+id/frame_container"
7	android:layout_width="match_parent"
8	android:layout_height="match_parent"
	tools:context=".MainActivity">
9	</FrameLayout>
10	

fragment_detail.xml

Tabel 31. Source Code fragment_detail.xml Soal 1 Modul 4 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
	xmlns:android="http://schemas.android.com/apk/res/a
3	ndroid"
	xmlns:tools="http://schemas.android.com/tools"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent"
6	xmlns:app="http://schemas.android.com/apk/res-
7	auto"
8	tools:context=".DetailFragment"
9	android:layout_margin="10dp">
10	
11	
12	<ImageView
13	android:id="@+id/img_poster"
14	android:layout_width="wrap_content"
15	android:layout_height="400dp"

16	android:layout_marginTop="12dp"
17	android:contentDescription="Poster"
18	tools:src="@drawable/nwh"
19	app:layout_constraintTop_toTopOf="parent"
20	
21	app:layout_constraintStart_toStartOf="parent"
22	</>
23	
24	<TextView
25	android:id="@+id/tv_item_title"
26	android:layout_width="350dp"
27	android:layout_height="wrap_content"
28	tools:text="Ini Judul"
29	
30	app:layout_constraintTop_toBottomOf="@id/img_poster
31	"
32	
33	app:layout_constraintStart_toStartOf="parent"
34	android:textSize="24sp"
35	android:textStyle="bold"
36	android:textColor="@color/black"
37	</>
38	
39	<TextView
40	android:id="@+id/tv_item_years"
41	android:layout_width="wrap_content"
	android:layout_height="wrap_content"
42	android:layout_marginTop="4dp"
43	tools:text="2020"
	android:textSize="20sp"

44	app:layout_constraintEnd_toEndOf="parent"
45	app:layout_constraintHorizontal_bias="1.0"
46	
47	app:layout_constraintStart_toEndOf="@id/tv_item_title"
48	
49	
50	app:layout_constraintTop_toBottomOf="@id/img_poster"
	" />
51	
52	<TextView
53	android:id="@+id/tv_item_plot"
54	android:layout_width="wrap_content"
55	android:layout_height="wrap_content"
56	tools:text="Plot :"
57	
58	app:layout_constraintStart_toStartOf="parent"
59	
60	app:layout_constraintTop_toBottomOf="@id/tv_item_title"
61	
	android:textSize="16sp"
62	android:textStyle="bold"
63	android:textColor="@color/black"
64	/>
65	
66	<TextView
	android:id="@+id/tv_item_desc"
67	android:layout_width="wrap_content"
68	android:layout_height="wrap_content"
	tools:text="deskripsi"

	<pre> app:layout_constraintTop_toBottomOf="@id/tv_item_plot" app:layout_constraintStart_toStartOf="parent" android:textSize="16sp" android:textStyle="bold" android:textColor="@color/black" /> </androidx.constraintlayout.widget.ConstraintLayout> </pre>
--	---

fragment_home.xml

Tabel 32. Source Code fragment_home.xml Soal 1 Modul 4 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	xmlns:tools="http://schemas.android.com/tools"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent"
8	tools:context=".HomeFragment">
9	
10	
11	<androidx.recyclerview.widget.RecyclerView
12	android:id="@+id/rv_movie"
13	android:layout_width="0dp"
14	android:layout_height="0dp"

15	android:layout_margin="15dp"
16	
17	app:layout_constraintBottom_toBottomOf="parent"
18	app:layout_constraintEnd_toEndOf="parent"
19	
	app:layout_constraintStart_toStartOf="parent"
	app:layout_constraintTop_toTopOf="parent" />
	</androidx.constraintlayout.widget.ConstraintLayout
	>

item_movie.xml

Tabel 33. Source Code item_movie.xml Soal 1 Modul 4 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.cardview.widget.CardView
3	xmlns:android="http://schemas.android.com/apk/res/
4	android"
5	xmlns:card_view="http://schemas.android.com/apk/re
6	s-auto"
7	xmlns:tools="http://schemas.android.com/tools"
8	android:id="@+id/card_view"
9	android:layout_width="match_parent"
10	android:layout_height="wrap_content"
11	android:layout_gravity="center"
12	android:layout_marginStart="8dp"
13	android:layout_marginTop="4dp"
14	android:layout_marginEnd="8dp"
15	android:layout_marginBottom="4dp"

16	card_view:cardCornerRadius="4dp">
17	
18	
19	<androidx.constraintlayout.widget.ConstraintLayout
20	android:layout_width="match_parent"
21	android:layout_height="wrap_content"
22	android:padding="8dp"
23	android:background="@color/gray">
24	
25	<ImageView
	android:id="@+id/img_poster"
26	android:layout_width="70dp"
	android:layout_height="100dp"
27	android:scaleType="centerCrop"
28	
29	card_view:layout_constraintBottom_toBottomOf="parent"
30	
31	
32	card_view:layout_constraintStart_toStartOf="parent"
33	
34	
35	card_view:layout_constraintTop_toTopOf="parent"
36	tools:src="@drawable/nwh" />
37	
38	<TextView
39	android:id="@+id/tv_item_title"
40	android:layout_width="200dp"
	android:layout_height="wrap_content"
41	android:textColor="@color/white"
	android:layout_marginStart="12dp"

42	android:textSize="24sp"
43	android:textStyle="bold"
44	tools:layout_editor_absoluteX="78dp"
45	tools:layout_editor_absoluteY="2dp"
46	
47	card_view:layout_constraintStart_toEndOf="@id/img_
48	poster"
49	
50	card_view:layout_constraintEnd_toStartOf="@id/tv_i
51	tem_years"
52	tools:text="Ini Judul" />
53	
	<TextView
54	android:id="@+id/tv_item_years"
55	android:layout_width="wrap_content"
56	android:layout_height="wrap_content"
57	android:layout_marginStart="16dp"
58	android:layout_marginTop="12dp"
59	tools:text="2020"
60	android:textColor="@color/white"
61	
62	card_view:layout_constraintTop_toTopOf="parent"
63	
64	card_view:layout_constraintStart_toEndOf="@id/tv_i
65	tem_title"
66	tools:layout_editor_absoluteY="29dp" />
67	<TextView
	android:id="@+id/tv_item_plot"
68	android:layout_width="wrap_content"

69	android:layout_height="wrap_content"
70	android:layout_marginTop="8dp"
71	android:layout_marginStart="12dp"
72	tools:text="Plot :"
73	android:textColor="@color/white"
74	android:textSize="15sp"
75	android:textStyle="bold"
76	card_view:layout_constraintStart_toEndOf="@id/img_poster"
77	card_view:layout_constraintTop_toBottomOf="@id/tv_item_title" />
78	
79	
80	<TextView
81	android:id="@+id/tv_item_desc"
82	android:layout_width="0dp"
83	android:layout_height="wrap_content"
84	android:layout_marginStart="12dp"
85	android:textColor="@color/white"
86	
87	card_view:layout_constraintTop_toTopOf="@id/tv_item_plot"
88	
89	card_view:layout_constraintStart_toEndOf="@id/tv_item_plot"
90	
91	card_view:layout_constraintEnd_toEndOf="parent"
	tools:layout_editor_absoluteY="42dp"
	tools:text="Deskripsi Film" />

92	
93	<pre> <Button </pre>
94	<pre> android:id="@+id/btn_detail" </pre>
95	<pre> android:layout_width="wrap_content" </pre>
96	<pre> android:layout_height="wrap_content" </pre>
97	<pre> android:layout_marginTop="4dp" </pre>
98	<pre> android:text="@string/btn_detail" </pre>
99	
	<pre> card_view:layout_constraintEnd_toEndOf="parent" </pre>
100	
	<pre> card_view:layout_constraintHorizontal_bias="0.508" </pre>
101	
	<pre> card_view:layout_constraintStart_toEndOf="@id/btn_ </pre>
102	<pre> imdb" </pre>
103	
104	<pre> card_view:layout_constraintTop_toBottomOf="@+id/tv _item_desc" /> </pre>
	<pre> <Button </pre>
	<pre> android:id="@+id/btn_imdb" </pre>
	<pre> android:layout_width="wrap_content" </pre>
	<pre> android:layout_height="wrap_content" </pre>
	<pre> android:layout_marginStart="144dp" </pre>
	<pre> android:layout_marginTop="4dp" </pre>
	<pre> android:text="@string/btn_imdb" </pre>
	<pre> card_view:layout_constraintStart_toStartOf="parent " </pre>

	<pre> card_view:layout_constraintTop_toBottomOf="@id/tv_ item_desc" /> </androidx.constraintlayout.widget.ConstraintLayou t> </androidx.cardview.widget.CardView> </pre>
--	--

Compose:

String.xml

Tabel 34. Source String Data Soal 1 Modul 4 Compose

	<pre> <resources> <string name="app_name">Movie List</string> <string name="title1">Iron Man 3</string> <string name="title2">Captain America: Civil War</string> <string name="title3">Avengers: Endgame</string> <string name="title4">Avengers: Infinity War</string> <string name="title5">Spider-man: No Way Home</string> <string name="releaseDate1">2013</string> <string name="releaseDate2">2016</string> <string name="releaseDate3">2019</string> <string name="releaseDate4">2018</string> <string name="releaseDate5">2021</string> </pre>
--	--

	<string name="desc1">Brilliant industrialist Tony Stark / Iron Man faces an enemy that knows no bounds. Tony world is destroyed and he investigates who is responsible for the destruction. Armed with his tools and instincts, Tony sets out to save those he loves. An old question comes back to haunt him; did humans make the Iron Man suit or did the suit define his identity as a human?</string> <string name="desc2">The heavy civilian casualties have led the government to control the superheroes. This heated situation has Iron Man and Captain America camps opposing each other. The battle is inevitable.</string> <string name="desc3">The remaining Avengers try to reverse Thanos actions that destroyed half the universe. They travel through time to collect the Infinity Stones, then use the energy of the stones to go back in time and change the events that caused the Snap.</string> <string name="desc4">The Avengers and their allies must be willing to sacrifice everything in an attempt to defeat the powerful Thanos before he succeeds in destroying the universe.</string> <string name="desc5">For the first time in Spider-Man big-screen history, the affable hero true identity is revealed, causing his superpower responsibilities to clash with his normal life, and putting everyone closest to him at risk.</string>
--	---

	<pre> <string name="description1">Brilliant industrialist Tony Stark / Iron Man faces an enemy that knows no bounds. Tony world is destroyed and he investigates who is responsible for the destruction. Armed with his tools and instincts, Tony sets out to save those he loves. An old question comes back to haunt him; did humans make the Iron Man suit or did the suit define his identity as a human?</string> <string name="description2">The heavy civilian casualties have led the government to control the superheroes. This heated situation has Iron Man and Captain America camps opposing each other. The battle is inevitable.</string> <string name="description3">The remaining Avengers try to reverse Thanos actions that destroyed half the universe. They travel through time to collect the Infinity Stones, then use the energy of the stones to go back in time and change the events that caused the Snap.</string> <string name="description4">The Avengers and their allies must be willing to sacrifice everything in an attempt to defeat the powerful Thanos before he succeeds in destroying the universe.</string> <string name="description5">For the first time in Spider-Man big-screen history, the affable hero true identity is revealed, causing his superpower responsibilities to clash with his normal life, and putting everyone closest to him at risk.</string> </resources> </pre>
--	---

DataMovie.kt

Tabel 35. Source String Code DataMovie.kt Soal 1 Modul 4 Compose

1	package com.example.mlistcompose.data
2	
3	import com.example.mlistcompose.R
4	import com.example.mlistcompose.model.ListMovie
5	
6	class DataMovie{
7	fun loadMovieList(): List<ListMovie> = listOf(
8	ListMovie(R.string.title1,
	"https://www.imdb.com/title/tt1300854/?ref_=nv_sr_s
	rsg_0_tt_8_nm_0_in_0_q_Iron%2520Man%25203" ,
	R.drawable.ironman3, R.string.releaseDate1,
	R.string.desc1, R.string.description1),
9	ListMovie(R.string.title2,
	"https://www.imdb.com/title/tt3498820/?ref_=nv_sr_s
	rsg_0_tt_6_nm_0_in_0_q_captain%2520civil%2520war",
	R.drawable.civilwar, R.string.releaseDate2,
	R.string.desc2, R.string.description2),
10	ListMovie(R.string.title3,
	"https://www.imdb.com/title/tt4154756/?ref_=nv_sr_s
	rsg_0_tt_8_nm_0_in_0_q_infinity%2520war",
	R.drawable.infinitywar, R.string.releaseDate3,
	R.string.desc3, R.string.description3),
11	ListMovie(R.string.title4,
	"https://www.imdb.com/title/tt4154796/?ref_=nv_sr_s
	rsg_0_tt_8_nm_0_in_0_q_endgame", R.drawable.endgame,
	R.string.releaseDate4, R.string.desc4,
	R.string.description4),
12	

	<pre> ListMovie(R.string.title5, "https://www.imdb.com/title/tt10872600/?ref_=nv_sr_ srsg_0_tt_8_nm_0_in_0_q_no%2520way", R.drawable.nwh, 13 R.string.releaseDate5, R.string.desc5, 14 R.string.description5)) } </pre>
--	--

ListMovie.kt

Tabel 36. Source Code ListMovie.kt Soal 1 Modul 4 Compose

1	package com.example.mlistcompose.model
2	
3	import androidx.annotation.DrawableRes
4	import androidx.annotation.StringRes
5	
6	data class ListMovie(
7	@StringRes val title: Int,
8	val url: String,
9	@DrawableRes val image: Int,
10	@StringRes val years: Int,
11	@StringRes val desc: Int,
12	@StringRes val description: Int
13	)

DetailActivity.kt

Tabel 37. Source Code DetailActivity.kt Soal 1 Modul 4 Compose

1	package com.example.mlistcompose
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.compose.foundation.Image
7	import androidx.compose.material3.Surface
8	import androidx.compose.material3.Text
9	import androidx.compose.ui.Alignment
10	import androidx.compose.ui.Modifier
11	import androidx.compose.foundation.layout.*
12	import androidx.compose.ui.graphics.Color
13	import androidx.compose.ui.res.painterResource
14	import androidx.compose.ui.res.stringResource
15	import androidx.compose.ui.text.TextStyle
16	import androidx.compose.ui.text.font.FontWeight
17	import androidx.compose.ui.unit.dp
18	import androidx.compose.ui.unit.sp
19	import
20	com.example.mlistcompose.ui.theme.MListComposeTheme
21	
22	class DetailActivity : ComponentActivity() {
23	override fun onCreate(savedInstanceState:
24	Bundle?) {
25	super.onCreate(savedInstanceState)
26	val title = intent.getIntExtra("title", 0)
27	val years = intent.getIntExtra("years", 0)
	val image = intent.getIntExtra("image", 0)

28	val	description	=
29	intent.getIntExtra("description", 0)		
30			
31	setContent {		
32	MListComposeTheme {		
33	Surface(		
34	modifier		=
35	Modifier.fillMaxSize(),		
36	color = Color.Black		
37	) {		
38	Column (		
39	modifier = Modifier		
40	.fillMaxWidth()		
41	.height(200.dp)		
42	.padding(16.dp)		
	){		
43	Image(		
44	modifier		=
	Modifier.height(225.dp).fillMaxWidth(),		
45	painter		=
46	painterResource(image),		
47	contentDescription		=
48	stringResource(title)		
49	)		
50	Row (		
51	modifier = Modifier		
52	.fillMaxWidth()		
53	){		
54	Text(		

55		text	=
56	stringResource(title),		
57		style = TextStyle(	
		fontSize	=
58	40.sp,		
59		fontWeight	=
60	FontWeight.Bold,		
61		color	=
62	Color.White		
63		),	
64		modifier	=
65	Modifier.padding(bottom = 4.dp)		
66		.weight(1F)	
		)	
		Text(	
67		text	=
	stringResource(years),		
68		style = TextStyle(	
69		fontSize	=
70	24.sp,		
71		color	=
72	Color.White		
73		),	
74		modifier	=
75	Modifier.padding(bottom = 4.dp)		
76	.wrapContentWidth(Alignment.End)		
77			
78	.align(Alignment.CenterVertically)		
79		)	

80	}
	Text(
81	text = "Plot :",
82	style = TextStyle(
83	fontSize =
84	16.sp,
85	fontWeight =
86	FontWeight.Bold,
87	color =
88	Color.White
89	)
90	)
91	Text(
	text =
	stringResource(description),
	style = TextStyle(
	fontSize =
	16.sp,
	color =
	Color.White
	)
	)
	}
	}
	}
	}
	}
	}

MainActivity.kt

Tabel 38. Source Code MainActivity.kt Soal 1 Modul 4 Compose

1	package com.example.mlistcompose
2	
3	import android.content.Intent
4	import android.os.Bundle
5	import androidx.activity.ComponentActivity
6	import androidx.activity.compose.setContent
7	import androidx.activity.enableEdgeToEdge
8	import androidx.activity.viewModels
9	import androidx.compose.foundation.Image
10	import androidx.compose.foundation.layout.*
11	import androidx.compose.foundation.lazy.LazyColumn
12	import androidx.compose.foundation.lazy.items
13	import
14	androidx.compose.foundation.shape.RoundedCornerSha
15	pe
16	import androidx.compose.material3.*
17	import androidx.compose.runtime.Composable
18	import androidx.compose.runtime.collectAsState
19	import androidx.compose.runtime.getValue
20	import androidx.compose.ui.Alignment
21	import androidx.compose.ui.Modifier
22	import androidx.compose.ui.draw.clip
23	import androidx.compose.ui.graphics.Color
24	import androidx.compose.ui.layout.ContentScale
25	import androidx.compose.ui.platform.LocalContext
26	import androidx.compose.ui.res.painterResource
27	import androidx.compose.ui.res.stringResource
28	import androidx.compose.ui.text.TextStyle

29	import androidx.compose.ui.text.font.FontWeight
30	import androidx.compose.ui.unit.dp
31	import androidx.compose.ui.unit.sp
32	import androidx.core.net.toUri
33	import com.example.mlistcompose.model.ListMovie
34	import
35	com.example.mlistcompose.ui.theme.MListComposeThem
36	e
37	import timber.log.Timber
38	
39	class MainActivity : ComponentActivity() {
40	
41	private val viewModel: MovieViewModel by
42	viewModels {
43	MovieViewModelFactory("Test Debugging")
44	}
45	
46	override fun onCreate(savedInstanceState:
47	Bundle?) {
48	super.onCreate(savedInstanceState)
49	enableEdgeToEdge()
50	Timber.plant(Timber.DebugTree())
51	setContent {
52	MListComposeTheme {
53	Surface(
54	modifier
55	Modifier.fillMaxSize(),
56	color = Color.Black
57	) {
58	MovieListScreen(viewModel)

59	}
60	}
61	}
	}
62	}
63	@Composable
64	fun MovieListScreen(viewModel: MovieViewModel) {
65	val movieList by
66	viewModel.movies.collectAsState()
67	val navigateToUrl by
68	viewModel.navigateToUrl.collectAsState()
69	val navigateToDetail by
70	viewModel.navigateToDetail.collectAsState()
71	val context = LocalContext.current
72	
	navigateToUrl?.let { url ->
73	val intent = Intent(Intent.ACTION_VIEW,
74	url.toUri())
75	context.startActivity(intent)
76	viewModel.onUrlNavigated()
77	}
78	
79	navigateToDetail?.let { movie ->
80	val intent = Intent(context,
81	DetailActivity::class.java).apply {
82	putExtra("title", movie.title)
83	putExtra("years", movie.years)
84	putExtra("image", movie.image)
85	

86	putExtra("description",	
87	movie.description)	
88	}	
89	context.startActivity(intent)	
	viewModel.onDetailNavigated()	
90	}	
91	LazyColumn(	
92	contentPadding = PaddingValues(16.dp),	
93	verticalArrangement	=
94	Arrangement.spacedBy(16.dp)	
95	) {	
96	items(movieList) { movie ->	
97	MovieItems(	
98	movie = movie,	
99	onImdbClicked	= {
100	viewModel.onLinkClicked(movie.url) },	
101	onDetailClicked	= {
102	viewModel.onDetailClicked(movie) }	
103	)	
104	}	
105	}	
106	}	
107		
108	@Composable	
109	fun MovieItems(	
	movie: ListMovie,	
110	onImdbClicked: () -> Unit,	
111	onDetailClicked: () -> Unit	
112	) {	

113	val releaseDate = stringResource(movie.years)	
114		
115	Card(	
116	shape = RoundedCornerShape(16.dp),	
117	modifier = Modifier	
118	.fillMaxWidth()	
119	.wrapContentHeight(),	
	colors	=
120	CardDefaults.cardColors(containerColor	=
121	Color(0xFF343539)),	
122	elevation	=
123	CardDefaults.cardElevation(4.dp)	
124	) {	
125	Row(	
126	modifier = Modifier	
127	.padding(16.dp)	
128	.fillMaxWidth()	
130	) {	
131	Image(	
132	painter	=
133	painterResource(movie.image),	
134	contentDescription	=
135	stringResource(movie.title),	
136	contentScale = ContentScale.Crop,	
137	modifier = Modifier	
138	.size(width = 90.dp, height =	
139	200.dp)	
140		
141	.clip(RoundedCornerShape(8.dp))	
142		

143	
144	.align(Alignment.CenterVertically)
145	)
146	Spacer(modifier =
147	Modifier.width(12.dp))
148	Column(
149	modifier = Modifier
150	.fillMaxWidth()
151	
	.align(Alignment.CenterVertically)
152	) {
153	Row(
154	modifier = Modifier
	.fillMaxWidth()
155	.padding(8.dp)
156	) {
157	Text(
158	text =
159	stringResource(movie.title),
160	style = TextStyle(
161	fontSize = 26.sp,
162	fontWeight =
163	FontWeight.Bold,
164	color = Color.White
165	),
166	modifier = Modifier
167	.padding(bottom = 4.dp)
168	.weight(1f)
169	)
170	Text(

171	text = releaseDate,
	style = TextStyle(fontSize
172	= 20.sp, color = Color.White),
	modifier = Modifier
173	.padding(bottom = 4.dp)
174	
175	.wrapContentWidth(Alignment.End)
176	
177	.align(Alignment.CenterVertically)
178	)
179	}
180	Row(
181	modifier =
182	Modifier.fillMaxWidth()
183	) {
184	Text(
185	text = "Plot :",
186	style = TextStyle(
187	fontSize = 16.sp,
188	fontWeight =
189	FontWeight.Bold,
190	color = Color.White
	)
	)
	Text(
	text =
	stringResource(movie.desc),
	style = TextStyle(fontSize
	= 16.sp, color = Color.White),

	<pre> modifier = Modifier.padding(bottom = 8.dp)) } Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.End) { Button(onClick = onImdbClicked) { Text("IMDB") } Spacer(modifier = Modifier.width(4.dp)) Button(onClick = onDetailClicked) { Text("Detail") } } } }</pre>
--	--

MovieViewModel.kt

Tabel 39. Source Code MovieViewModel.kt Soal 1 Modul 4 Compose

1	<pre>package com.example.mlistcompose</pre>
---	---

2	
3	import androidx.lifecycle.ViewModel
4	import com.example.mlistcompose.data.DataMovie
5	import com.example.mlistcompose.model.ListMovie
6	import kotlinx.coroutines.flow.MutableStateFlow
7	import kotlinx.coroutines.flow.StateFlow
8	import kotlinx.coroutines.flow.asStateFlow
9	import timber.log.Timber
10	
11	class MovieViewModel(someParam: String) :
12	ViewModel() {
	private val _movies =
13	MutableStateFlow<List<ListMovie>>(emptyList())
	val movies: StateFlow<List<ListMovie>> =
14	_movies.asStateFlow()
15	
	private val _navigateToUrl =
16	MutableStateFlow<String?>(null)
	val navigateToUrl: StateFlow<String?> =
17	_navigateToUrl.asStateFlow()
18	
	private val _navigateToDetail =
19	MutableStateFlow<ListMovie?>(null)
	val navigateToDetail: StateFlow<ListMovie?> =
20	_navigateToDetail.asStateFlow()
21	init {
22	Timber.d("Loading movie list with param:
23	\$someParam")
	_movies.value = DataMovie().loadMovieList()
24	

25	Timber.d("Movie list loaded with
26	\${_movies.value.size} items")
27	}
28	
29	fun onLinkClicked(url: String) {
30	Timber.d("URL clicked: \$url")
31	_navigateToUrl.value = url
32	}
33	
34	fun onDetailClicked(movie: ListMovie) {
35	Timber.d("Detail clicked for movie:
36	\${movie.title}")
37	_navigateToDetail.value = movie
38	}
39	
40	fun onUrlNavigated() {
41	_navigateToUrl.value = null
42	}
43	
	fun onDetailNavigated() {
	_navigateToDetail.value = null
	}
	}

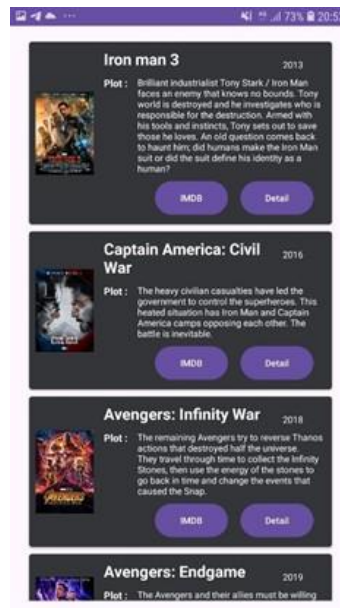
MovieViewModelFactory.kt

Tabel 40. Source Code MovieViewModelFactory.kt Soal 1 Modul 4 Compose

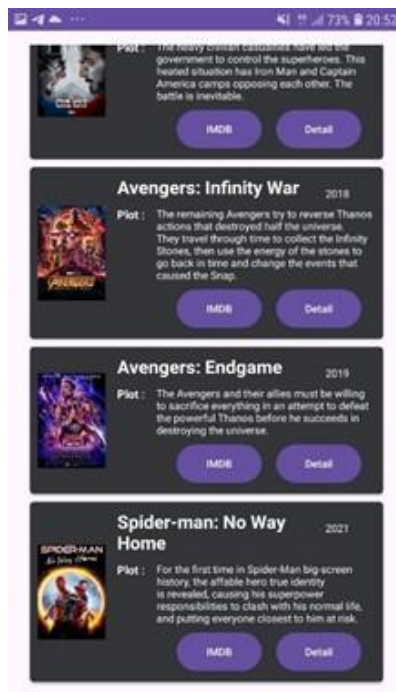
1	package com.example.mlistcompose
2	
3	import androidx.lifecycle.ViewModel

4	import androidx.lifecycle.ViewModelProvider
5	
6	class MovieViewModelFactory(private val someParam: String) : ViewModelProvider.Factory {
7	override fun <T : ViewModel> create(modelClass: Class<T>): T {
8	if (modelClass.isAssignableFrom(MovieViewModel::class. java)) {
9	
10	@Suppress("UNCHECKED_CAST")
11	return MovieViewModel(someParam) as T
12	}
	throw IllegalArgumentException("Unknown ViewModel class")
13	}
14	}

B. Output Program



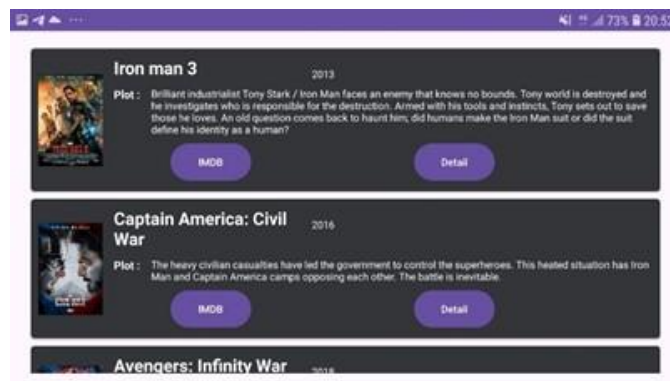
Gambar 33. Screenshot Hasil Tampilan Awal Soal 1 Modul 4 XML



Gambar 34. Screenshot Hasil Tampilan Ketika di Scroll Soal 1 Modul 4 XML



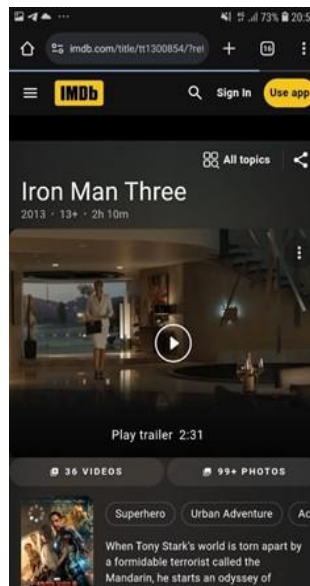
Gambar 35. Screenshot Hasil Tampilan Tombol Detail Soal 1 Modul 4 XML



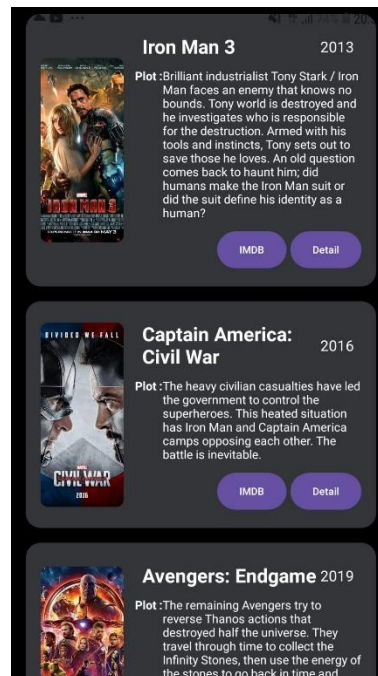
Gambar 36. Screenshot Hasil Tampilan List (landscape) Soal 1 Modul 4 XML



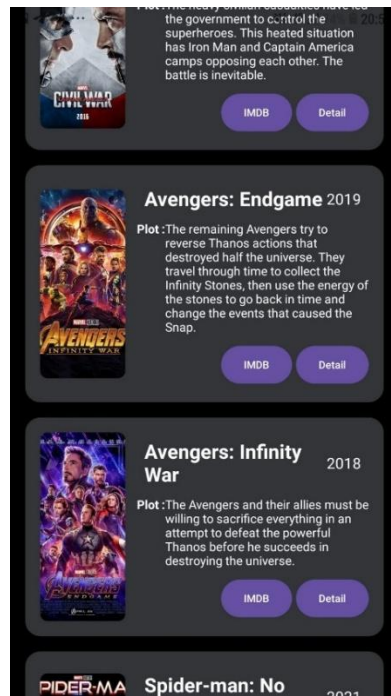
Gambar 37. Screenshot Hasil Tampilan Detail (landscape) Soal 1 Modul 4 XML



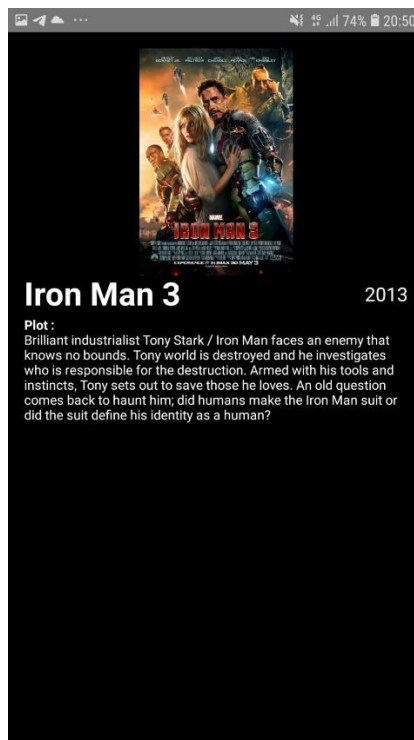
Gambar 38. Screenshot Hasil Tampilan Tombol IMDB Soal 1 Modul 4



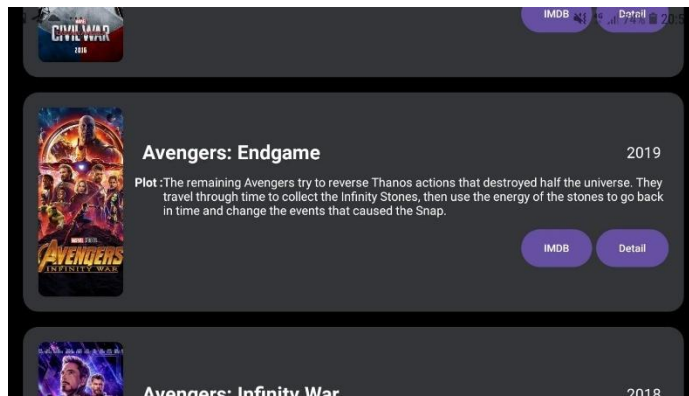
Gambar 39. Screenshot Hasil Tampilan Awal Soal 1 Modul 4 Compose



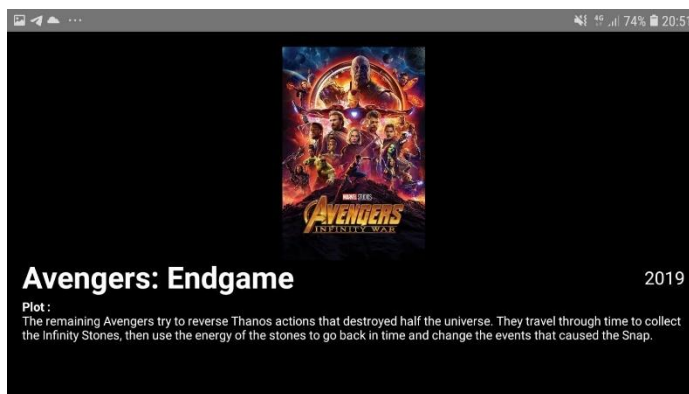
Gambar 40. Screenshot Hasil Tampilan Ketika di Scroll Soal 1 Modul 4 Compose



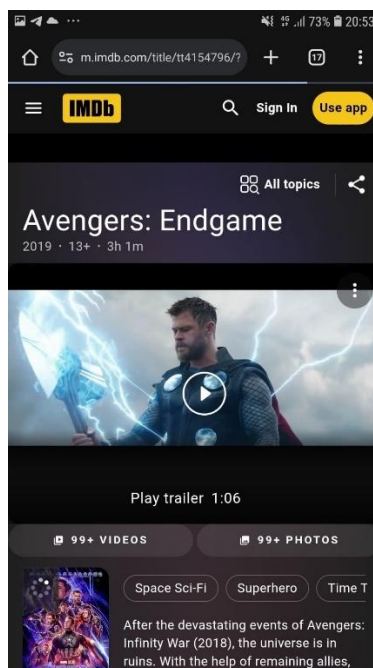
Gambar 41. Screenshot Hasil Tampilan Tombol Detail Soal 1 Modul 4 Compose



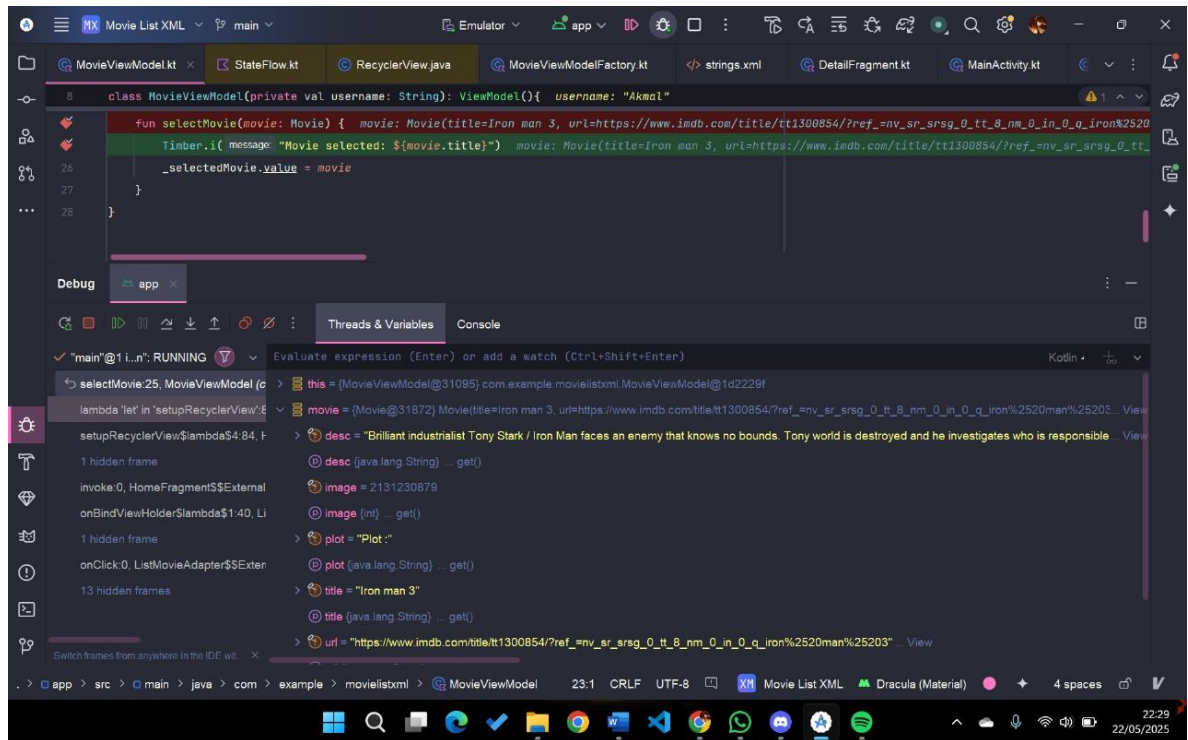
Gambar 42. Screenshot Hasil Tampilan List (landscape) Soal 1 Modul 4 Compose



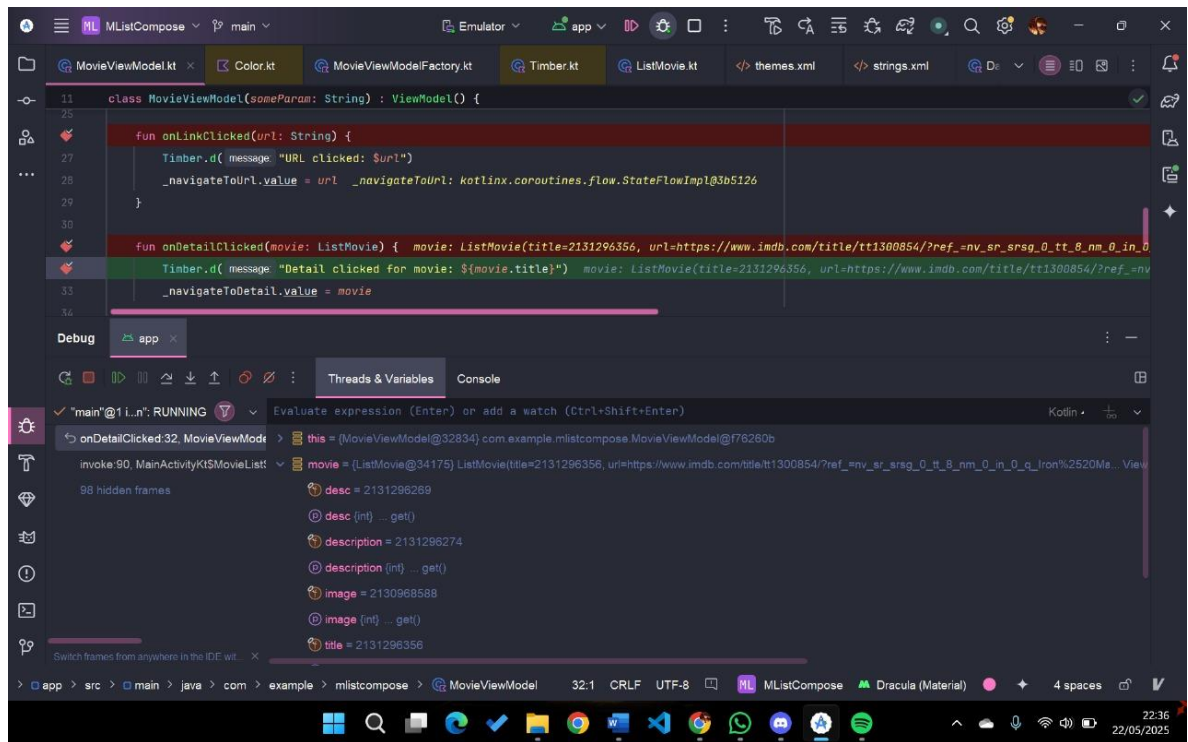
Gambar 43. Screenshot Hasil Tampilan Detail (landscape) Soal 1 Modul 4 Compose



Gambar 44. Screenshot Hasil Tampilan Tombol IMDB Soal 1 Modul 4



Gambar 45. Gambar Debugger Di XML Modul 4



Gambar 46. Gambar Debugger Di Jetpack Compose Modul 4

C. Pembahasan

XML:

MainActivity.kt

Pada baris [1 – 5], mendeklarasikan package dan import yang digunakan untuk definisi file merupakan bagian dari package com.example.movielistxml

Pada baris [7], mendeklarasikan activity utama dari AppCompatActivity, yang merupakan MainActivity adalah layar utama aplikasi

Pada baris [8 – 23], method onCreate mengatur layout activity dengan setContentView, lalu menggunakan fragmentManager untuk memeriksa apakah HomeFragment sudah ada. Jika belum ada, HomeFragment akan dibuat dan

ditambahkan ke container `frame_container` dengan tag nama kelasnya, serta mencatat log debug menggunakan Timber.

Movie.kt

Pada baris [1 – 4]. Mendeklarasikan file dalam package yang sama dan interface untuk memungkinkan objek dikirim antar komponen android antar activity atau fragment. Parcelize untuk anotasi dari Kotlin extension untuk menghindari penulisan parcelable secara manual.

Pada baris [6 – 14], menginstruksikan Kotlin secara otomatis menghasilkan parcelable untuk kelas ini dan data yang digunakan untuk menyimpan dan mengelola data secara sederhana. Terdapat beberapa properti seperti title, url, image, plot, years, dan desc. Parcelable menandakan bahwa objek movie bisa diparcel dan dikirim dengan intent atau bundle.

ListMovieAdapter.kt

Pada baris [1 – 9], package aplikasi dideklarasikan dan beberapa kelas Android seperti `LayoutInflater`, `View`, `ViewGroup`, `Button`, `ImageView`, `TextView`, serta `RecyclerView` diimpor untuk membuat adapter daftar film menggunakan `RecyclerView`. Pada baris [12 – 16], berisi class yang berisi `listMovie` sebagai daftar film yang akan ditampilkan, lambda function yang menerima url dan detail film saat di tekan. Selain itu mewarisi `RecyclerView.Adapter` dan `ViewHolder` sebagai `ViewHolder`nya

Pada baris [11 – 15], dibuat kelas `ListMovieAdapter` yang merupakan adapter `RecyclerView` untuk menampilkan daftar film dengan parameter `listMovie` sebagai data film dan dua fungsi callback untuk tombol IMDB dan Detail agar bisa mengirim data saat tombol diklik.

Pada baris [17 - 25], dibuat kelas `ViewHolder` yang menyimpan referensi ke elemen-elemen tampilan di layout item `RecyclerView`, seperti gambar poster, judul film, tahun, plot, deskripsi, dan dua tombol untuk IMDB serta Detail.

Pada baris [26 - 29], fungsi onCreateViewHolder membuat dan mengembalikan objek ViewHolder dengan meng-inflate layout item_movie sebagai tampilan tiap item dalam RecyclerView.

Pada baris [30], berfungsi untuk mengembalikan jumlah item dalam listMovie, yang digunakan RecyclerView untuk mengetahui berapa banyak item yang harus ditampilkan.

Pada baris [32 - 41], fungsi onBindViewHolder mengatur data film ke tampilan item RecyclerView, seperti judul, plot, gambar, tahun, dan deskripsi. Tombol IMDb dan Detail diberikan aksi klik untuk memicu fungsi yang mengirim data URL dan detail film.

Pada baris [43 - 47], fungsi updateData digunakan untuk memperbarui daftar film di adapter dengan data baru, lalu memberi tahu RecyclerView agar tampilan diperbarui sesuai data terbaru.

HomeFragment.kt

Pada baris [1 - 16], terdapat import berbagai library yang dibutuhkan untuk fragment, pengelolaan lifecycle, tampilan RecyclerView, data binding, coroutine, dan logging menggunakan Timber.

Pada baris [18 - 27], didefinisikan kelas HomeFragment yang merupakan subclass Fragment. Ada variabel _binding untuk binding layout yang diakses lewat properti binding. Juga dideklarasikan movieAdapter untuk RecyclerView dan viewModel yang diinisialisasi menggunakan MovieViewModelFactory dengan parameter "Akmal".

Pada baris [29 - 35], fungsi onCreateView dibuat untuk meng-inflate layout fragment menggunakan FragmentHomeBinding. Binding ini disimpan di _binding, lalu mengembalikan binding.root sebagai tampilan fragment yang akan ditampilkan.

Pada baris [37 - 70], fungsi onViewCreated mengatur RecyclerView dengan memanggil setupRecyclerView(). Kemudian data film diambil dari getListMovie()

dan dikirim ke ViewModel. Dua coroutine diluncurkan untuk mengamati perubahan data film (movieList) dan film yang dipilih (selectedMovie). Saat daftar film diperbarui, adapter diperbarui dengan data baru. Jika ada film yang dipilih, aplikasi akan membuat dan menampilkan DetailFragment dengan data film tersebut, lalu mengganti fragment saat ini dengan fragment detail dan menambahkan transaksi ke back stack.

Pada baris [73 - 94], fungsi setupRecyclerView membuat adapter ListMovieAdapter dengan list kosong dan dua callback: saat tombol IMDB diklik, aplikasi membuka browser ke URL film, dan saat tombol Detail diklik, aplikasi mencari film berdasarkan judul lalu memilih film tersebut di ViewModel. Setelah itu, RecyclerView diatur dengan LinearLayoutManager, adapter yang sudah dibuat, dan pengaturan ukuran tetap agar performa lebih baik.

Pada baris [96 - 110], fungsi getListMovie() mengambil data film dari resource seperti judul, link, gambar, plot, tahun, dan deskripsi. Data tersebut dimasukkan ke dalam list ArrayList<Movie> secara berurutan menggunakan perulangan, lalu list ini dikembalikan sebagai hasil fungsi. dataImage.recycle() dipanggil untuk membebaskan resource gambar setelah digunakan.

Pada baris [112 - 115], onDestroyView() dipanggil saat tampilan fragment dihancurkan, dan di sini variabel binding di-set ke null untuk menghindari memory leak dengan membersihkan referensi ke view.

DetailFragment.kt

Pada baris [1 - 8], ini adalah deklarasi paket dan impor kelas penting yang dibutuhkan untuk membuat Fragment di Android, termasuk FragmentDetailBinding yang menghubungkan layout XML fragment detail dengan kode Kotlin agar bisa mengakses elemen UI secara langsung.

Pada baris [10 - 13], dibuat kelas DetailFragment yang merupakan subclass dari Fragment. Di sini juga dideklarasikan variabel _binding yang bertipe nullable FragmentDetailBinding untuk mengelola binding layout, serta properti binding yang

non-nullable agar memudahkan akses elemen UI dengan aman setelah binding diinisialisasi.

Pada baris [15 - 36], method `onCreateView` di-override untuk membuat dan menghubungkan tampilan fragment. Binding layout diinisialisasi dengan `FragmentDetailBinding.inflate`. Data film seperti judul, gambar, tahun, plot, dan deskripsi diambil dari argumen fragment dengan key yang sesuai. Nilai tersebut kemudian ditampilkan di elemen UI menggunakan binding, misalnya `tvItemTitle`, `tvItemPlot`, `tvItemYears`, `tvItemDesc`. Gambar dipasang jika ada nilai image. Akhirnya, root view dari binding dikembalikan sebagai tampilan fragment. Ada kesalahan kecil pada penempatan data: `tvItemPlot` diisi dengan years, dan `tvItemYears` diisi dengan plot (seharusnya dibalik).

Pada baris [37 - 40], method `onDestroyView` di-override untuk membersihkan binding ketika view fragment dihancurkan. Ini penting untuk menghindari memory leak dengan menyetel `_binding` menjadi null saat view sudah tidak digunakan lagi.

MovieViewModel.kt

Pada baris [1 - 6], kode mengimpor paket utama untuk membuat `ViewModel`, mengelola `StateFlow` dari Kotlin Coroutines untuk data yang dapat diobservasi, dan `Timber` untuk logging/debugging di aplikasi Android.

Pada baris [8 - 13], kelas `MovieViewModel` mendefinisikan dua properti `MutableStateFlow` privat untuk menyimpan daftar film (`_movieList`) dan film yang dipilih (`_selectedMovie`). Properti publik `movieList` dan `selectedMovie` mengexpose `StateFlow` hanya baca untuk pengamatan dari luar kelas. Konstruktor menerima parameter `username` yang disimpan sebagai properti kelas.

Pada baris [15 - 17], blok `init` akan langsung dijalankan saat instance `MovieViewModel` dibuat, di mana terdapat logging menggunakan `Timber` untuk mencatat pembuatan `ViewModel` beserta `username` yang diterima.

Pada baris [19 - 22], fungsi `setMovieList` menerima parameter `movies` berupa daftar `Movie`, mencatat jumlah item yang diterima menggunakan `Timber`, lalu memperbarui nilai `_movieList` dengan daftar tersebut.

Pada baris [24 - 27], fungsi `selectMovie` menerima objek `movie` bertipe `Movie`, mencatat judul film yang dipilih dengan `Timber`, lalu menyimpan objek `movie` tersebut ke dalam `_selectedMovie` untuk menandai film yang sedang dipilih.

MovieViewModelFactory.kt

Pada baris [1 - 3], `package com.example.movielistxml` dideklarasikan, lalu `ViewModel` dari `androidx.lifecycle` diimpor untuk digunakan sebagai kelas dasar dalam membuat kelas `ViewModel` yang bertugas mengelola data dan logika UI secara `lifecycle-aware`.

Pada baris [5 - 13], didefinisikan kelas `MovieViewModelFactory` yang mengimplementasikan `ViewModelProvider.Factory`. Fungsi `create` akan membuat instance `MovieViewModel` dengan parameter `username` jika `modelClass` sesuai. Jika tidak, akan melempar `IllegalArgumentException` karena kelas `ViewModel` tidak dikenal. Ini digunakan untuk membuat `ViewModel` dengan konstruktor yang membutuhkan argumen.

MyApplication.kt

Pada baris [1 - 13], kode ini mendefinisikan kelas `MyApplication` yang merupakan subclass dari `Application`. Di dalam metode `onCreate()`, kode memanggil `super.onCreate()` untuk menjalankan inisialisasi standar aplikasi. Kemudian, menggunakan pengecekan `BuildConfig.DEBUG` untuk memastikan bahwa fitur logging dengan `Timber` hanya diaktifkan saat aplikasi berjalan dalam mode debug. Jika benar, `Timber.DebugTree()` dipasang agar log dari aplikasi bisa dicatat dan ditampilkan di `Logcat` selama pengembangan. Ini membantu developer memonitor jalannya aplikasi dengan mudah tanpa menampilkan log di versi rilis.

***activity_main.xml**

Pada baris [1 – 9], membuat ID di Kotlin MainActivity sebagai tempat menampung fragment, match parent akan mengisi seluruh layar dan contextnya akan terkait dengan MainActivity

Terdapat 2 activity_activity main dengan isi yang sama tapi salah satu nya itu berfungsi sebagai landscape saat layar di rotate

***fragment_detail.xml**

Pada baris [10 – 19], <ImageView layout ini akan digunakan untuk penempatan gambar khusus poster film dengan id img_poster yang digunakan pada kode Kotlin. Peletakkannya di bagian atas layout dan rata kiri

Pada baris [21 – 31], <TextView layout ini digunakan untuk penempatan title dengan id tv_item_title yang berada tepat di bawah poster.

Pada baris [33 – 43], <TextView layout ini digunakan untuk penempatan tahun dengan id tv_item_years yang terletak di sebelah kanan judul

Pada baris {45 – 55}, <TextView layout ini digunakan untuk penempatan label teks sebelum deskripsi plot dengan id tv_item_plot

Pada baris [57 – 67], <TextView layout ini digunakan untuk penempatan deskripsi film dengan id tv_item_desc

Terdapat 2 fragment_detail juga dengan isi yang sama tapi yang satu berfungsi untuk menjaga tampilan saat landscape atau layar di rotate

***fragment_home.xml**

Pada baris [1 – 19], membuat container fleksibel untuk penempatan berdasarkan constraint view dan ke parent, id rv_movie diakses di HomeFragment untuk menampilkan list movie dengan beberapa aturan lebar, tinggi, margin dan layout constraint untuk Recycler view memenuhi semua area parent dengan margin

***item_movie.xml**

Pada baris [1 – 19], membuat constraint layout CardView untuk mengatur posisi elemen secara fleksible dan memberi warna latar abu abu

Pada baris [21 – 29], <ImageView untuk membuat layout gambar dengan scaletype center agar gambar memenuhi area dan constraint ke atas, bawah, kiri parent

Pada baris [31 – 43], <TextView untuk membuat judul film meletakkannya disebelah kanan poster dan diapit img poster dan tv_item_years

Pada baris [45 – 55], <TextView untuk membuat tahun film yang peletakkannya di kanan judul dan menampilkan rilis filmnya kapan.

Pada baris [57 – 68], <TextView untuk membuat label plot di bawah judul dan dikanan poster

Pada baris [70 – 80], <TextView untuk membuat deskripsi film di sebelah label plot dan menampilkan ringkasan cerita film

Pada baris [82 – 91], <Button untuk tombol detail pindah ke DetailFragment di bawah deskripsi, dikanan bersebelahan dengan tombol IMDB

Pada baris [93 – 101], <Button untuk tombol pindah halaman ke link IMDB dan terletak di bawah deskripsi di kiri.

Jetpack Compose:

***strings.xml**

Pada string ini, berisikan data data yang akan digunakan pada kode Kotlin yang akan dijelaskan setelahnya. StringArray ini berfungsi sebagai data yang akan digunakan untuk menampilkan film atau deskripsi dan semacamnya.

DataMovie.kt

Pada baris [1 – 4], menempatkan file di package data, mengimpor R untuk akses ke resource seperti R.string atau R.drawable. mengimpor ListMovie dari class model yang merepresentasikan satu film

Pada baris [6 -13], fungsi loadMovieList() bertugas menghasilkan daftar data film dalam bentuk List<ListMovie>. Fungsi ini mengembalikan data statis yang berisi beberapa objek ListMovie, di mana setiap objek memuat informasi seperti judul film (title), tautan IMDb (url), poster (image), tanggal rilis (releaseDate), ringkasan (desc), dan deskripsi lengkap (description). Nilai-nilai ini tidak ditulis langsung dalam bentuk string atau gambar, melainkan mengacu pada ID dari resource yang disimpan di strings.xml dan drawable. Pendekatan ini memudahkan pengelolaan konten dan mendukung penerjemahan jika dibutuhkan. Data hasil fungsi ini nantinya akan digunakan di UI, seperti pada LazyColumn, untuk menampilkan daftar film secara dinamis berdasarkan resource yang telah ditentukan.

ListMovie.kt

Pada baris [1 – 4], menempatkan file di package model karena representasi data, @StringRes dan @DrawableRes untuk anotasi validasi tipe resource

Pada baris [6 – 13], membuat data class untuk menyimpan data film yang masing masing bertipe data Int karena akan mengambil dari resource ID. Properti image dianotasi dengan @DrawableRes karena nilainya merujuk pada ID gambar di drawable. Sementara itu, properti url bertipe String tanpa anotasi karena nilainya berupa URL langsung dan bukan ID resource. Ini mencerminkan bahwa URL tidak diambil dari strings.xml, melainkan ditulis langsung dalam bentuk string literal.

DetailActivity.kt

Pada baris [1 – 19], membuat activity dasar compose dan mengimpor lainnya seperti elemen UI compose seperti Image, Text, Column, Row dan sebagainya

Pada baris [21 – 22], membuat activity yang digunakan untuk menampilkan detail film yang dipilih dari list, dan onCreate untuk format awal eksekusi UI

Pada baris [24 – 27], mengambil data dari intent dan data dikirim dalam bentuk resource ID sesuai `@StringRes` dan `@DrawableRes`

Pada baris [29 – 34], surface untuk memberi tempat dengan latar hitam dan `MListComposeTheme` membungkus UI agar mengikuti tema aplikasi

Pada baris [35 – 40], menggunakan column untuk menampilkan elemen secara vertikal

Pada baris [41 – 45], menampilkan gambar dari resource ID dan `contentDescription` diisi dengan judul film untuk aksesibilitas

Pada baris [46 – 70], menggunakan row agar judul dan tahun sejajar secara horizontal dan `weight 1F` pada judul agar mengisi ruang, dan tahun mengikuti di kanan

Pada baris [71 – 85], Text yang akan digunakan untuk label dan isi deskripsi film

MainActivity.kt

Pada baris [1 – 33], merupakan bagian awal dari file yang menandakan bahwa file ini berada dalam package `com.example.mlistcompose`. Bagian ini juga mengimpor berbagai komponen dari Jetpack Compose, Android framework, tema `MListComposeTheme`, dan pustaka eksternal seperti Timber untuk keperluan logging. Pada baris [48 – 63], activity utama yang memanggil `MovieListScreen()` saat app dibuka dengan menggunakan tema compose dan meletakkannya di surface untuk interface dengan background hitam. Data list film diambil dari `DataMovie().loadMovieList()`

Pada baris [35 – 56], kelas `MainActivity` berfungsi sebagai activity utama aplikasi yang menggunakan Jetpack Compose. `ViewModel` `MovieViewModel` diinisialisasi untuk mengatur data yang akan ditampilkan. Di dalam fungsi `onCreate`, `enableEdgeToEdge()` dipanggil agar tampilan memenuhi layar, lalu `Timber.plant()` digunakan untuk debugging. Selanjutnya, `setContent` menampilkan UI dengan tema `MListComposeTheme`, kemudian seluruh isi layar dibungkus `Surface` berwarna hitam

dan menampilkan MovieListScreen sebagai tampilan utama yang menampilkan daftar film.

Pada baris [58 – 64], fungsi MovieListScreen adalah composable yang menampilkan daftar film menggunakan data dari MovieViewModel. Fungsi ini mengambil state film, navigasi ke URL, dan navigasi ke detail dengan collectAsState(). Jika ada URL untuk dibuka, akan memicu intent browser, sedangkan jika ada film untuk detail, akan membuka DetailActivity dengan data film dikirim lewat intent. Daftar film ditampilkan dalam LazyColumn dengan jarak antar item, di mana setiap item dipanggil melalui MovieItems yang menerima aksi klik untuk membuka link IMDB atau detail film.

Pada baris [96 – 189], fungsi MovieItems adalah composable yang menampilkan satu item film dalam bentuk kartu dengan desain sudut membulat dan warna gelap. Di dalam kartu ini, ada gambar film yang ditampilkan dengan ukuran khusus dan diapit oleh teks judul dan tahun rilis yang diambil dari resource string. Di bawah judul, ada deskripsi singkat plot film. Di bagian bawah kartu terdapat dua tombol, yaitu tombol "IMDB" untuk membuka link film di browser dan tombol "Detail" untuk membuka halaman detail film. Tata letak menggunakan kombinasi Row dan Column agar elemen tampil rapi dan proporsional dengan padding dan jarak antar elemen.

MovieViewModel.kt

Pada baris [1 – 9], ini adalah ViewModel yang mengelola data film dan navigasi menggunakan StateFlow, serta memakai Timber untuk logging debug.

Pada baris [11 – 19], kelas MovieViewModel menyimpan daftar film dan status navigasi menggunakan MutableStateFlow yang bersifat private, serta menyediakan akses publik lewat StateFlow agar data bisa dipantau tanpa bisa diubah langsung dari luar.

Pada baris [20 – 24], blok init menjalankan kode saat ViewModel dibuat dengan mencatat log proses loading data film menggunakan Timber, lalu memuat daftar film dari DataMovie ke dalam _movies, dan mencatat jumlah item yang berhasil dimuat.

Pada baris [26 – 29], fungsi `onLinkClicked` mencatat log saat URL diklik dan mengatur nilai `_navigateToUrl` dengan URL tersebut untuk memicu navigasi ke browser.

Pada baris [31 – 34], fungsi `onDetailClicked` mencatat log saat detail film diklik dan mengatur nilai `_navigateToDetail` dengan data film tersebut untuk memicu navigasi ke halaman detail.

Pada baris [36 – 42], fungsi `onUrlNavigated` dan `onDetailNavigated` digunakan untuk mengatur ulang nilai navigasi menjadi null setelah navigasi selesai, agar event navigasi tidak terjadi berulang.

MovieViewModelFactory.kt

Pada baris [1 – 4], package didefinisikan sebagai `com.example.mlistcompose` dan mengimpor kelas `ViewModel` serta `ViewModelProvider` dari `AndroidX Lifecycle` untuk mendukung pembuatan `ViewModel` dengan parameter khusus.

Pada baris [6 – 14], dibuat kelas `MovieViewModelFactory` yang bertugas membuat instance `MovieViewModel` dengan parameter `someParam`, menggunakan override fungsi `create` dari `ViewModelProvider.Factory` untuk mengontrol pembuatan `ViewModel` dan memastikan tipe yang benar, serta melemparkan exception jika kelas `ViewModel` tidak dikenali.

Step Into, Step Over, dan Step Out:

Debugger adalah alat yang penting untuk pengembangan aplikasi, dengan debugger kita bisa memahami alur eksekusi kode, memeriksa nilai variable, mengidentifikasi bug, dan menguji kondisi tertentu. Pada gambar terdapat beberapa breakpoint yang menghentikan eksekusi program, pada gambar juga terlihat breakpoint akan menghasilkan apa yang terjadi atau muncul saat item di klik atau diakses. Di window debug kita akan melihat beberapa seperti frame yang menunjukkan tumpukan fungsi, variables, threads dan console.

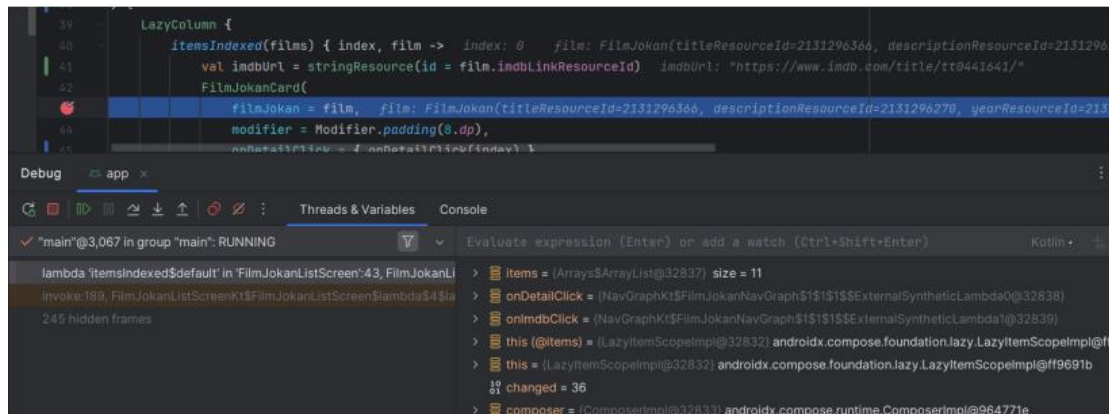
Fitur Step Over pada gambar jika berada pada baris 27 akan meloncat ke baris 28 atau 29 tanpa metode internal Timber.d

Step Into, pada gambar memungkinkan membawa ke kode sumber dari fungsi timber.d, jika berada di baris 29 atau navigate to detail url akan membawa ke baris pertama dari metode

Step Out, jika kita menggunakan Step Into dan ada di implementasi timber.d dan step out ditekan, akan mengembalikan ke baris panggilan Timber.d di ModelViewModel.kt

SOAL 2

Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya Aplikasi harus dapat mempertahankan fitur-fitur yang sudah dibuat pada modul sebelumnya. Berikut adalah contoh debugging dalam Android Studio.



A. Pembahasan

Dalam android, application class adalah kelas dasar untuk menjaga state global aplikasi. Ini adalah singleton di seluruh siklus hidup atau lifecycle aplikasi. Application class dibuat ketika proses aplikasi dimulai dan akan dihancurkan ketika proses dimatikan. Fungsinya adalah menginisialisasi global, menyimpan data yang tersedia di seluruh komponen, dan class ini memiliki callback yang dipanggil saat aplikasi pertama kali dibuat.

Secara singkat dalam gambar ini menjaga fitur seperti verifikasi data yang dibuat memiliki nilai, jika ada kesalahan maka akan bisa dilacak ke sumber data.

MODUL 5 : CONNECT TO THE INTERNET

SOAL 1

Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
- b. Gunakan KotlinX Serialization sebagai library JSON.
- c. Gunakan library seperti Coil atau Glide untuk image loading.
- d. API yang digunakan pada modul ini adalah The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API:
<https://developer.themoviedb.org/docs/getting-started>
- e. Implementasikan konsep data persistence (aplikasi menyimpan data walau pengguna keluar dari aplikasi) dengan SharedPreferences untuk menyimpan data ringan (seperti pengaturan aplikasi) dan Room untuk data relasional.
- f. Gunakan caching strategy pada Room. Dibebaskan untuk memilih caching strategy yang sesuai, dan sertakan penjelasan kenapa menggunakan caching strategy tersebut.
- g. Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

A. Source Code

Compose:

data/local/AppDatabase.kt

Tabel 41. Source Code AppDatabase.kt Soal 1 Modul 5 Compose

1	package com.example.mlistcompose.data.local
2	
3	import android.content.Context
4	import androidx.room.Database
5	import androidx.room.Room
6	import androidx.room.RoomDatabase
7	
8	@Database(entities = [MovieEntity::class], version =
	1, exportSchema = false)
9	abstract class AppDatabase : RoomDatabase() {
10	abstract fun movieDao(): MovieDao
11	
12	companion object {
13	@Volatile
14	private var INSTANCE: AppDatabase? = null
15	
16	fun getDatabase(context: Context):
17	AppDatabase {
18	return INSTANCE ?: synchronized(this) {
19	val instance = Room.databaseBuilder(
20	context.applicationContext,
21	AppDatabase::class.java,
22	"movie_database"
23	).build()
24	INSTANCE = instance
25	instance
26	}
27	}
28	

	}
	}

data/local/MovieDao.kt

Tabel 42. Source Code MovieDao.kt Soal 1 Modul 5 Compose

1	package com.example.mlistcompose.data.local
2	
3	import androidx.room.Dao
4	import androidx.room.Insert
5	import androidx.room.OnConflictStrategy
6	import androidx.room.Query
7	import kotlinx.coroutines.flow.Flow
8	
9	@Dao
10	interface MovieDao {
11	@Insert(onConflict = OnConflictStrategy.REPLACE)
12	suspend fun insertMovies(movies:
13	List<MovieEntity>)
14	
15	@Query("SELECT * FROM movies ORDER BY
16	releaseDate DESC")
17	fun getAllMovies(): Flow<List<MovieEntity>>
18	@Query("DELETE FROM movies")
19	suspend fun deleteAllMovies()
	}

data/local/MovieEntity.kt

Tabel 43. Source Code MovieEntity.kt Compose

1	package com.example.mlistcompose.data.local
2	
3	import androidx.room.Entity
4	import androidx.room.PrimaryKey
5	
6	@Entity(tableName = "movies")
7	data class MovieEntity(
8	@PrimaryKey val id: Int,
9	val title: String,
10	val posterPath: String?,
11	val releaseDate: String?,
12	val overview: String?,
13	val voteAverage: Double?
14	)

data/local/preferences/SettingManager.kt

Tabel 44. Source Code SettingManager.kt Compose

1	package com.example.mlistcompose.data.preferences
2	
3	import android.content.Context
4	import android.content.SharedPreferences
5	import androidx.core.content.edit
6	import kotlinx.coroutines.channels.awaitClose
7	import kotlinx.coroutines.flow.Flow
8	import kotlinx.coroutines.flow.callbackFlow
9	
10	class SettingsManager(context: Context) {
11	private val prefs: SharedPreferences =
	context.getSharedPreferences("app_settings",

	Context.MODE_PRIVATE)
12	
13	companion object {
14	const val KEY_DARK_MODE = "dark_mode"
15	}
16	
17	fun getDarkModeEnabledAsFlow(): Flow<Boolean> =
	callbackFlow {
18	val listener =
	SharedPreferences.OnSharedPreferenceChangeListener
	{ _, key ->
19	if (key == KEY_DARK_MODE) {
20	trySend(isDarkModeEnabled())
21	}
22	}
23	prefs.registerOnSharedPreferenceChangeListener(list
24	ener)
25	trySend(isDarkModeEnabled())
	awaitClose {
	prefs.unregisterOnSharedPreferenceChangeListener(li
26	stener) }
27	}
28	
29	fun isDarkModeEnabled(): Boolean {
30	return prefs.getBoolean(KEY_DARK_MODE,
31	false)
32	}
33	
	fun setDarkModeEnabled(isEnabled: Boolean) {

34	<code>prefs.edit { putBoolean(KEY_DARK_MODE,</code>
35	<code>isEnabled) }</code>
	<code> }</code>
	<code>}</code>

model/network/MovieApi.kt

Tabel 45. Source Code MovieApi.kt Compose

1	<code>package com.example.mlistcompose.model.network</code>
2	
3	<code>import kotlinx.serialization.SerialName</code>
4	<code>import kotlinx.serialization.Serializable</code>
5	
6	<code>@Serializable</code>
7	<code>data class MovieApi(</code>
8	<code> val id: Int,</code>
9	<code> val title: String?,</code>
10	<code> @SerialName("poster_path") val posterPath:</code>
11	<code>String?,</code>
	<code> @SerialName("release_date") val releaseDate:</code>
12	<code>String?,</code>
13	<code> val overview: String?,</code>
14	<code> @SerialName("vote_average") val voteAverage:</code>
	<code>Double?</code>
	<code>)</code>

model/network/MovieResponse.kt

Tabel 46. Source Code MovieResponse.kt Compose

1	<code>package com.example.mlistcompose.model.network</code>
2	

3	import kotlinx.serialization.SerialName
4	import kotlinx.serialization.Serializable
5	
6	@Serializable
7	data class MovieResponse(
8	val page: Int,
9	val results: List<MovieApi>,
10	@SerialName("total_pages") val totalPages: Int,
11	@SerialName("total_results") val totalResults:
12	Int
	)

model/ListMovie.kt

Tabel 47. Source Code ListMovie.kt Compose

1	package com.example.mlistcompose.model
2	
3	data class ListMovie(
4	val id: Int,
5	val title: String,
6	val imageUrl: String?,
7	val releaseDate: String?,
8	val description: String?,
9	val voteAverage: Double?
10	)

network/RetrofitClient.kt

Tabel 48. Source Code RetrofitClient.kt Compose

1	package com.example.mlistcompose.network
2	

```

3 import com.example.mlistcompose.utils.Constants
4 import
  com.jakewharton.retrofit2.converter.kotlinx.serialization.asConverterFactory
5 import
  kotlinx.serialization.ExperimentalSerializationApi
6 import kotlinx.serialization.json.Json
7 import okhttp3.Interceptor
8 import okhttp3.MediaType.Companion.toMediaType
9 import okhttp3.OkHttpClient
10 import okhttp3.logging.HttpLoggingInterceptor
11 import retrofit2.Retrofit
12
13 object RetrofitClient {
14
15     private val json = Json { ignoreUnknownKeys =
true }
16
17     private val loggingInterceptor =
HttpLoggingInterceptor().apply {
18         setLevel(HttpLoggingInterceptor.Level.BODY)
19     }
20
21     private val authInterceptor = Interceptor {
chain ->
22         val originalRequest = chain.request()
23         val newUrl =
24 originalRequest.url.newBuilder()
                .addQueryParameter("api_key",
25 Constants.TMDB_API_KEY)

```

26	<pre> .build() val newRequest = 27 originalRequest.newBuilder().url(newUrl).build() 28 chain.proceed(newRequest) 29 } 30 31 private val okHttpClient = 32 OkHttpClient.Builder() 33 .addInterceptor(authInterceptor) 34 .addInterceptor(loggingInterceptor) 35 .build() 36 37 @OptIn(ExperimentalSerializationApi::class) 38 val tmdbApiService: TmdbApiService by lazy { 39 Retrofit.Builder() 40 .baseUrl(Constants.TMDB_BASE_URL) 41 .client(okHttpClient) 42 43 .addConverterFactory(json.asConverterFactory("applic 44 ation/json".toMediaType())) 45 .build() 46 .create(TmdbApiService::class.java) 47 } 48 } </pre>
----	---

network/TmdbApiService.kt

Tabel 49. Source Code TmdbApiService.kt Compose

1	package com.example.mlistcompose.network
2	
3	import
	com.example.mlistcompose.model.network.MovieResponse
4	import retrofit2.http.GET
5	import retrofit2.http.Query
6	
7	interface TmdbApiService {
8	@GET("movie/popular")
9	suspend fun getPopularMovies(
10	@Query("language") language: String = "en-
11	US",
12	@Query("page") page: Int = 1
13	): MovieResponse
	}

repository/MovieRepository.kt

Tabel 50. Source Code MovieRepository.kt Compose

1	package com.example.mlistcompose.repository
2	
3	import com.example.mlistcompose.data.local.MovieDao
4	import
5	com.example.mlistcompose.data.local.MovieEntity
6	import
7	com.example.mlistcompose.model.network.MovieApi
8	import
9	com.example.mlistcompose.network.TmdbApiService
10	import com.example.mlistcompose.utils.Resource
11	import kotlinx.coroutines.flow.Flow

```

12 import kotlinx.coroutines.flow.flow
13 import kotlinx.coroutines.flow.firstOrNull
14 import timber.log.Timber
15
16 class MovieRepository(
17     private val apiService: TmdbApiService,
18     private val movieDao: MovieDao
19 ) {
20     fun getPopularMovies(
21         fetchFromRemote: Boolean
22     ): Flow<Resource<List<MovieEntity>>> = flow {
23         emit(Resource.Loading())
24
25         val localMovies =
26         movieDao.getAllMovies().firstOrNull()
27         if (!localMovies.isNullOrEmpty()) {
28             emit(Resource.Success(localMovies))
29             Timber.d("Loaded ${localMovies.size}
30             movies from cache.")
31         }
32
33         val shouldLoadFromApi = fetchFromRemote ||
34         localMovies.isNullOrEmpty()
35
36         if (shouldLoadFromApi) {
37             try {
38                 Timber.d("Fetching movies from
39                 API...")
40                 val response =
41                 apiService.getPopularMovies()

```

37	val remoteMovies = response.results
38	val movieEntities =
39	remoteMovies.map { it.toMovieEntity() }
40	movieDao.deleteAllMovies()
41	
	movieDao.insertMovies(movieEntities)
42	Timber.d("Fetched and cached
	`\${movieEntities.size}` movies from API.")
43	
44	val newLocalMovies =
45	movieDao.getAllMovies().firstOrNull()
	if (newLocalMovies != null) {
46	
47	emit(Resource.Success(newLocalMovies))
48	} else {
49	emit(Resource.Error("Failed to
	retrieve new data from cache after API call."))
50	}
51	
	} catch (e: Exception) {
52	Timber.e(e, "Error fetching movies
53	from API: `\${e.message}`)
	if (localMovies.isNullOrEmpty()) {
54	emit(Resource.Error("Couldn't
55	reach server. Check your internet connection.",
56	emptyList()))
57	} else {
58	emit(Resource.Error("Failed to
	update data. Displaying cached data.",

59	localMovies))
60	}
61	}
62	} else {
63	if (localMovies.isNullOrEmpty()) {
64	emit(Resource.Error("No cached data
65	available and not fetching from remote."))
66	}
67	}
68	}
69	
70	private fun MovieApi.toMovieEntity():
71	MovieEntity {
72	return MovieEntity(
73	id = <i>this</i> .id,
	title = <i>this</i> .title ?: "No Title",
	posterPath = <i>this</i> .posterPath,
	releaseDate = <i>this</i> .releaseDate,
	overview = <i>this</i> .overview,
	voteAverage = <i>this</i> .voteAverage
	)
	}
	}

screen/DetailActivity

Tabel 51. Source Code DetailActivity.kt Compose

1	package com.example.mlistcompose.screen
2	
3	import android.os.Bundle

4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.compose.foundation.layout.*
7	import
8	androidx.compose.foundation.rememberScrollState
9	import androidx.compose.foundation.verticalScroll
10	import androidx.compose.material3.MaterialTheme
11	import androidx.compose.material3.Surface
12	import androidx.compose.material3.Text
13	import androidx.compose.ui.Alignment
14	import androidx.compose.ui.Modifier
15	import androidx.compose.ui.layout.ContentScale
16	import androidx.compose.ui.text.TextStyle
17	import androidx.compose.ui.text.font.FontWeight
18	import androidx.compose.ui.unit.dp
19	import androidx.compose.ui.unit.sp
20	import coil.compose.AsyncImage
	import
21	com.example.mlistcompose.ui.theme.MListComposeThem e
	import
22	com.example.mlistcompose.data.preferences.Settings
23	Manager
24	
	class DetailActivity : ComponentActivity() {
25	override fun onCreate(savedInstanceState:
26	Bundle?) {
27	super.onCreate(savedInstanceState)
28	val title = intent.getStringExtra("title")

	?: "N/A"
29	val imageUrl =
	intent.getStringExtra("imageUrl") ?: ""
30	val releaseDate =
	intent.getStringExtra("releaseDate") ?: "N/A"
	val overview =
31	intent.getStringExtra("description") ?: "No
	description available."
32	val voteAverage =
33	intent.getDoubleExtra("voteAverage", 0.0)
34	
	val settingsManager =
35	SettingsManager(this)
36	val isDarkModeEnabled =
37	settingsManager.isDarkModeEnabled()
38	setContent {
39	MListComposeTheme(darkTheme =
40	isDarkModeEnabled) {
	Surface(
41	modifier =
42	Modifier.fillMaxSize(),
43	color =
44	MaterialTheme.colorScheme.background
45	) {
	Column(
46	modifier = Modifier
47	.fillMaxWidth()
48	.padding(16.dp)
49	

50	<code>.verticalScroll(rememberScrollState())</code>
	<code>) {</code>
51	<code> AsyncImage(</code>
52	<code> model = imageUrl,</code>
53	<code> contentDescription =</code>
	<code>title,</code>
54	<code> modifier = Modifier</code>
55	<code> .height(300.dp)</code>
	<code> .fillMaxWidth()</code>
56	
57	<code> .align(Alignment.CenterHorizontally),</code>
	<code> contentScale =</code>
58	<code>ContentScale.Fit</code>
59	<code>)</code>
60	<code> Spacer(modifier =</code>
61	<code>Modifier.height(16.dp))</code>
62	
63	<code> Text(</code>
	<code> text = title,</code>
64	<code> style = TextStyle(</code>
	<code> fontSize = 36.sp,</code>
65	<code> fontWeight =</code>
66	<code>FontWeight.Bold,</code>
	<code> color =</code>
67	<code>MaterialTheme.colorScheme.onBackground</code>
68	<code>),</code>
69	<code> modifier =</code>
	<code>Modifier.padding(bottom = 4.dp)</code>
70	<code>)</code>
71	<code> Text(</code>

72	text = "Release Date:
	\$releaseDate",
73	style = TextStyle(
74	fontSize = 18.sp,
	color =
75	MaterialTheme.colorScheme.onBackground
76	),
77	modifier =
	Modifier.padding(bottom = 4.dp)
78	)
79	Text(
80	text = "Rating:
	%.1f/10".format(voteAverage),
81	style = TextStyle(
82	fontSize = 18.sp,
	color =
83	MaterialTheme.colorScheme.onBackground
84	),
85	modifier =
86	Modifier.padding(bottom = 16.dp)
87	)
88	Text(
	text = "Plot :",
89	style = TextStyle(
	fontSize = 20.sp,
90	fontWeight =
91	FontWeight.Bold,
92	color =
	MaterialTheme.colorScheme.onBackground
93	)

94	)
95	Spacer(modifier =
96	Modifier.height(8.dp))
97	Text(
	text = overview,
98	style = TextStyle(
99	fontSize = 16.sp,
100	color =
101	MaterialTheme.colorScheme.onBackground
102	)
103	)
104	}
105	}
	}
	}
	}
	}

screen/MovieListScreen.kt

Tabel 52. Source Code MovieListScreen.kt Compose

```
1 package com.example.mlistcompose.screen
2
3 import android.content.Intent
4 import
5 androidx.compose.foundation.layout.Arrangement
6 import androidx.compose.foundation.layout.Column
7 import
8 androidx.compose.foundation.layout.PaddingValues
9 import androidx.compose.foundation.layout.Row
```

10	<code>import androidx.compose.foundation.layout.Spacer</code>
11	<code>import</code>
12	<code>androidx.compose.foundation.layout.fillMaxSize</code>
13	<code>import</code>
14	<code>androidx.compose.foundation.layout.fillMaxWidth</code>
15	<code>import androidx.compose.foundation.layout.height</code>
	<code>import androidx.compose.foundation.layout.padding</code>
16	<code>import androidx.compose.foundation.layout.size</code>
	<code>import androidx.compose.foundation.layout.width</code>
17	<code>import</code>
18	<code>androidx.compose.foundation.layout.wrapContentHeight</code>
19	<code>import</code>
	<code>androidx.compose.foundation.layout.wrapContentWidth</code>
20	<code>import androidx.compose.foundation.lazy.LazyColumn</code>
21	<code>import androidx.compose.foundation.lazy.items</code>
22	<code>import</code>
23	<code>androidx.compose.foundation.shape.RoundedCornerShape</code>
	<code>import androidx.compose.material3.Button</code>
24	<code>import androidx.compose.material3.Card</code>
25	<code>import androidx.compose.material3.CardDefaults</code>
26	<code>import</code>
27	<code>androidx.compose.material3.CircularProgressIndicator</code>
28	<code>import androidx.compose.material3.MaterialTheme</code>
29	<code>import androidx.compose.material3.Text</code>
30	<code>import androidx.compose.runtime.Composable</code>
31	<code>import androidx.compose.runtime.collectAsState</code>
32	<code>import androidx.compose.runtime.getValue</code>
33	<code>import androidx.compose.ui.Alignment</code>
34	<code>import androidx.compose.ui.Modifier</code>
35	<code>import androidx.compose.ui.draw.clip</code>

36	<code>import androidx.compose.ui.graphics.Color</code>
37	<code>import androidx.compose.ui.layout.ContentScale</code>
38	<code>import androidx.compose.ui.platform.LocalContext</code>
39	<code>import androidx.compose.ui.text.TextStyle</code>
40	<code>import androidx.compose.ui.text.font.FontWeight</code>
41	<code>import androidx.compose.ui.unit.dp</code>
42	<code>import androidx.compose.ui.unit.sp</code>
43	<code>import androidx.core.net.toUri</code>
44	<code>import androidx.navigation.NavController</code>
45	<code>import coil.compose.AsyncImage</code>
46	<code>import</code>
47	<code>com.example.mlistcompose.viewmodel.MovieViewModel</code>
48	<code>import com.example.mlistcompose.model.ListMovie</code>
49	<code>import com.example.mlistcompose.utils.Resource</code>
50	<code>import androidx.compose.material3.Scaffold</code>
51	<code>import androidx.compose.material3.TopAppBar</code>
52	<code>import androidx.compose.material3.TopAppBarDefaults</code>
53	<code>import androidx.compose.material3.IconButton</code>
	<code>import androidx.compose.material3.Icon</code>
54	<code>import androidx.compose.material.icons.Icons</code>
55	<code>import</code>
56	<code>androidx.compose.material.icons.filled.Settings</code>
	<code>import</code>
57	<code>androidx.compose.material3.ExperimentalMaterial3Api</code>
58	<code>import com.example.mlistcompose.utils.Constants</code>
59	
60	<code>@OptIn(ExperimentalMaterial3Api::class,</code>
61	<code>ExperimentalMaterial3Api::class)</code>
62	<code>@Composable</code>
	<code>fun MovieListScreen(</code>

63	<code>viewModel: MovieViewModel,</code>
	<code>navController: NavController</code>
64	<code>) {</code>
	<code>val moviesResource by</code>
65	<code>viewModel.movies.collectAsState()</code>
66	<code>val navigateToUrl by</code>
67	<code>viewModel.navigateToUrl.collectAsState()</code>
68	<code>val navigateToDetail by</code>
	<code>viewModel.navigateToDetail.collectAsState()</code>
69	<code>val context = LocalContext.current</code>
70	
71	<code>navigateToUrl?.let { url -></code>
72	<code>val intent = Intent(Intent.ACTION_VIEW,</code>
73	<code>url.toUri())</code>
74	<code>context.startActivity(intent)</code>
	<code>viewModel.onUrlNavigated()</code>
75	<code>}</code>
76	
77	<code>navigateToDetail?.let { movie -></code>
78	<code>val intent = Intent(context,</code>
79	<code>DetailActivity::class.java).apply {</code>
80	<code>putExtra("id", movie.id)</code>
81	<code>putExtra("title", movie.title)</code>
82	<code>putExtra("imageUrl", movie.imageUrl)</code>
83	<code>putExtra("releaseDate",</code>
84	<code>movie.releaseDate)</code>
85	<code>putExtra("description",</code>
86	<code>movie.description)</code>
87	<code>putExtra("voteAverage",</code>
88	<code>movie.voteAverage)</code>

89	<pre> } context.startActivity(intent) viewModel.onDetailNavigated() } Scaffold(topBar = { TopAppBar(title = { Text("Film Populer", color = MaterialTheme.colorScheme.onPrimary) }, actions = { IconButton(onClick = { navController.navigate("settings") }) { Icon(imageVector = Icons.Filled.Settings, contentDescription = "Settings", tint = MaterialTheme.colorScheme.onPrimary) } }, colors = TopAppBarDefaults.topAppBarColors(containerColor = MaterialTheme.colorScheme.primary, titleContentColor = MaterialTheme.colorScheme.onPrimary)) }) </pre>
----	---

```

110         )
111     },
112     containerColor =
113     MaterialTheme.colorScheme.background
114     ) { paddingValues ->
115         Column(
116             modifier = Modifier
117                 .padding(paddingValues)
118                 .fillMaxSize()
119         ) {
120             when (moviesResource) {
121                 is Resource.Loading -> {
122                     Column(
123                         modifier =
124                         Modifier.fillMaxSize(),
125                         verticalArrangement =
126                         Arrangement.Center,
127                         horizontalAlignment =
128                         Alignment.CenterHorizontally
129                     ) {
130                         CircularProgressIndicator(color = Color.White)
131                         Spacer(modifier =
132                         Modifier.height(8.dp))
133                         Text("Loading movies...",
134                             color = Color.White)
135                     }
136                 }
137                 is Resource.Success -> {
138                     val movieList =

```

```

131 moviesResource.data ?: emptyList()
132         LazyColumn(
133             contentPadding =
134                 PaddingValues(16.dp),
135             verticalArrangement =
136                 Arrangement.spacedBy(16.dp)
137         ) {
138             items(movieList) { movie ->
139                 MovieItems(
140                     movie = movie,
141                     onDetailClicked = {
142                         viewModel.onDetailClicked(movie) },
143                     onTmdbClicked = {
144                         url -> viewModel.onImdbClicked(url) }
145                 )
146             }
147         }
148     is Resource.Error -> {
149         val errorMessage =
150             moviesResource.message ?: "An unknown error
151             occurred"
152         val cachedData =
153             moviesResource.data ?: emptyList()
154
155         if (cachedData.isNotEmpty()) {
156             LazyColumn(
157                 contentPadding =
158                     PaddingValues(16.dp),
159                 verticalArrangement =

```

	Arrangement.spacedBy(16.dp),
154	modifier =
	Modifier.fillMaxSize()
155	) {
156	item {
157	Column(
	modifier =
158	Modifier
	.fillMaxWidth()
159	.padding(bottom = 16.dp),
160	
161	
162	horizontalAlignment = Alignment.CenterHorizontally
	) {
163	Text(
	"Error:
164	\$errorMessage",
	color =
165	Color.Red,
166	fontSize =
	18.sp
167	)
	Text(
168	"Displaying
	cached data.",
169	color =
170	Color.Gray,
171	fontSize =
172	14.sp

	)
173	Spacer(modifier
174	= Modifier.height(16.dp))
175	Button(onClick =
	{ viewModel.fetchMovies(true) }) {
176	Text("Retry", color = Color.White)
177	}
178	}
179	}
180	items(cachedData) {
181	movie ->
182	MovieItems(
	movie = movie,
183	onDetailClicked
	= { viewModel.onDetailClicked(movie) },
184	onTmdbClicked =
	{ url -> viewModel.onImdbClicked(url) }
185	)
186	}
	}
187	} else {
	Column(
189	modifier =
	Modifier.fillMaxSize(),
190	verticalArrangement =
	Arrangement.Center,
191	horizontalAlignment =
192	Alignment.CenterHorizontally
193	) {

```

194         Text("Error:
195 $errorMessage", color = Color.Red, fontSize = 18.sp)
196         Spacer(modifier =
197 Modifier.height(16.dp))
198         Button(onClick = {
199 viewModel.fetchMovies(true) }) {
200             Text("Retry", color
201 = Color.White)
202         }
203     }
204 }
205 }
206 }
207 }
208 }
209 }
210
211 @Composable
212 fun MovieItems(
213     movie: ListMovie,
214     onDetailClicked: () -> Unit,
215     onTmdbClicked: (String) -> Unit
216 ) {
217     Card(
218         shape = RoundedCornerShape(16.dp),
219         modifier = Modifier
220             .fillMaxWidth()
221             .wrapContentHeight(),
222         colors =
223 CardDefaults.cardColors(containerColor =

```

```

224 MaterialTheme.colorScheme.surface),
225         elevation = CardDefaults.cardElevation(4.dp)
226     ) {
227         Row(
228             modifier = Modifier
229                 .padding(16.dp)
230                 .fillMaxWidth()
231         ) {
232             AsyncImage(
233                 model = movie.imageUrl,
234                 contentDescription = movie.title,
235                 contentScale = ContentScale.Crop,
236                 modifier = Modifier
237                     .size(width = 90.dp, height =
238                         200.dp)
239                     .clip(RoundedCornerShape(8.dp))
240             ).align(Alignment.CenterVertically)
241         )
242         Spacer(modifier = Modifier.width(12.dp))
243         Column(
244             modifier = Modifier
245                 .fillMaxWidth()
246             ).align(Alignment.CenterVertically)
247         ) {
248             Row(
249                 modifier = Modifier
250                     .fillMaxWidth()
251                     .padding(8.dp)

```

251	) {
252	Text (
	text = movie.title,
253	style = TextStyle(
254	fontSize = 26.sp,
255	fontWeight =
	FontWeight.Bold,
256	color =
257	MaterialTheme.colorScheme.onSurface
258	),
	modifier = Modifier
259	.padding(bottom = 4.dp)
	.weight(1f)
260	)
261	Text (
262	text =
263	movie.releaseDate?.substringBefore("-") ?: "N/A",
264	style = TextStyle(
265	fontSize = 20.sp,
266	color =
267	MaterialTheme.colorScheme.onSurface
268	),
269	modifier = Modifier
270	.padding(bottom = 4.dp)
271	.wrapContentWidth(Alignment.End)
272	.align(Alignment.CenterVertically)
273	)
274	}

	Row (
275	modifier =
276	Modifier. <i>fillMaxWidth</i> ()
	) {
277	Text (
278	text = "Plot :",
279	style = TextStyle(
	fontSize = 16.sp,
280	fontWeight =
281	FontWeight.Bold,
	color =
282	MaterialTheme.colorScheme.onSurface
283	)
284	)
285	Spacer(modifier =
286	Modifier. <i>width</i> (4.dp))
	Text (
287	text =
288	(movie.description?.take(100) + "..."),
289	style = TextStyle(
	fontSize = 16.sp,
290	color =
291	MaterialTheme.colorScheme.onSurface
292	),
	modifier =
293	Modifier. <i>padding</i> (bottom = 8.dp)
294	)
	}
295	Row (
296	modifier =

```

297 Modifier.fillMaxWidth(),
298             horizontalArrangement =
299 Arrangement.End
300         ) {
301             Button(onClick = {
302                 val tmdbUrl =
303 Constants.TMDB_MOVIE_WEB_BASE_URL + movie.id
                    onTmdbClicked(tmdbUrl)
                    }) {
                        Text("TMDB", color =
MaterialTheme.colorScheme.onPrimary)
                    }
                        Spacer(modifier =
Modifier.width(8.dp))
                        Button(onClick =
onDetailClicked) {
                            Text("Detail", color =
MaterialTheme.colorScheme.onPrimary)
                        }
                    }
                }
            }
        }
    }
}

```

screen/SettingScreen.kt

Tabel 53. Source Code SettingScreen.kt Compose

1	package
2	com.example.mlistcompose.screen
3	
4	import
5	androidx.compose.foundation.layout.*
	import
6	androidx.compose.material.icons.Icons
7	import
8	androidx.compose.material.icons.automated
9	filled.ArrowBack
10	import androidx.compose.material3.*
11	import
12	androidx.compose.runtime.Composable
13	import
14	androidx.compose.runtime.collectAsState
15	in
16	import
17	androidx.compose.runtime.getValue
18	import androidx.compose.ui.Alignment
19	import androidx.compose.ui.Modifier
20	import androidx.compose.ui.unit.dp
21	import
22	androidx.navigation.NavController
	import
23	com.example.mlistcompose.viewmodel.Movie
24	ViewModel
25	
26	@OptIn(ExperimentalMaterial3Api::class)
27	

	@Composable
28	fun SettingsScreen(
29	<i>navController</i> : NavController,
	<i>viewModel</i> : MovieViewModel
30	) {
31	val isDarkMode by
	viewModel.isDarkModeEnabled.collectAsS
32	tate()
33	
	Scaffold(
34	topBar = {
35	TopAppBar(
36	title = {
37	Text("Settings", color =
38	MaterialTheme.colorScheme.onPrimary)
	},
39	navigationIcon = {
	IconButton(onClick
40	= { <i>navController</i> .popBackStack() }) {
41	Icon(
42	
43	imageVector =
44	Icons.AutoMirrored.Filled.ArrowBack,
45	
46	contentDescription = "Back",
47	tint =
48	MaterialTheme.colorScheme.onPrimary
49	)
50	}
	},

51	colors =
52	TopAppBarDefaults.topAppBarColors(
53	containerColor =
54	MaterialTheme.colorScheme.primary,
55	titleContentColor
56	= MaterialTheme.colorScheme.onPrimary
57	)
	)
58	},
	containerColor =
59	MaterialTheme.colorScheme.background
60	) { <i>paddingValues</i> ->
61	Column(
62	modifier = Modifier
	<i>.fillMaxSize()</i>
63	
	<i>.padding(paddingValues)</i>
64	<i>.padding(16.dp),</i>
65	horizontalAlignment =
66	Alignment.CenterHorizontally,
67	verticalArrangement =
68	Arrangement.Top
69	) {
70	Row(
71	modifier = Modifier
	<i>.fillMaxWidth()</i>
72	<i>.padding(vertical</i>
	<i>= 8.dp),</i>
73	verticalAlignment =
	Alignment.CenterVertically,

```

74         horizontalArrangement
= Arrangement.SpaceBetween
75     ) {
76         Text (
77             "Dark Mode",
78             color =
79 MaterialTheme.colorScheme.onBackground
80 ,
            style =
MaterialTheme.typography.titleMedium
        )
        Switch (
            checked =
isDarkMode,
            onCheckedChange =
{ isChecked ->
viewModel.setDarkMode(isChecked)
            },
            colors =
SwitchDefaults.colors(
checkedThumbColor =
MaterialTheme.colorScheme.primary,
checkedTrackColor =
MaterialTheme.colorScheme.secondary,
uncheckedThumbColor =
MaterialTheme.colorScheme.onSurface,

```

	<pre> uncheckedTrackColor = MaterialTheme.colorScheme.surface)) } } } } </pre>
--	---

utils/Constants.kt

Tabel 54. Source Code Constants.kt Compose

1	package com.example.mlistcompose.utils
2	
3	object Constants {
4	const val TMDB_API_KEY =
	"01c9634cd909bf359669c502c2c8b378"
5	const val TMDB_BASE_URL =
	"https://api.themoviedb.org/3/"
6	const val TMDB_IMAGE_BASE_URL =
	"https://image.tmdb.org/t/p/w500/"
7	const val TMDB_MOVIE_WEB_BASE_URL =
	"https://www.themoviedb.org/movie/"
8	}

utils/resource.kt

Tabel 55. Source Code Resource.kt Compose

1	package com.example.mlistcompose.utils
2	

3	sealed class Resource<T>(val data: T? = null, val
4	message: String? = null) {
	class Success<T>(data: T) : Resource<T>(data)
	class Error<T>(message: String, data: T? = null)
5	: Resource<T>(data, message)
	class Loading<T>(data: T? = null) :
6	Resource<T>(data)
7	}

viewmodel/MovieViewModel.kt

Tabel 56. Source Code RetrofitClient.kt Compose

1	package com.example.mlistcompose.viewmodel
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.viewModelScope
5	import
	com.example.mlistcompose.data.preferences.SettingsMa
	nager
6	import com.example.mlistcompose.model.ListMovie
7	import
	com.example.mlistcompose.repository.MovieRepository
8	import com.example.mlistcompose.utils.Constants
9	import com.example.mlistcompose.utils.Resource
10	import kotlinx.coroutines.flow.MutableStateFlow
11	import kotlinx.coroutines.flow.SharingStarted
12	import kotlinx.coroutines.flow.StateFlow
13	import kotlinx.coroutines.flow.asStateFlow
14	import kotlinx.coroutines.flow.distinctUntilChanged
15	import kotlinx.coroutines.flow.stateIn

```

16 import kotlinx.coroutines.launch
17 import timber.log.Timber
18
19 class MovieViewModel(
20     private val repository: MovieRepository,
21     private val settingsManager: SettingsManager
22 ) : ViewModel() {
23     private val _movies =
24         MutableStateFlow<Resource<List<ListMovie>>>(Resource
25             .Loading()) // Resource of ListMovie
26     val movies: StateFlow<Resource<List<ListMovie>>>
27         = _movies.asStateFlow()
28
29     private val _navigateToUrl =
30         MutableStateFlow<String?>(null)
31     val navigateToUrl: StateFlow<String?> =
32         _navigateToUrl.asStateFlow()
33
34     private val _navigateToDetail =
35         MutableStateFlow<ListMovie?>(null)
36     val navigateToDetail: StateFlow<ListMovie?> =
37         _navigateToDetail.asStateFlow()
38
39     private val _navigateToDetail =
40         MutableStateFlow<ListMovie?>(null)
41     val navigateToDetail: StateFlow<ListMovie?> =
42         _navigateToDetail.asStateFlow()
43
44     private val isDarkModeEnabled: StateFlow<Boolean> =
45         settingsManager.getDarkModeEnabledAsFlow()
46             .distinctUntilChanged()
47             .stateIn(
48                 scope = viewModelScope,
49                 started =
50                     SharingStarted.WhileSubscribed(5000),

```

37	initialValue =
	settingsManager.isDarkModeEnabled()
38	)
39	
40	init {
41	fetchMovies()
42	}
43	
44	fun fetchMovies(<i>fetchFromRemote</i> : Boolean = true)
45	{
	<i>viewModelScope.launch</i> {
46	
	repository.getPopularMovies(<i>fetchFromRemote</i>).collect
47	{ <i>result</i> ->
48	<i>_movies.value</i> = when (<i>result</i>) {
49	is Resource.Success -> {
	val uiMovies =
50	<i>result.data?.map</i> { <i>movieEntity</i> ->
51	ListMovie(
52	id = <i>movieEntity.id</i> ,
	title =
53	<i>movieEntity.title</i> ,
	imageUrl = if
	(! <i>movieEntity.posterPath.isNullOrEmpty()</i>)
	Constants.TMDB_IMAGE_BASE_URL +
54	<i>movieEntity.posterPath</i> else null,
	releaseDate =
55	<i>movieEntity.releaseDate</i> ,
	description =
56	<i>movieEntity.overview</i> ,

	voteAverage =
57	<i>movieEntity</i> .voteAverage
58	)
59	} ?: <i>emptyList</i> ()
60	Resource.Success(uiMovies)
61	}
	is Resource.Error ->
	Resource.Error(<i>result</i> .message ?: "An unexpected
62	error occurred", <i>result</i> .data?.map { <i>movieEntity</i> ->
63	ListMovie(
64	id = <i>movieEntity</i> .id,
65	title =
	<i>movieEntity</i> .title,
	imageUrl = if
	(! <i>movieEntity</i> .posterPath.isNullOrEmpty())
66	Constants.TMDB_IMAGE_BASE_URL +
	<i>movieEntity</i> .posterPath else null,
67	releaseDate =
	<i>movieEntity</i> .releaseDate,
68	description =
	<i>movieEntity</i> .overview,
69	voteAverage =
70	<i>movieEntity</i> .voteAverage
71	)
	} ?: <i>emptyList</i> ()
72	is Resource.Loading ->
73	Resource.Loading()
74	}
75	}
76	}

77	}
78	
79	fun setDarkMode(<i>isEnabled</i> : Boolean) {
80	
81	settingsManager.setDarkModeEnabled(<i>isEnabled</i>)
82	}
83	
84	fun onImdbClicked(<i>url</i> : String) {
85	Timber.d("URL clicked: <i>\$url</i> ")
86	_navigateToUrl.value = <i>url</i>
87	}
88	
89	fun onDetailClicked(<i>movie</i> : ListMovie) {
90	Timber.d("Detail clicked for movie: \${ <i>movie.title</i> }")
91	_navigateToDetail.value = <i>movie</i>
92	}
93	
94	fun onUrlNavigated() {
95	_navigateToUrl.value = null
96	}
97	
98	fun onDetailNavigated() {
	_navigateToDetail.value = null
	}
	}

viewmodel/ModelViewModelFactory.kt

Tabel 57. Source Code MovieViewModelFactory.kt Compose

1	package com.example.mlistcompose.viewmodel
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	import com.example.mlistcompose.data.preferences.SettingsMa nager
6	import com.example.mlistcompose.repository.MovieRepository
7	
8	class MovieViewModelFactory(9 private val repository: MovieRepository, 10 private val settingsManager: SettingsManager 11) : ViewModelProvider.Factory { 12 override fun <T : ViewModel> create(modelClass: Class<T>): T { 13 if (modelClass.isAssignableFrom(MovieViewModel::class.j ava)) { 14 @Suppress("UNCHECKED_CAST") 15 return MovieViewModel(repository, settingsManager) as T 16 } 17 throw IllegalArgumentException("Unknown ViewModel class") 18 } 19 }

MListComposeApplication.kt

Tabel 58. Source Code MovieViewModelFactory.kt Compose

1	package com.example.mlistcompose
2	
3	import android.app.Application
4	import
5	com.example.mlistcompose.data.local.AppDatabase
6	import
	com.example.mlistcompose.network.RetrofitClient
7	import
8	com.example.mlistcompose.repository.MovieRepository
	import timber.log.Timber
	import
9	com.example.mlistcompose.viewmodel.MovieViewModelFac tory
	import
10	com.example.mlistcompose.data.preferences.SettingsMa nager
11	
12	
13	class MListComposeApplication : Application() {
14	private val database: AppDatabase by lazy { AppDatabase.getDatabase(this) }
15	private val movieDao by lazy { database.movieDao() }
16	private val tmdbApiService by lazy { RetrofitClient.tmdbApiService }
17	private val settingsManager: SettingsManager by 18 lazy { SettingsManager(this) }
19	
20	val movieRepository: MovieRepository by lazy {

21	MovieRepository(tmdbApiService, movieDao)
22	}
23	val movieViewModelFactory: MovieViewModelFactory
	by lazy {
24	MovieViewModelFactory(movieRepository,
25	settingsManager)
26	}
27	
28	override fun onCreate() {
29	super.onCreate()
30	if (BuildConfig.DEBUG) {
31	Timber.plant(Timber.DebugTree())
32	}
	}
	}

MainActivity.kt

Tabel 59. Source Code MainActivity.kt Compose

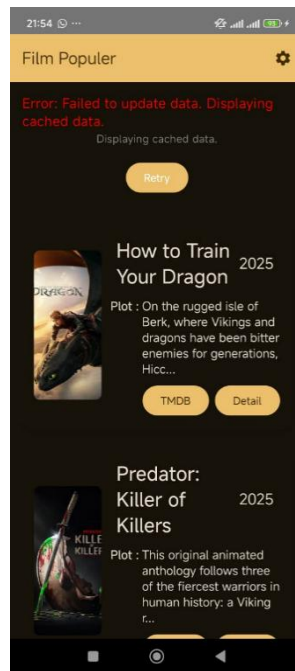
1	package com.example.mlistcompose
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.activity.viewModels
7	import
8	androidx.compose.foundation.layout.fillMaxSize
9	import androidx.compose.material3.MaterialTheme
10	import androidx.compose.material3.Surface

11	<code>import androidx.compose.runtime.collectAsState</code>
12	<code>import androidx.compose.runtime.getValue</code>
13	<code>import androidx.compose.ui.Modifier</code>
14	<code>import androidx.navigation.compose.NavHost</code>
15	<code>import androidx.navigation.compose.composable</code>
	<code>import</code>
16	<code>androidx.navigation.compose.rememberNavController</code>
17	<code>import</code>
18	<code>com.example.mlistcompose.screen.MovieListScreen</code>
	<code>import</code>
19	<code>com.example.mlistcompose.screen.SettingsScreen</code>
	<code>import</code>
	<code>com.example.mlistcompose.viewmodel.MovieViewModel</code>
20	<code>import</code>
	<code>com.example.mlistcompose.data.preferences.SettingsM</code>
21	<code>anager</code>
22	<code>import</code>
23	<code>com.example.mlistcompose.ui.theme.MListComposeTheme</code>
24	<code>class MainActivity : ComponentActivity() {</code>
25	<code> override fun onCreate(<i>savedInstanceState</i>:</code>
26	<code>Bundle?) {</code>
27	<code> <i>super</i>.onCreate(<i>savedInstanceState</i>)</code>
28	<code> val viewModel: MovieViewModel by <i>viewModels</i></code>
29	<code>{</code>
30	<code> (<i>application</i> as</code>
31	<code>MListComposeApplication).movieViewModelFactory</code>
32	<code> }</code>
33	

	<pre> val settingsManager = SettingsManager(this) 34 setContent { 35 val isDarkModeEnabled by viewModel.isDarkModeEnabled.collectAsState(initial 36 = settingsManager.isDarkModeEnabled()) 37 38 MListComposeTheme(darkTheme = isDarkModeEnabled) { 39 Surface(40 modifier = Modifier.fillMaxSize(), 41 color = 42 MaterialTheme.colorScheme.background 43) { 44 val navController = 45 rememberNavController() 46 47 NavHost(48 navController = 49 navController, 50 startDestination = "movieList" 51) { 52 composable("movieList") { 53 MovieListScreen(54 viewModel = viewModel, 55 navController = navController </pre>
--	--

```
56         )
57     }
58     composable("settings") {
59
60     SettingsScreen(navController = navController,
viewModel = viewModel)
        }
    }
}
}
```

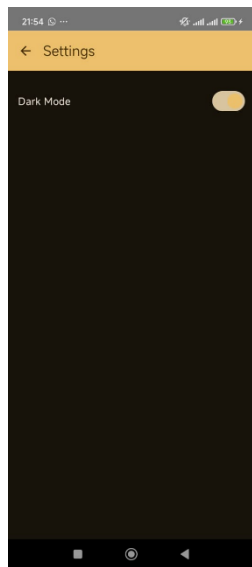
B. Output Program



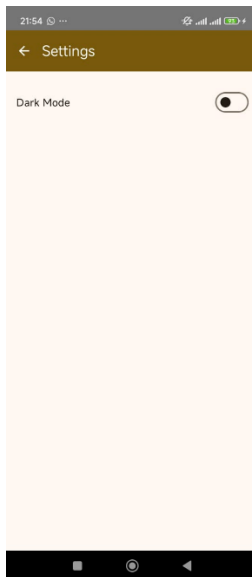
Gambar 47. Tampilan Aplikasi Ketika Offline Modul 5



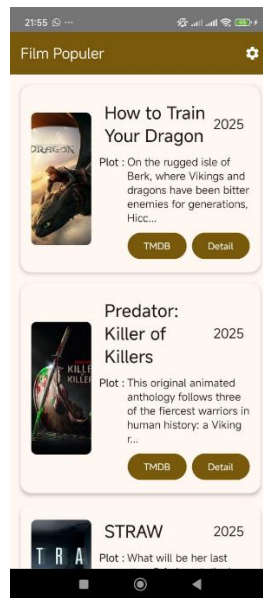
Gambar 48. Tampilan Aplikasi Ketika Offline dalam Light Mode Modul 5



Gambar 49. Tampilan Tombol Ubah Dark/Light Mode Modul 5



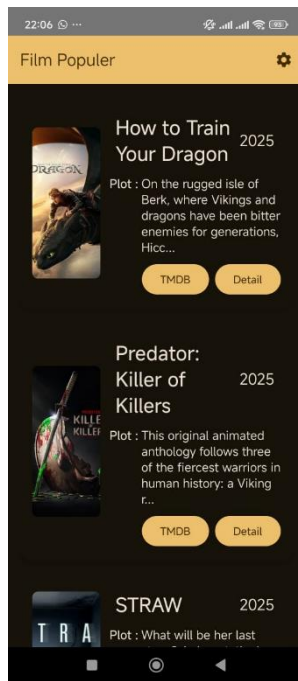
Gambar 50. Tampilan Tombol Ubah Dark/Light Mode Modul 5



Gambar 51. Tampilan Aplikasi Saat Retry Ditekan dan Online dalam Light Mode Modul 5



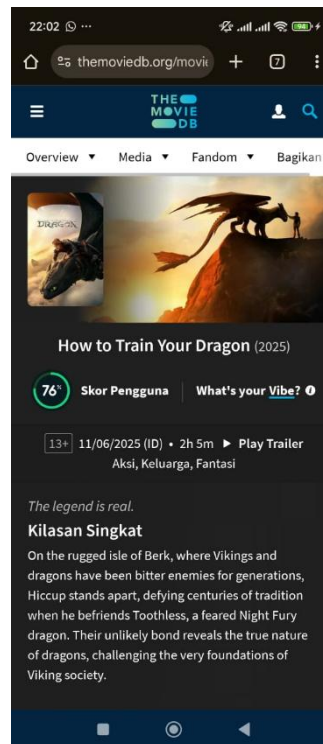
Gambar 52. Tombol Detail Ditekan dalam Light Mode Modul 5



Gambar 53. Tampilan Aplikasi Saat Retry Ditekan dan Online dalam Dark Mode Modul 5



Gambar 54. Tombol Detail Ditekan dalam Dark Mode Modul 5



Gambar 55. Tombol TMDb Saat Ditekan Modul 5

C. Pembahasan

Jetpack Compose:

```
data/local/AppDatabase.kt
```

Pada baris [1–6], file ditempatkan dalam package `data.local` yang menunjukkan lokasi penyimpanan data lokal, lalu mengimpor `Context` untuk keperluan akses sistem, serta mengimpor komponen-komponen penting dari library `Room` seperti `@Database`, `Room`, dan `RoomDatabase` yang digunakan untuk membangun dan mengelola database lokal dalam aplikasi Android.

Pada baris [8–10], class `AppDatabase` dideklarasikan sebagai abstract class yang mewarisi `RoomDatabase`, dan diberi anotasi `@Database` untuk mendefinisikan entitas database yaitu `MovieEntity`, versi database (1), serta menonaktifkan ekspor skema; di

dalamnya terdapat fungsi abstrak `movieDao()` yang digunakan Room untuk mengakses DAO (`MovieDao`) sebagai antarmuka operasi terhadap tabel film.

Pada baris [12–28], dideklarasikan objek companion object untuk menyediakan akses global ke instance database, di mana variabel `INSTANCE` bertipe nullable `AppDatabase` ditandai dengan anotasi `@Volatile` agar perubahan nilainya terlihat oleh semua thread; fungsi `getDatabase(context: Context)` akan mengembalikan instance database yang sudah ada atau membuat instance baru secara thread-safe menggunakan `synchronized`, dengan membangun database bernama "movie_database" menggunakan `Room.databaseBuilder`, lalu menyimpannya ke dalam `INSTANCE` agar dapat digunakan kembali.

`data/local/MovieDao.kt`

Pada baris [1–7], file ditempatkan dalam package `data.local` yang menandakan bahwa isi file ini berkaitan dengan data lokal, lalu mengimpor anotasi dan fungsi dari library Room seperti `@Dao`, `@Insert`, `OnConflictStrategy`, dan `@Query` untuk mendefinisikan operasi database, serta mengimpor Flow dari Kotlin Coroutines yang memungkinkan pemrosesan data secara asinkron dan berkelanjutan dari database.

Pada baris [9–19], dideklarasikan interface `MovieDao` yang ditandai dengan anotasi `@Dao` sebagai Data Access Object untuk Room, yang berisi tiga fungsi: `insertMovies()` untuk menyisipkan daftar film ke database dengan strategi konflik `REPLACE`, `getAllMovies()` untuk mengambil seluruh data film secara real-time dan terurut berdasarkan tanggal rilis secara menurun dalam bentuk Flow, serta `deleteAllMovies()` untuk menghapus seluruh data film dari tabel `movies`.

`data/local/MovieEntity.kt`

Pada baris [1–4], file ditempatkan dalam package `data.local` sebagai bagian dari struktur penyimpanan data lokal, lalu mengimpor anotasi `@Entity` untuk

mendefinisikan class sebagai entitas tabel dalam Room, serta `@PrimaryKey` untuk menandai properti sebagai kunci utama tabel.

Pada baris [6–14], dideklarasikan data class `MovieEntity` yang ditandai dengan anotasi `@Entity(tableName = "movies")` sebagai representasi tabel `movies` dalam database Room, dengan properti `id` sebagai `@PrimaryKey`, serta properti lainnya seperti `title`, `posterPath`, `releaseDate`, `overview`, dan `voteAverage` yang masing-masing merepresentasikan kolom-kolom dalam tabel dan menyimpan informasi detail mengenai film.

`data/local/preferences/SettingManager.kt`

Pada baris [1–8], file ditempatkan dalam package `data.preferences` yang menunjukkan bahwa isinya berkaitan dengan pengelolaan preferensi pengguna, lalu mengimpor `Context` dan `SharedPreferences` untuk mengakses dan menyimpan data preferensi, edit dari `androidx.core.content` untuk mempermudah proses penyimpanan data, serta `Flow`, `callbackFlow`, dan `awaitClose` dari Kotlin Coroutines untuk memungkinkan pemantauan perubahan data preferensi secara asinkron dan reaktif.

Pada baris [10–11], dideklarasikan class `SettingsManager` yang menerima `Context` sebagai parameter untuk mengakses sistem Android, lalu membuat objek `prefs` bertipe `SharedPreferences` dengan nama `"app_settings"` dan mode privat (`Context.MODE_PRIVATE`) agar data hanya dapat diakses oleh aplikasi itu sendiri, digunakan untuk menyimpan dan mengambil pengaturan aplikasi secara permanen.

Pada baris [13–15], dideklarasikan companion object yang berisi konstanta `KEY_DARK_MODE` untuk menyimpan nama kunci (`"dark_mode"`) yang digunakan dalam `SharedPreferences`, agar konsisten saat menyimpan atau mengambil nilai terkait pengaturan mode gelap aplikasi.

Pada baris [17–26], fungsi `getDarkModeEnabledAsFlow()` mengembalikan `Flow<Boolean>` menggunakan `callbackFlow`, yang memungkinkan pengamatan

perubahan nilai dari `SharedPreferences`; di dalamnya dibuat listener untuk mendeteksi perubahan pada kunci `KEY_DARK_MODE` dan mengirim nilai terbaru melalui `trySend()`, lalu listener tersebut didaftarkan ke `prefs`, nilai awal dikirim segera setelah fungsi dipanggil, dan `awaitClose` digunakan untuk mencabut listener saat aliran tidak lagi digunakan, memastikan pembersihan sumber daya yang aman.

Pada baris [28–30], fungsi `isDarkModeEnabled()` mengembalikan nilai Boolean yang diambil dari `SharedPreferences` dengan kunci `KEY_DARK_MODE`, dan akan menghasilkan `false` secara default jika belum ada nilai yang disimpan, berfungsi untuk mengecek apakah mode gelap diaktifkan oleh pengguna.

Pada baris [32–34], fungsi `setDarkModeEnabled(isEnabled: Boolean)` digunakan untuk menyimpan nilai mode gelap ke dalam `SharedPreferences` dengan memanfaatkan ekstensi `edit` agar lebih ringkas, di mana nilai `isEnabled` disimpan menggunakan kunci `KEY_DARK_MODE` untuk mencatat apakah mode gelap diaktifkan atau tidak.

`model/network/MovieApi.kt`

Pada baris [1–4], file ditempatkan dalam package `model.network` yang menunjukkan bahwa isinya berkaitan dengan model data dari jaringan, lalu mengimpor anotasi `@SerializedName` dan `@Serializable` dari Kotlin Serialization yang digunakan untuk memetakan dan mengatur serialisasi/deserialisasi objek data dari dan ke format JSON saat berkomunikasi dengan API.

Pada baris [6–14], dideklarasikan data class `MovieApi` yang ditandai dengan anotasi `@Serializable` agar dapat diserialisasi dan dideserialisasi secara otomatis saat komunikasi dengan API, dengan properti seperti `id`, `title`, `posterPath`, `releaseDate`, `overview`, dan `voteAverage`, di mana beberapa properti menggunakan anotasi

@SerializedName untuk memetakan nama field JSON (poster_path, release_date, vote_average) ke nama variabel Kotlin yang sesuai.

model/network/MovieResponse.kt

Pada baris [1–4], file ditempatkan dalam package model.network yang menandakan bahwa isinya merupakan model data dari jaringan (network), lalu mengimpor anotasi @SerializedName dan @Serializable dari Kotlin Serialization untuk memungkinkan objek data disesuaikan dengan format JSON saat proses serialisasi dan deserialisasi data dari API.

Pada baris [6–12], dideklarasikan data class MovieResponse yang ditandai dengan anotasi @Serializable untuk memungkinkan serialisasi dan deserialisasi data dari API, dengan properti page sebagai nomor halaman saat ini, results sebagai daftar objek MovieApi, serta totalPages dan totalResults yang masing-masing dipetakan dari nama field JSON "total_pages" dan "total_results" menggunakan anotasi @SerializedName.

model/ListMovie.kt

Pada baris [1–10], file ditempatkan dalam package model yang menunjukkan bahwa class ini merupakan bagian dari model data dalam aplikasi, lalu dideklarasikan data class ListMovie yang merepresentasikan struktur data film untuk digunakan di UI, dengan properti id, title, imageUrl, releaseDate, description, dan voteAverage yang masing-masing menyimpan informasi dasar mengenai sebuah film dalam bentuk yang lebih ringan dan terformat untuk ditampilkan.

network/RetrofitClient.kt

Pada baris [1–11], file ditempatkan dalam package network yang menunjukkan bahwa isinya berkaitan dengan konfigurasi jaringan, lalu mengimpor Constants untuk mengambil nilai konstan seperti base URL atau API key, converter dari Kotlin Serialization untuk Retrofit, objek Json untuk parsing JSON, serta komponen dari OkHttp seperti Interceptor, OkHttpClient, MediaType, dan HttpLoggingInterceptor,

dan terakhir Retrofit sebagai library utama untuk melakukan komunikasi HTTP dengan API.

Pada baris [13], dideklarasikan object `RetrofitClient` sebagai singleton, yaitu objek tunggal yang digunakan untuk mengelola dan menyediakan instance Retrofit secara global di seluruh aplikasi, sehingga konfigurasi client HTTP tidak perlu dibuat berulang kali.

Pada baris [15], dibuat objek json dari `Json` dengan konfigurasi `ignoreUnknownKeys = true`, yang berarti saat melakukan deserialisasi dari JSON, properti yang tidak dikenali (tidak ada dalam data class) akan diabaikan agar tidak menyebabkan error, sehingga aplikasi tetap stabil meskipun respons API mengandung field tambahan yang tidak dipakai.

Pada baris [17–19], dibuat `loggingInterceptor` menggunakan `HttpLoggingInterceptor` dan disetel levelnya ke `BODY`, yang berarti semua detail permintaan dan respons HTTP—including header, body, dan metadata—akan dicatat ke log, berguna untuk debugging dan memantau lalu lintas jaringan selama pengembangan.

Pada baris [21–28], dibuat `authInterceptor` sebagai `Interceptor` kustom yang secara otomatis menambahkan parameter `query api_key` dengan nilai dari `Constants.TMDB_API_KEY` ke setiap permintaan jaringan, dengan mengambil `originalRequest`, membangun ulang URL-nya menggunakan `newBuilder()`, lalu membuat `newRequest` dengan URL baru tersebut, dan akhirnya meneruskannya (`proceed`) ke jaringan — ini mempermudah otentikasi otomatis ke API tanpa perlu menambahkan API key secara manual di setiap endpoint.

Pada baris [30–33], dibuat instance `okHttpClient` menggunakan `OkHttpClient.Builder()` dengan menambahkan dua `interceptor` yaitu `authInterceptor` untuk menyisipkan API key secara otomatis ke setiap request, dan `loggingInterceptor` untuk mencatat detail permintaan dan respons HTTP, lalu semuanya dibangun menjadi

satu objek `OkHttpClient` yang akan digunakan oleh Retrofit untuk menjalankan komunikasi jaringan.

Pada baris [35–43], properti `tmdbApiService` dideklarasikan dengan anotasi `@OptIn(ExperimentalSerializationApi::class)` karena menggunakan fitur eksperimental dari Kotlin Serialization, dan dibuat secara lazy agar hanya diinisialisasi saat pertama kali diakses; di dalamnya dibuat instance Retrofit dengan `baseUrl` dari `Constants.TMDB_BASE_URL`, menggunakan `okHttpClient` yang sudah dikonfigurasi sebelumnya, serta menambahkan `converter factory` dari json untuk memproses konten bertipe "application/json", lalu membangun dan menghasilkan implementasi dari `TmdbApiService` yang siap digunakan untuk mengakses API TMDB.

`network/TmdbApiService.kt`

Pada baris [1–5], file ditempatkan dalam package `network` yang menunjukkan bahwa isinya berkaitan dengan komunikasi API, lalu mengimpor `MovieResponse` sebagai model respons dari server, serta anotasi `@GET` dan `@Query` dari Retrofit yang digunakan untuk mendefinisikan metode HTTP dan parameter query dalam interface layanan API.

Pada baris [7–13], dideklarasikan interface `TmdbApiService` yang merepresentasikan layanan API Retrofit, dengan satu fungsi `getPopularMovies()` yang ditandai dengan anotasi `@GET("movie/popular")` untuk mengambil data film populer dari endpoint tersebut, lalu menggunakan anotasi `@Query` untuk menyisipkan parameter `language` dan `page` dalam URL permintaan secara default, dan fungsi ini bersifat suspend agar dapat dipanggil dalam coroutine, serta mengembalikan objek `MovieResponse` sebagai hasil dari respons API.

`repository/MovieRepository.kt`

Pada baris [1–11], file ditempatkan dalam package repository yang menunjukkan bahwa fungsinya sebagai penghubung antara data dari jaringan dan database lokal, lalu mengimpor berbagai komponen seperti MovieDao dan MovieEntity dari layer lokal, MovieApi sebagai model data dari API, TmdbApiService sebagai interface untuk akses ke API TMDB, dan Resource sebagai wrapper status data (loading, success, error), serta Flow, flow, dan firstOrNull dari coroutine untuk pemrosesan data secara asinkron, dan Timber untuk logging yang membantu dalam pencatatan error atau debugging.

Pada baris [13–16], dideklarasikan kelas MovieRepository yang memiliki dua parameter konstruktor: apiService untuk melakukan permintaan data ke API TMDB, dan movieDao untuk berinteraksi dengan database lokal Room, sehingga repository ini bertugas mengelola logika pengambilan dan penyimpanan data dari kedua sumber tersebut.

Pada baris [17–20], fungsi getPopularMovies didefinisikan untuk mengembalikan aliran data (Flow) yang dibungkus dalam Resource agar statusnya dapat merepresentasikan loading, success, atau error, dengan parameter fetchFromRemote untuk menentukan apakah perlu mengambil data terbaru dari API; fungsi ini langsung memancarkan status Resource.Loading() sebagai indikator bahwa proses pengambilan data sedang berlangsung.

Pada baris [22–26], data film diambil terlebih dahulu dari database lokal menggunakan movieDao.getAllMovies().firstOrNull(), lalu jika hasilnya tidak kosong (isEmpty() bernilai false), data tersebut langsung dipancarkan sebagai Resource.Success, menandakan data berhasil dimuat dari cache, dan juga dicatat jumlah datanya ke log menggunakan Timber untuk keperluan debugging.

Pada baris [28], variabel shouldLoadFromApi digunakan untuk menentukan apakah perlu mengambil data dari API, dengan logika bahwa pengambilan dilakukan jika fetchFromRemote bernilai true (dipaksa ambil dari API) atau jika localMovies kosong/null, sehingga memastikan data selalu tersedia dan diperbarui jika dibutuhkan.

Pada baris [30–46], jika `shouldLoadFromApi` bernilai `true`, maka dilakukan proses pengambilan data film dari API dalam blok `try`, dimulai dengan logging proses melalui `Timber`, lalu memanggil `apiService.getPopularMovies()` untuk mendapatkan respons film populer yang hasilnya diubah menjadi `MovieEntity` melalui `map` dan fungsi ekstensi `toMovieEntity()`, setelah itu database lokal dikosongkan (`deleteAllMovies()`) dan diisi ulang dengan data baru (`insertMovies()`), kemudian log jumlah data yang berhasil di-cache, dan sebagai langkah akhir, data terbaru dari database kembali diambil untuk dipancarkan sebagai `Resource.Success`, atau jika gagal, dipancarkan sebagai `Resource.Error` dengan pesan kegagalan mengambil cache.

Pada baris [48–55], blok `catch` menangani jika terjadi `exception` selama proses pengambilan data dari API, dengan mencatat error melalui `Timber.e`, lalu memeriksa apakah `localMovies` kosong; jika iya, maka dipancarkan `Resource.Error` dengan pesan bahwa server tidak dapat dijangkau dan mengirimkan list kosong, sedangkan jika `localMovies` masih tersedia, maka tetap dipancarkan `Resource.Error` tetapi dengan data cache yang ada, sehingga pengguna tetap dapat melihat data meskipun update gagal.

Pada baris [56–61], blok `else` dijalankan jika `shouldLoadFromApi` bernilai `false` (artinya tidak mengambil data dari API), lalu dilakukan pengecekan kembali apakah `localMovies` kosong; jika iya, maka dipancarkan `Resource.Error` dengan pesan bahwa tidak ada data cache yang tersedia dan pengambilan dari remote juga tidak dilakukan, sehingga kondisi ini menangani skenario terburuk saat data tidak bisa dimuat dari manapun.

Pada baris [63–72], didefinisikan fungsi ekstensi `toMovieEntity()` yang mengubah objek `MovieApi` menjadi `MovieEntity`, dengan mengisi masing-masing properti dari data API ke entitas lokal, termasuk memberi nilai default "No Title" jika judul (`title`) bernilai `null`, sehingga fungsi ini memastikan data yang diterima dari jaringan dapat langsung disimpan ke database lokal `Room` dalam format yang sesuai.

`screen/DetailActivity`

Pada baris [1–21], file ini berada di package screen dan mengimpor berbagai komponen yang dibutuhkan untuk membuat tampilan layar detail menggunakan Jetpack Compose, termasuk `ComponentActivity`, fungsi layout seperti `Column`, `Box`, dan `Spacer`, komponen Material3 seperti `Text` dan `Surface`, fungsi scroll, pengaturan gaya teks (`TextStyle`, `FontWeight`, `sp`), serta library Coil (`AsyncImage`) untuk memuat gambar secara asinkron, dan juga `SettingsManager` untuk pengaturan preferensi, semua ini disiapkan dalam konteks penggunaan tema `MListComposeTheme`.

Pada baris [23–25], kelas `DetailActivity` diturunkan dari `ComponentActivity` dan mengoverride fungsi `onCreate`, yang digunakan untuk menginisialisasi aktivitas ketika pertama kali dibuat, serta memanggil `super.onCreate(savedInstanceState)` agar logika inisialisasi bawaan tetap dijalankan.

Pada baris [27–34], data intent seperti `title`, `imageUrl`, `releaseDate`, `description`, dan `voteAverage` diambil dari parameter yang dikirim dari aktivitas sebelumnya dengan nilai default jika null, lalu `SettingsManager` diinisialisasi untuk membaca preferensi dark mode dan menentukan apakah mode gelap aktif melalui `isDarkModeEnabled`.

Pada baris [36–41], `setContent` digunakan untuk menetapkan UI Compose dengan tema `MListComposeTheme` yang disesuaikan berdasarkan status dark mode, lalu sebuah `Surface` dibuat untuk mengisi seluruh layar (`fillMaxSize`) dan menerapkan warna latar belakang sesuai skema warna tema aktif.

Pada baris [42–47], digunakan `Column` sebagai layout vertikal yang mengisi lebar penuh layar (`fillMaxWidth()`), diberi padding 16dp di sekelilingnya, dan diatur agar bisa discroll secara vertikal dengan `verticalScroll(rememberScrollState())`, memungkinkan konten panjang tetap dapat ditampilkan dengan baik.

Pada baris [48–57], komponen `AsyncImage` digunakan untuk menampilkan poster film yang dimuat dari URL `imageUrl`, dengan deskripsi gambar berupa judul `title`. Gambar diatur memiliki tinggi 300dp, lebar memenuhi layar, dan ditempatkan di tengah secara horizontal. Skala konten diatur ke `Fit` agar seluruh gambar tampil tanpa terpotong.

Setelah gambar, Spacer ditambahkan untuk memberi jarak vertikal sebesar 16dp sebelum elemen UI berikutnya.

Pada baris [59–99], beberapa komponen Text digunakan untuk menampilkan detail informasi film. Pertama, judul film (title) ditampilkan dengan ukuran huruf besar (36sp), tebal (Bold), dan warna mengikuti tema (onBackground). Kemudian, tanggal rilis (releaseDate) dan rating film (voteAverage) ditampilkan dengan ukuran font sedang (18sp), serta diberi padding bawah untuk pemisahan visual. Setelah itu, label "Plot :" dicetak dengan ukuran 20sp dan huruf tebal sebagai penanda deskripsi film. Spacer dengan tinggi 8dp ditambahkan untuk memberi jarak sebelum deskripsi (overview), yang ditampilkan dalam font 16sp dan warna tema. Semua teks mengikuti skema warna dari tema agar responsif terhadap mode gelap/terang.

screen/MovieListScreen.kt

Pada baris [1–53], sejumlah import seperti LazyColumn, AsyncImage, Scaffold, TopAppBar, MovieViewModel, Resource, dan Intent digunakan untuk membangun tampilan daftar film, memuat gambar, menangani navigasi, serta menghubungkan data dari ViewModel ke UI dengan komponen Material 3.

Pada baris [55–60], fungsi @Composable bernama MovieListScreen dideklarasikan dengan anotasi @OptIn(ExperimentalMaterial3Api::class, ExperimentalMaterial3Api::class) untuk mengaktifkan fitur eksperimental dari Material 3, dan menerima parameter viewModel bertipe MovieViewModel serta navController untuk navigasi antar layar.

Pada baris [61–64], beberapa state Compose diobservasi: moviesResource memantau daftar film dari viewModel.movies, navigateToUrl dan navigateToDetail memantau event navigasi dari ViewModel, semuanya menggunakan collectAsState() untuk reaktifitas UI, serta context diambil dari LocalContext.current untuk akses konteks Android saat ini.

Pada baris [66–70], jika `navigateToUrl` tidak null, kode membuat `Intent` untuk membuka URL di browser menggunakan `Intent.ACTION_VIEW` dan `toUri()`, lalu memulai aktivitas tersebut melalui `context.startActivity(intent)`, setelahnya memanggil `viewModel.onUrlNavigated()` untuk menandai bahwa navigasi URL telah diproses agar event tidak di-trigger ulang.

Pada baris [72–83], jika `navigateToDetail` tidak null, kode membuat `Intent` untuk berpindah ke `DetailActivity` dengan menyisipkan data film seperti `id`, `title`, `imageUrl`, `releaseDate`, `description`, dan `voteAverage` menggunakan `putExtra()`. Setelah itu, `intent` dijalankan dengan `context.startActivity(intent)`, dan `viewModel.onDetailNavigated()` dipanggil untuk menandai bahwa navigasi ke detail telah selesai agar event tidak ter-trigger ulang.

Pada baris [85–122], `Scaffold` digunakan untuk membuat layout utama dengan `TopAppBar` berjudul "Film Populer" dan tombol pengaturan yang menavigasi ke layar "settings" saat ditekan. Warna latar dan teks mengikuti skema tema. Di dalam konten `Scaffold`, sebuah `Column` dengan padding responsif dari `Scaffold` menampilkan UI berdasarkan status `moviesResource`: saat status `Loading`, ditampilkan `CircularProgressIndicator` dan teks "Loading movies..." di tengah layar dengan warna putih untuk memberi tahu pengguna bahwa data film sedang dimuat.

Pada baris [123–137], ketika `moviesResource` berstatus `Success`, data film yang berhasil diambil (`movieList`) ditampilkan menggunakan `LazyColumn` dengan padding 16dp dan jarak antar item 16dp. Setiap item dalam daftar di-render dengan komponen `MovieItems` yang menerima data film, serta dua callback untuk aksi klik: membuka detail film (`onDetailClicked`) dan membuka URL TMDb (`onTmdbClicked`), keduanya memanggil fungsi di `viewModel` sesuai aksi pengguna.

Pada baris [138–178], saat `moviesResource` berstatus `Error`, pesan error disimpan dalam `errorMessage` dan data cache dalam `cachedData`. Jika data cache tersedia, daftar film tetap ditampilkan menggunakan `LazyColumn` dengan padding dan jarak antar item, dan di atas daftar ditampilkan pesan error berwarna merah, keterangan bahwa

data cache sedang ditampilkan berwarna abu-abu, serta tombol "Retry" yang memicu pengambilan ulang data dari API lewat `viewModel.fetchMovies(true)`. Setiap item film di-render dengan `MovieItems` dan callback untuk membuka detail atau URL TMDb seperti sebelumnya.

Pada baris [179–190], jika status error terjadi dan tidak ada data cache, maka ditampilkan tampilan penuh yang berisi pesan error berwarna merah dan tombol "Retry" berwarna putih di tengah layar, yang ketika ditekan akan memicu pengambilan ulang data film melalui `viewModel.fetchMovies(true)`.

Pada baris [198–203], didefinisikan fungsi composable `MovieItems` yang menerima parameter `movie` bertipe `ListMovie`, serta dua callback yaitu `onDetailClicked` tanpa argumen yang dipanggil saat item detail diklik, dan `onTmdbClicked` dengan argumen `String` yang dipanggil saat tautan TMDb diklik. Fungsi ini bertugas menampilkan item film dalam daftar dengan interaksi yang sesuai.

Pada baris [204–298], komponen `MovieItems` membangun kartu (`Card`) dengan sudut membulat dan elevasi ringan, yang berisi tata letak `Row` untuk menampilkan gambar film (`AsyncImage`) di sisi kiri dengan ukuran tetap dan sudut membulat, serta kolom di kanan yang berisi judul film (besar dan tebal), tanggal rilis (diambil dari substring sebelum tanda "-" atau "N/A"), dan ringkasan plot yang dipotong hingga 100 karakter dengan tanda ellipsis. Di bagian bawah, ada dua tombol: satu untuk membuka URL TMDb berdasarkan ID film yang memanggil `onTmdbClicked` dengan URL lengkap, dan satu lagi untuk melihat detail yang memanggil `onDetailClicked`. Semua elemen teks dan tombol menggunakan warna dan gaya yang konsisten dengan tema aplikasi agar responsif terhadap mode gelap/terang.

`screen/SettingScreen.kt`

Pada baris [1–10], kode mengimpor paket dan library yang diperlukan untuk membuat UI dengan Jetpack Compose, termasuk layout dasar (`Column`, `Row`, `Spacer`, `Modifier`), ikon `ArrowBack` untuk navigasi, komponen Material3 seperti `TopAppBar`, `Button`, dan

Text, serta fungsi pendukung Compose seperti `@Composable` dan state management (`collectAsState`, `getValue`). Selain itu, `NavController` diimpor untuk mengelola navigasi antar layar, dan `MovieViewModel` untuk mengakses data film dari layer `ViewModel`.

Pada baris [16–21], fungsi composable `SettingsScreen` didefinisikan dengan anotasi `@OptIn` untuk menggunakan API eksperimental `Material3`, menerima `NavController` untuk navigasi layar dan `viewModel` untuk mengakses data atau logika terkait film, sebagai parameter input.

Pada baris [22–79], state `isDarkMode` diambil dari `viewModel` menggunakan `collectAsState` untuk memantau status mode gelap secara reaktif, lalu `Scaffold` digunakan untuk membangun tampilan dengan `TopAppBar` berjudul "Settings" yang memiliki tombol navigasi kembali; di dalam konten utama terdapat `Column` yang berisi `Row` dengan teks label "Dark Mode" dan `Switch` yang mencerminkan status `isDarkMode` serta memungkinkan pengguna mengubah mode gelap yang kemudian diteruskan ke `viewModel.setDarkMode` untuk memperbarui pengaturan, dengan warna `switch` yang menyesuaikan tema `Material3` agar konsisten dengan mode terang atau gelap.

`utils/Constants.kt`

Pada baris [1–8], didefinisikan objek `Constants` yang berisi konstanta API untuk `TMDB`, termasuk `TMDB_API_KEY` untuk autentikasi, URL dasar API (`TMDB_BASE_URL`), URL dasar untuk mengambil gambar film (`TMDB_IMAGE_BASE_URL`), dan URL dasar untuk mengakses halaman web detail film di `TMDB` (`TMDB_MOVIE_WEB_BASE_URL`).

`utils/resource.kt`

Pada baris [1–7], dibuat kelas sealed `Resource` generik untuk merepresentasikan status data yang diambil, dengan tiga subclass: `Success` untuk data berhasil dengan isi data,

Error untuk kondisi gagal dengan pesan error dan opsional data, serta Loading untuk status pemuatan yang bisa membawa data sementara.

`viewmodel/MovieViewModel.kt`

Pada baris [1–17], dilakukan import berbagai kelas dan pustaka yang diperlukan dalam `MovieViewModel`, termasuk kelas `ViewModel` dan `viewModelScope` dari `Android Lifecycle`, kelas dan utilitas terkait flow dari `Kotlin Coroutines`, model data `ListMovie`, repository `MovieRepository`, manajer preferensi `SettingsManager`, konstanta aplikasi, kelas `Resource` untuk pengelolaan status data, serta pustaka logging `Timber` untuk pencatatan log.

Pada baris [19–22], dideklarasikan kelas `MovieViewModel` yang merupakan subclass dari `ViewModel`. Kelas ini menerima dua dependency melalui konstruktor: repository bertipe `MovieRepository` untuk mengelola data film, dan `settingsManager` bertipe `SettingsManager` untuk mengelola preferensi aplikasi seperti pengaturan tema.

Pada baris [23–24], dibuat variabel `private` bernama `_movies` dengan tipe `MutableStateFlow` yang menyimpan objek `Resource` berisi daftar `ListMovie`, diinisialisasi dengan status `Resource.Loading()` untuk menandakan data sedang dimuat; lalu variabel publik `movies` bertipe `StateFlow` yang hanya dapat dibaca, mengambil nilai dari `_movies` menggunakan fungsi `asStateFlow()` agar UI dapat mengamati perubahan data tanpa bisa memodifikasinya langsung.

Pada baris [26–27], dibuat variabel `private` `_navigateToUrl` berupa `MutableStateFlow` yang menyimpan URL bertipe `String?` dan diinisialisasi `null` (belum ada URL); lalu variabel publik `navigateToUrl` bertipe `StateFlow<String?>` yang hanya dapat dibaca, mengambil nilai dari `_navigateToUrl` dengan `asStateFlow()` agar UI bisa mengamati perubahan URL untuk navigasi tanpa bisa mengubahnya langsung.

Pada baris [29–30], dibuat variabel `private` `_navigateToDetail` berupa `MutableStateFlow` yang menyimpan objek `ListMovie?` dengan nilai awal `null`, serta

variabel publik `navigateToDetail` berupa `StateFlow<ListMovie?>` yang hanya bisa dibaca dan meng-expose data dari `_navigateToDetail` menggunakan `asStateFlow()`, memungkinkan UI mengamati perubahan detail film untuk navigasi ke halaman detail tanpa dapat memodifikasi langsung.

Pada baris [32–38], `isDarkModeEnabled` adalah `StateFlow<Boolean>` yang mengamati status mode gelap dari `settingsManager` secara reaktif; data diambil dari `getDarkModeEnabledAsFlow()`, di-filter agar hanya update unik dengan `distinctUntilChanged()`, dan diubah menjadi `StateFlow` yang aktif selama 5 detik setelah subscriber terakhir berhenti dengan `stateIn()` menggunakan `viewModelScope`, serta nilai awal diambil dari `settingsManager.isDarkModeEnabled()`.

Pada baris [40–42], blok `init` berfungsi sebagai inisialisasi awal `ViewModel` dengan langsung memanggil fungsi `fetchMovies()` untuk mulai mengambil data film ketika `ViewModel` dibuat.

Pada baris [44–75], `fetchMovies` adalah fungsi di dalam `ViewModel` yang memanggil `repository.getPopularMovies(fetchFromRemote)` dalam `coroutine viewModelScope.launch`, lalu mengumpulkan hasilnya secara reaktif dengan `collect`; hasil `Resource` kemudian dipetakan ke `_movies.value` sesuai dengan statusnya: jika `Success`, data film diubah ke bentuk `ListMovie` dan dikemas kembali dalam `Resource.Success`; jika `Error`, pesan error disertakan dan data juga dipetakan ke `ListMovie` bila tersedia; dan jika `Loading`, nilai `Resource.Loading()` langsung diberikan ke `_movies` untuk merepresentasikan status pemuatan.

Pada baris [77–79], `setDarkMode` adalah fungsi sederhana yang menerima parameter `isEnabled` bertipe `Boolean` untuk mengatur status mode gelap; fungsi ini meneruskan nilai tersebut ke `settingsManager.setDarkModeEnabled(isEnabled)` sehingga preferensi mode gelap disimpan secara persisten melalui `settingsManager`.

Pada baris [81–84], `onImdbClicked` adalah fungsi yang dipanggil saat pengguna mengklik tautan IMDb; fungsi ini mencatat URL yang diklik menggunakan `Timber.d`

untuk keperluan debugging, lalu menetapkan nilai URL tersebut ke `_navigateToUrl.value` agar dapat diamati oleh UI dan memicu navigasi ke tautan tersebut secara reaktif.

Pada baris [86–89], `onDetailClicked` adalah fungsi yang dijalankan saat pengguna memilih salah satu film untuk melihat detailnya; fungsi ini mencatat judul film yang diklik menggunakan `Timber.d` sebagai log debug, lalu menetapkan objek movie ke `_navigateToDetail.value` agar dapat diamati oleh UI dan memicu navigasi ke halaman detail film secara reaktif.

Pada baris [91–93], `onUrlNavigated` adalah fungsi yang dipanggil setelah navigasi ke URL selesai dilakukan; fungsi ini menetapkan nilai `_navigateToUrl.value` menjadi null untuk mereset event navigasi, sehingga tidak terjadi navigasi ulang saat nilai `LiveData` atau `StateFlow` diamati kembali oleh UI.

Pada baris [95–97], `onDetailNavigated` adalah fungsi yang dijalankan setelah navigasi ke halaman detail selesai; fungsi ini menetapkan `_navigateToDetail.value` menjadi null untuk mereset event navigasi, sehingga mencegah pemicu ulang navigasi saat observer UI membaca nilai tersebut kembali.

`viewmodel/ModelViewModelFactory.kt`

Pada baris [1–6], paket `com.example.mlistcompose.viewmodel` dideklarasikan sebagai lokasi kelas `ViewModel`; kemudian beberapa impor dilakukan: `ViewModel` dan `ViewModelProvider` dari `androidx.lifecycle` untuk mendukung arsitektur MVVM; `SettingsManager` dari `data.preferences` untuk mengelola preferensi aplikasi seperti mode gelap; serta `MovieRepository` dari `repository` yang digunakan untuk mengakses data film dari sumber data yang disediakan.

Pada baris [8–19], `MovieViewModelFactory` adalah kelas turunan dari `ViewModelProvider.Factory` yang digunakan untuk membuat instance `MovieViewModel` dengan dependency injection; konstruktor menerima dua parameter

yaitu repository dan settingsManager yang akan diteruskan ke MovieViewModel; dalam override fungsi create, dilakukan pengecekan apakah modelClass adalah turunan dari MovieViewModel, dan jika ya maka objek MovieViewModel dikembalikan setelah dilakukan cast dengan penekanan peringatan `@SuppressWarnings("UNCHECKED_CAST");` jika tidak cocok, maka `IllegalArgumentException` dilempar dengan pesan bahwa kelas ViewModel tidak dikenal.

MListComposeApplication.kt

Pada baris [1–9], paket `com.example.mlistcompose` dideklarasikan sebagai ruang lingkup utama aplikasi; beberapa impor dilakukan untuk mendukung inisialisasi dan dependency injection pada level aplikasi, termasuk `Application` dari Android sebagai superclass untuk membuat custom application class, `AppDatabase` untuk akses database lokal, `RetrofitClient` untuk komunikasi jaringan, `MovieRepository` sebagai pengelola logika data, `Timber` sebagai alat logging, `MovieViewModelFactory` untuk membuat instance ViewModel dengan dependensi, serta `SettingsManager` untuk pengelolaan preferensi pengguna seperti mode gelap.

Pada baris [11–16], `MListComposeApplication` adalah kelas turunan dari `Application` yang digunakan untuk mengelola inisialisasi global saat aplikasi pertama kali dijalankan; properti database diinisialisasi secara lazy untuk mendapatkan instance `AppDatabase`; `movieDao` diambil dari database sebagai akses objek DAO ke tabel film; `tmdbApiService` adalah service Retrofit yang disediakan oleh `RetrofitClient` untuk melakukan permintaan API TMDB; dan `settingsManager` diinisialisasi dengan konteks aplikasi untuk mengelola preferensi seperti mode gelap secara persisten.

Pada baris [18–20], `movieRepository` adalah properti publik bertipe `MovieRepository` yang diinisialisasi secara lazy; objek ini dibuat dengan menyuntikkan dependensi `tmdbApiService` untuk mengambil data dari API TMDB dan `movieDao` untuk akses database lokal, sehingga `movieRepository` dapat bertindak sebagai sumber data terpusat yang menggabungkan data dari jaringan dan lokal.

Pada baris [22–24], `movieViewModelFactory` adalah properti publik bertipe `MovieViewModelFactory` yang diinisialisasi secara lazy; objek ini dibuat dengan menyuntikkan `movieRepository` dan `settingsManager` sebagai dependensi, sehingga `movieViewModelFactory` dapat digunakan untuk menghasilkan instance `MovieViewModel` dengan konstruktor kustom yang memerlukan parameter tersebut, sesuai dengan pola dependency injection dalam arsitektur MVVM.

Pada baris [26–31], `onCreate` adalah fungsi override dari kelas `Application` yang dipanggil saat aplikasi pertama kali dimulai; di dalamnya dipanggil `super.onCreate()` untuk memastikan inisialisasi bawaan tetap dijalankan; kemudian dilakukan pengecekan terhadap `BuildConfig.DEBUG` untuk mengetahui apakah aplikasi sedang dalam mode debug, dan jika ya maka `Timber.plant(Timber.DebugTree())` dipanggil untuk menanam pohon debug Timber, memungkinkan pencatatan log secara detail selama pengembangan.

`MainActivity.kt`

Pada baris [1–20], paket `com.example.mlistcompose` dideklarasikan sebagai namespace utama aplikasi; berbagai impor disertakan untuk membangun tampilan utama aplikasi berbasis Jetpack Compose, termasuk `ComponentActivity` sebagai base activity, `setContent` untuk mengisi UI dengan komposisi, serta `viewModels` untuk inisialisasi `ViewModel`; elemen-elemen Compose seperti `Modifier`, `MaterialTheme`, `Surface`, `collectAsState`, dan `getValue` digunakan untuk pengaturan tema dan state UI; navigasi ditangani dengan `NavHost`, `composable`, dan `rememberNavController`; `MovieListScreen` dan `SettingsScreen` adalah tampilan yang dapat dinavigasi; `MovieViewModel` digunakan untuk mengelola logika data; `SettingsManager` dikelola untuk preferensi aplikasi; dan `MListComposeTheme` digunakan sebagai tema kustom aplikasi.

Pada baris [22–30], `MainActivity` adalah kelas turunan dari `ComponentActivity` yang berfungsi sebagai activity utama aplikasi; dalam fungsi `onCreate`, dipanggil `super.onCreate(savedInstanceState)` untuk menjalankan inisialisasi bawaan;

viewModel diinisialisasi menggunakan delegasi by viewModels dengan movieViewModelFactory yang diambil dari MListComposeApplication untuk memastikan ViewModel dibuat dengan dependensi yang benar; kemudian settingsManager diinisialisasi menggunakan konteks activity (this) untuk memungkinkan akses preferensi lokal seperti status mode gelap.

Pada baris [32–58], blok setContent digunakan untuk menyusun UI aplikasi dengan Jetpack Compose; isDarkModeEnabled diamati secara reaktif dari viewModel.isDarkModeEnabled menggunakan collectAsState, dengan nilai awal diambil dari settingsManager.isDarkModeEnabled(); MListComposeTheme menerapkan tema gelap atau terang berdasarkan nilai tersebut; di dalamnya, Surface digunakan sebagai wadah utama dengan ukuran penuh layar (fillMaxSize()) dan warna latar dari MaterialTheme; navController diinisialisasi untuk mengelola navigasi antar layar; NavHost didefinisikan dengan startDestination "movieList" dan dua composable yang mewakili rute "movieList" dan "settings"; masing-masing rute menampilkan MovieListScreen dan SettingsScreen dengan viewModel dan navController yang diteruskan sebagai parameter, memungkinkan interaksi dan navigasi antar tampilan.

Notes :

Untuk Cache Strategy Room yang digunakan adalah “Cache First – Network Second”

Aplikasi pertama-tama mencoba mengambil data dari cache lokal (Room). Jika data ditemukan, data tersebut segera ditampilkan untuk responsivitas yang cepat. Setelah itu, aplikasi melakukan permintaan ke API jaringan untuk mendapatkan data terbaru. Jika permintaan jaringan berhasil, data lokal di cache akan diperbarui dengan data baru dari API, dan UI kemudian akan menampilkan data yang paling mutakhir dari cache yang sudah diperbarui. Jika permintaan jaringan gagal (misalnya karena tidak ada koneksi internet), tetapi ada data yang sudah tersimpan di cache, aplikasi akan tetap menampilkan data dari cache tersebut sambil memberikan notifikasi error bahwa data tidak dapat diperbarui.

Tautan Git

Berikut adalah tautan untuk semua source code yang telah dibuat.

<https://github.com/Akmlsybn/Praktikum-Mobile>